MIVA OPEN UNIVERSITY · SCHOOL OF POSTGRADUATE STUDIES

CampusFix

A Full-Stack University Maintenance & Service Request System

MIT 8333 — Advanced Web Application Development (Virtual Lab)
Continuous Assessment, 2026/2027 Academic Session

Student name	[Your full name]
Matriculation no.	[Your matric number]
Programme	Master of Information Technology (MIT)
Course	MIT 8333 — Advanced Web Application Development
Technology stack	MERN — MongoDB, Express.js, React, Node.js
Live application	[https://your-app.onrender.com]
Source repository	[https://github.com/your-username/campusfix]
Date of submission	[Date]

This single PDF contains, as required by the submission guidelines: (1) the project report, (2) screenshots of the running application, and (3) the complete code base (Appendix A).

Contents

1. Introduction and Problem Statement . . . 3
2. System Objectives . . . 3
3. Requirement Analysis . . . 3
4. Frontend Technologies Used . . . 5
5. Backend Technologies Used . . . 5
6. Database and Relationships . . . 6
7. API Documentation . . . 7
8. Screenshots of Major Interfaces . . . 9
9. Testing Evidence . . . 25
10. Deployment Information . . . 27
11. Challenges Encountered and Solutions . . . 27
12. Conclusion . . . 28
Appendix A. Complete Code Base . . . 29

1. Introduction and Problem Statement

Universities depend on functioning infrastructure: lecture theatres need working projectors, hostels need water and electricity, and offices need internet access. At present the university receives maintenance complaints through phone calls, paper forms, WhatsApp messages and walk-in visits to the Works office. None of these channels produces a single, searchable record.

The consequences are predictable. Complaints are lost between channels, the same fault is reported several times, nobody can tell a student whether their request has been seen, and management cannot measure how quickly the maintenance unit responds. Because no record links a request to the officer who handled it, there is little accountability.

CampusFix is a full-stack web application that replaces these channels with one platform. Students and staff submit a structured request (category, location, priority, description and photo evidence) and receive a ticket number. Administrators triage and assign each request to a maintenance officer. Officers update progress from their own dashboard, and every change is time-stamped, attributed to a user, and pushed to the requester in real time.

2. System Objectives

- Provide a single online channel for submitting maintenance and service requests, with evidence photos.
- Give each of the three user groups — Student/Staff, Maintenance Officer and Administrator — a dashboard showing only what they need and are permitted to see.
- Let administrators assign requests to officers and re-assign them, with a full assignment history.
- Let officers record progress (in progress, completed) with comments the requester can read.
- Let requesters track each request through a visual progress bar and time-stamped activity timeline.
- Notify users in-app and in real time when something happens to their request.
- Give management reports: totals by status and category, officer workload, average time to fix, CSV export and a full audit trail.
- Protect the system with hashed passwords, JWT sessions, role-based access control, input validation and consistent error handling.
- Deploy the application online, connected to a cloud database.

3. Requirement Analysis

3.1 Users and roles

Role	Who	Main tasks
Student / Staff	Any student or staff member (self-registration)	Submit requests with photos; track status and timeline; cancel while pending; receive notifications

Role	Who	Main tasks
Maintenance Officer	Works-department technicians (created by admin)	View jobs assigned to them; start a job; mark it completed with notes
Administrator	Works-department managers	Manage users and roles; manage categories; assign/re-assign requests; override status; delete; reports, CSV export, audit log

3.2 Functional requirements

ID	Requirement	Implemented in
FR1	Register and log in; restore the session on reload	POST /api/auth/register, /login, GET /me
FR2	Role-based dashboards for the three roles	Dashboard.jsx, GET /api/requests/stats
FR3	Submit a service request with up to three evidence files	NewRequest.jsx, POST /api/requests (Multer)
FR4	Track a request: progress bar, details, timeline	RequestDetail.jsx, GET /api/requests/:id
FR5	Search, filter (status, category, priority) and paginate requests	Requests.jsx, GET /api/requests?q=&status;=&page;=
FR6	Admin assigns / re-assigns an officer	PATCH /api/requests/:id/assign
FR7	Officer updates progress; owner cancels; admin overrides	PATCH /api/requests/:id/status
FR8	Admin manages users (create, role, activate, delete)	Users.jsx, /api/users
FR9	Admin manages request categories	Categories.jsx, /api/categories
FR10	Notifications, live updates	/api/notifications, Socket.IO
FR11	Reports, CSV export, audit trail	Reports.jsx, /api/reports/summary, /export, /audit

3.3 Non-functional requirements

- **Security:** bcrypt (12 rounds) password hashing; signed JWTs that expire after one day; RBAC on every protected route; rate-limited login; Helmet security headers; file-type and size limits on uploads.
- **Usability:** clear navigation per role, inline form validation, success/error toast messages, responsive layout for phones.
- **Reliability:** central error handler returning a consistent JSON shape; a health-check endpoint that also reports database connectivity.
- **Maintainability:** layered structure (models, middleware, routes, utils; pages, components, context) and automated tests.

3.4 Advanced features implemented (eight of the nine listed)

Feature	How it is implemented
JWT authentication	jsonwebtoken signs a token at login; the protect middleware verifies it on every request
Role-based access control	authorize('admin') etc. on the API; ProtectedRoute and role-specific menus in React

Feature	How it is implemented
File/image upload	Multer disk storage, images/PDF only, 5 MB per file, max. 3 files; thumbnails shown on the tracking page
In-app notifications	Notification collection; bell icon with unread count, mark one/all as read
Search, filter, pagination	Regex search across ticket/title/location/description; status, category and priority filters; page/limit
Real-time status updates	Socket.IO with JWT handshake; private room per user; pages refresh automatically on request:updated
Audit trail / activity log	AuditLog written for logins, registrations, creates, updates, assignments, deletions and exports
API documentation	OpenAPI 3 specification served with Swagger UI at /api/docs
Data export	CSV export of requests honouring the current filters (admin)

4. Frontend Technologies Used

Technology	Purpose
React 19	Component-based user interface
Vite 8	Development server with API proxy and production build tool
React Router 7	Client-side routing and protected, role-aware routes
Axios	HTTP client; interceptors attach the JWT and handle expired sessions
Socket.IO client	Real-time notifications and live status refresh
React Context API	Global state for authentication, toasts and the socket connection
CSS3 (custom, responsive)	Design tokens, grid layouts, status colours, mobile sidebar
Vitest + React Testing Library	Component and form-validation tests

The frontend is organised into pages (one per screen), reusable components (Layout, RequestForm, RequestTable, StatusBadge, Pagination, ProtectedRoute) and context providers. All forms validate on the client before calling the API, and every API error is shown to the user in plain language through a shared `errorMessage()` helper.

5. Backend Technologies Used

Technology	Purpose
Node.js 22	JavaScript runtime
Express 5	REST API framework and static file server for the React build in production
Mongoose 9	MongoDB object modelling, schema validation, references and population
jsonwebtoken + bcryptjs	Stateless session tokens and password hashing
express-validator	Request body and query validation (returns 422 with per-field messages)
Multer	Multipart file upload for evidence photos
Socket.IO	WebSocket server for real-time events
Helmet, CORS, express-rate-limit	Security headers, cross-origin control, brute-force protection on login
swagger-ui-express	Interactive API documentation

Technology	Purpose
Jest + Supertest	Automated API endpoint tests

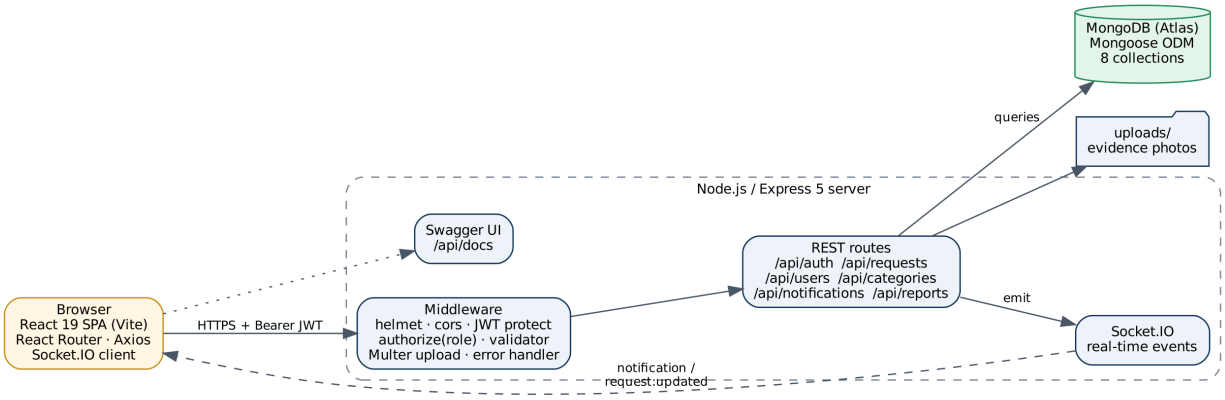


Figure 1. System architecture

6. Database and Relationships

The system uses **MongoDB**, a NoSQL document database, accessed through Mongoose. MongoDB Atlas (a managed cloud cluster) is used in deployment. All six required entities are separate collections, plus two supporting collections for notifications and the audit log.

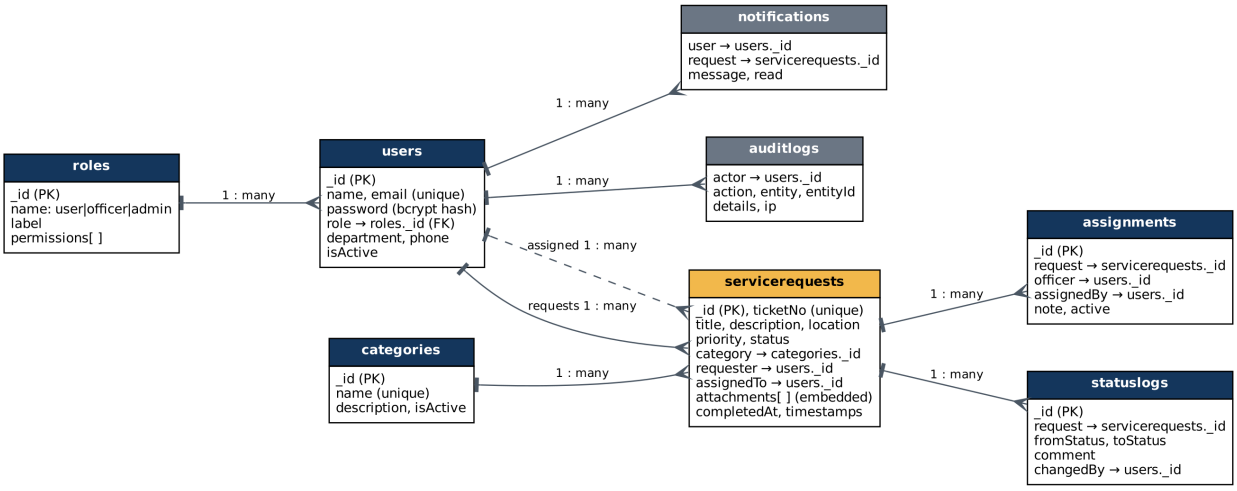


Figure 2. Entity–relationship diagram (crow's foot = "many")

Types of relationships supported

Although MongoDB is not a relational database, it supports relationships in two ways, and CampusFix uses both:

- **Referencing (normalised).** A document stores the `ObjectId` of a related document, like a foreign key, and Mongoose's `populate()` joins them at query time. This is used for all one-to-many relationships below.
- **Embedding (denormalised).** Related data that is only ever read with its parent is stored inside the parent document. A request's `attachments` array is embedded, so the

request and its photos are read in one query.

Relationship	Type	Implemented by
Role → Users	One-to-many	users.role references roles._id
User (requester) → Service requests	One-to-many	servicerequests.requester
User (officer) → Service requests	One-to-many	servicerequests.assignedTo
Category → Service requests	One-to-many	servicerequests.category
Service request → Assignments	One-to-many	assignments.request (history of re-assignments)
Service request → Status logs	One-to-many	statuslogs.request (the tracking timeline)
Users ↔ Service requests (via Assignments)	Many-to-many	An officer has many requests over time and a request may pass through several officers; Assignments is the linking collection
Service request → Attachments	One-to-few (embedded)	attachments[ ] inside the request document
User → Notifications / Audit logs	One-to-many	notifications.user, auditlogs.actor

Integrity rules are enforced in the API, since MongoDB has no foreign-key constraints: a category or user that is still referenced by requests cannot be deleted (409 Conflict — deactivate instead), and deleting a request also deletes its status logs and assignments. Unique indexes protect user e-mails, category names and ticket numbers.

7. API Documentation

The API follows REST conventions, exchanges JSON, and is documented interactively with Swagger UI at `/api/docs` (raw OpenAPI 3 JSON at `/api/docs.json`). Protected endpoints expect `Authorization: Bearer <token>`. Every response has the shape `{ success, message?, data?, pagination?, errors? }`.

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Create a Student/Staff account, returns JWT
POST	/api/auth/login	Public	Log in, returns JWT and user
GET	/api/auth/me	Any	Current user (session restore)
POST	/api/auth/logout	Any	Log out (recorded in audit log)
GET	/api/requests	Any (scoped)	List with q, status, category, priority, page, limit
GET	/api/requests/stats	Any (scoped)	Counts per status for dashboards
POST	/api/requests	User, Admin	Submit request (multipart, up to 3 files)
GET	/api/requests/:id	Owner, assignee, Admin	Detail with timeline and assignment history
PUT	/api/requests/:id	Owner (pending), Admin	Edit request details
PATCH	/api/requests/:id/status	Officer, Owner, Admin	Change status with a comment

Method	Endpoint	Access	Description
PATCH	/api/requests/:id/assign	Admin	Assign or re-assign an officer
DELETE	/api/requests/:id	Admin	Delete request and its history
GET/POST	/api/users	Admin	List (search, role filter, paging) / create user
PUT/DELETE	/api/users/:id	Admin	Update role, details, active flag / delete
GET	/api/users/officers	Admin	Active officers with open-job counts
GET/POST	/api/categories	Any / Admin	List / create categories
PUT/DELETE	/api/categories/:id	Admin	Update / delete category
GET	/api/notifications	Any	My latest notifications and unread count
PATCH	/api/notifications/:id/read, /read-all	Any	Mark as read
GET	/api/reports/summary	Admin	Totals by status, category, officer; average fix time
GET	/api/reports/export	Admin	CSV export (accepts list filters)
GET	/api/reports/audit	Admin	Paginated audit trail
GET	/api/health	Public	API and database health check

Status codes

200 OK · 201 Created · 400 Bad request (e.g. invalid id, closed request) · 401 Not authenticated / expired token · 403 Forbidden by role · 404 Not found · 409 Conflict (duplicate e-mail, record in use) · 422 Validation failed (with field list) · 429 Too many login attempts · 500 Server error.

Example

```
POST /api/auth/login
{ "email": "student@campusfix.edu", "password": "Password123" }

200 OK
{ "success": true, "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": { "_id": "...", "name": "Ada Obi", "role": { "name": "user", "label": "Student/Staff" } } }

PATCH /api/requests/6534.../status      (Authorization: Bearer <officer token>)
{ "status": "completed", "comment": "Socket replaced, wiring tested" }

422 Unprocessable Entity  (when the body is invalid)
{ "success": false, "message": "Validation failed",
  "errors": [ { "field": "status", "message": "Invalid status" } ] }
```



Figure 3. Swagger UI documentation at /api/docs

8. Screenshots of Major Interfaces

The screenshots below were captured from the production build (Express serving the compiled React app) connected to MongoDB, following one request — "Burnt wall socket sparking" — from submission to completion.

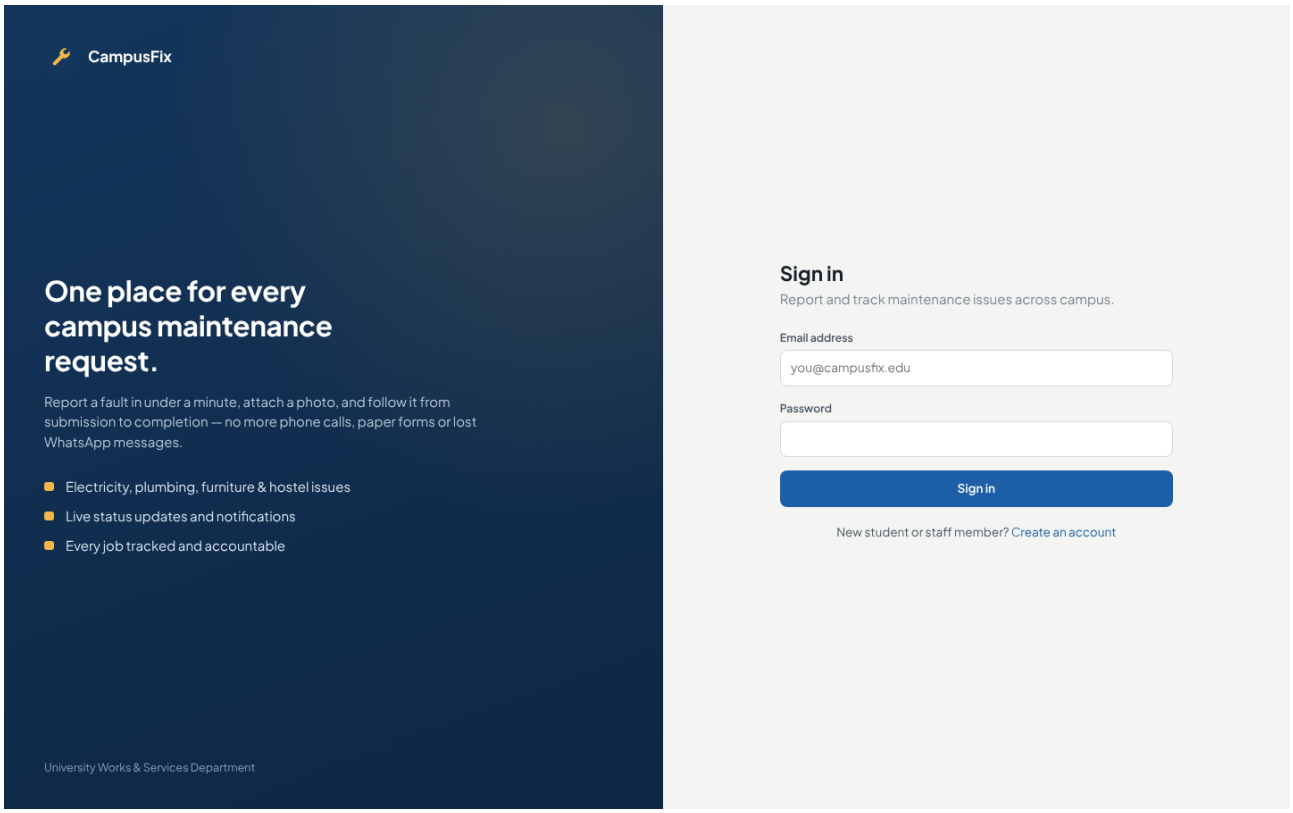


Figure 4. Login page

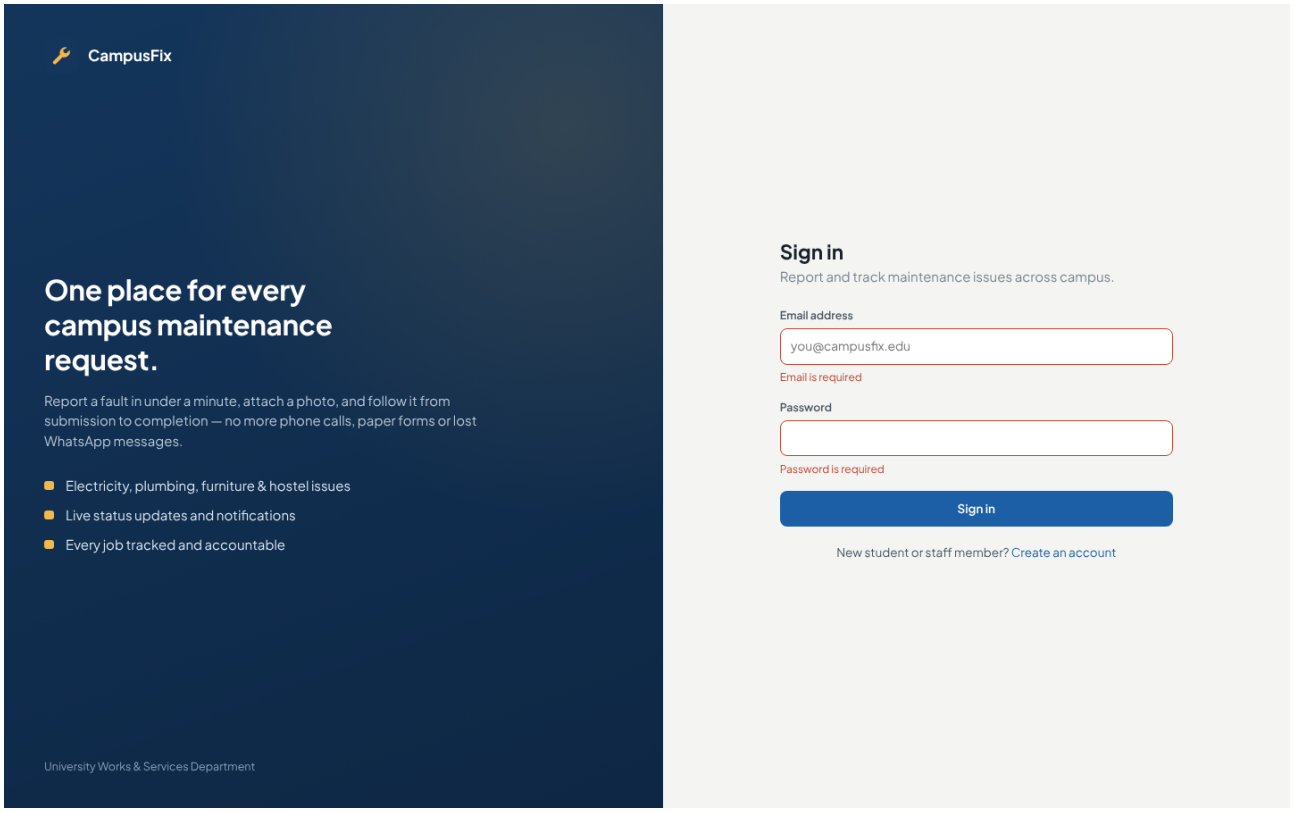


Figure 5. Client-side validation on the login form

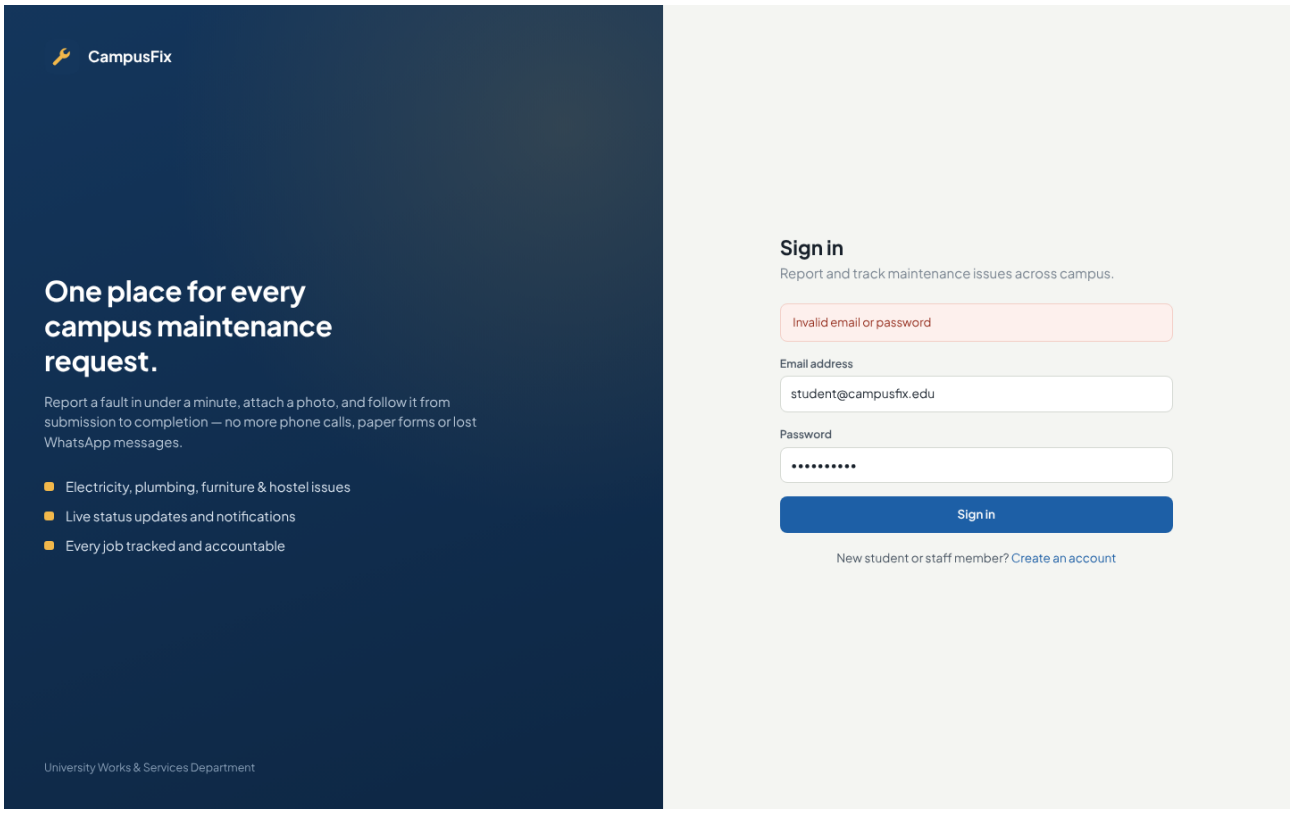


Figure 6. Server error message for a wrong password

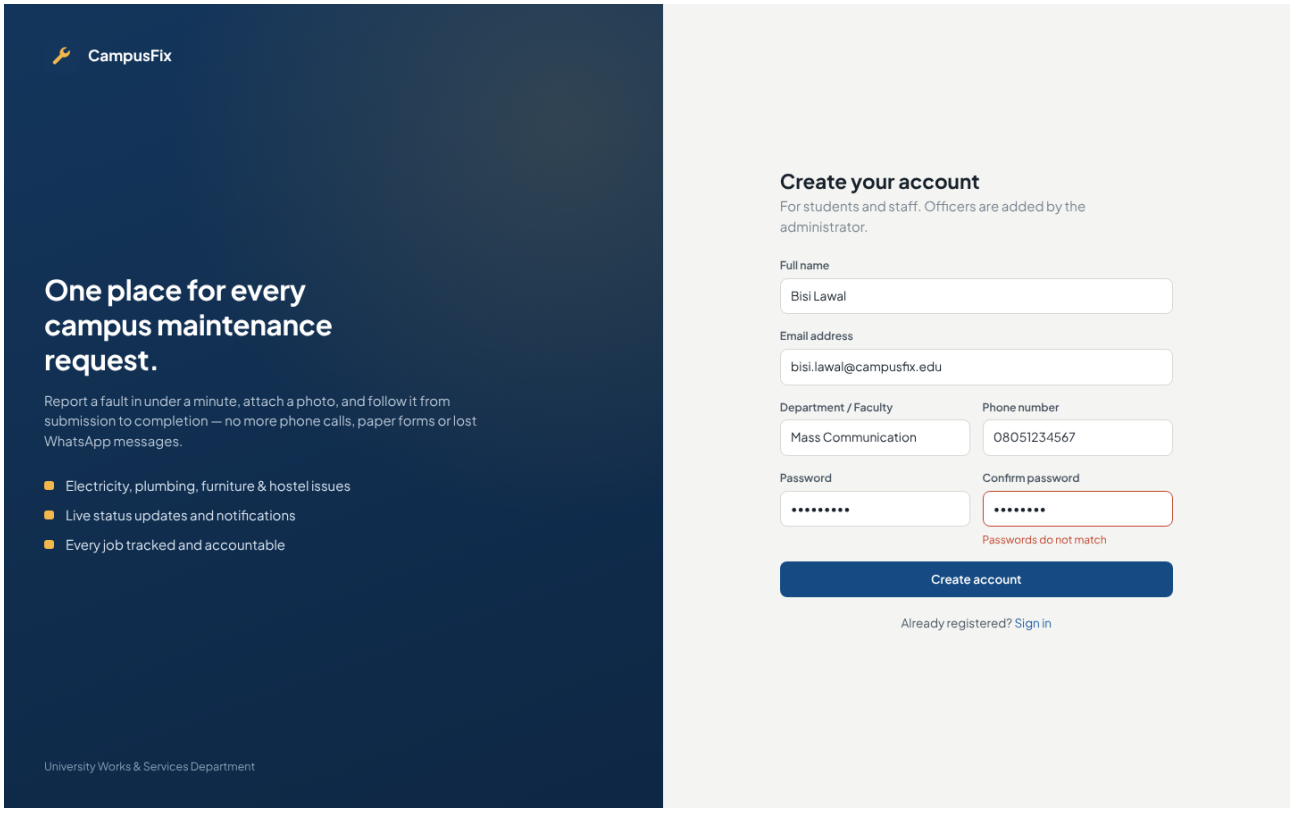


Figure 7. Registration page with password-match validation

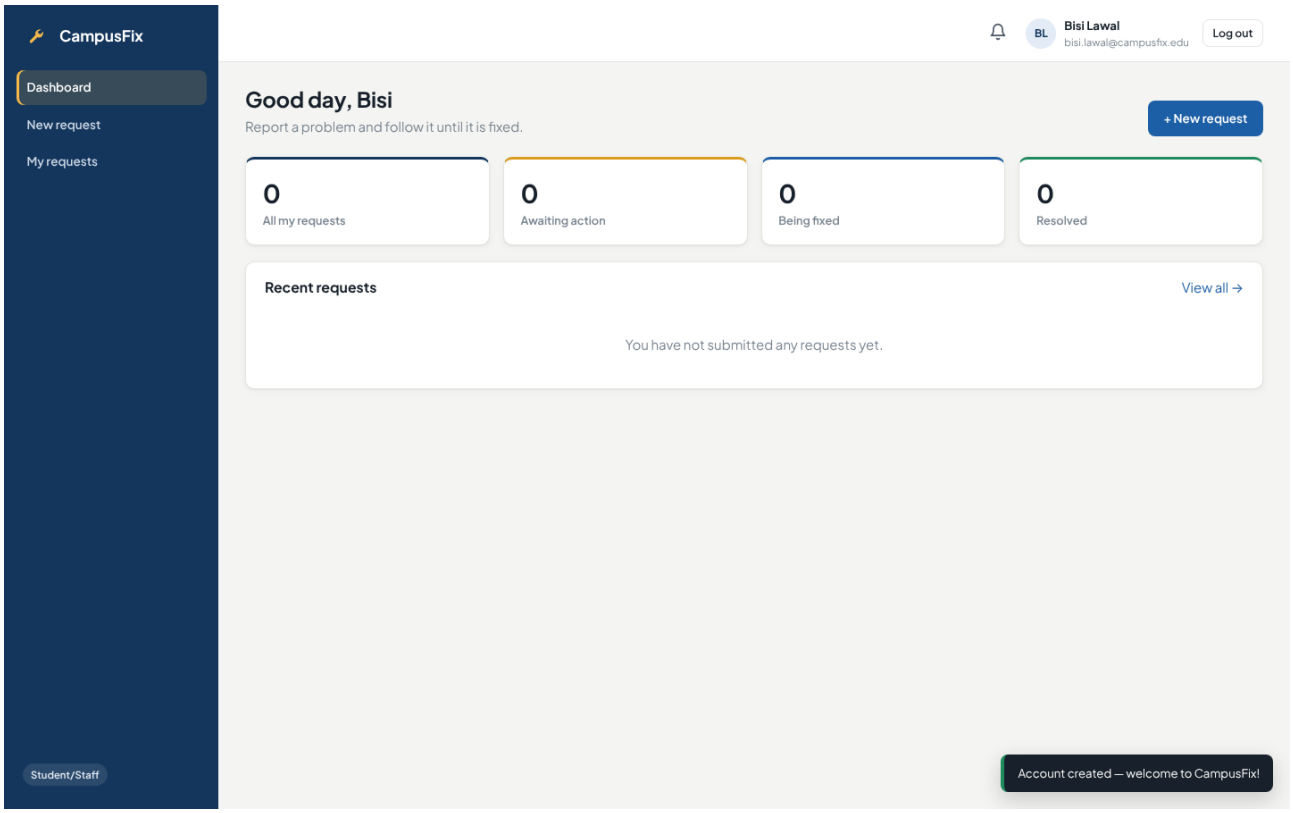


Figure 8. New account created — success toast and Student/Staff dashboard

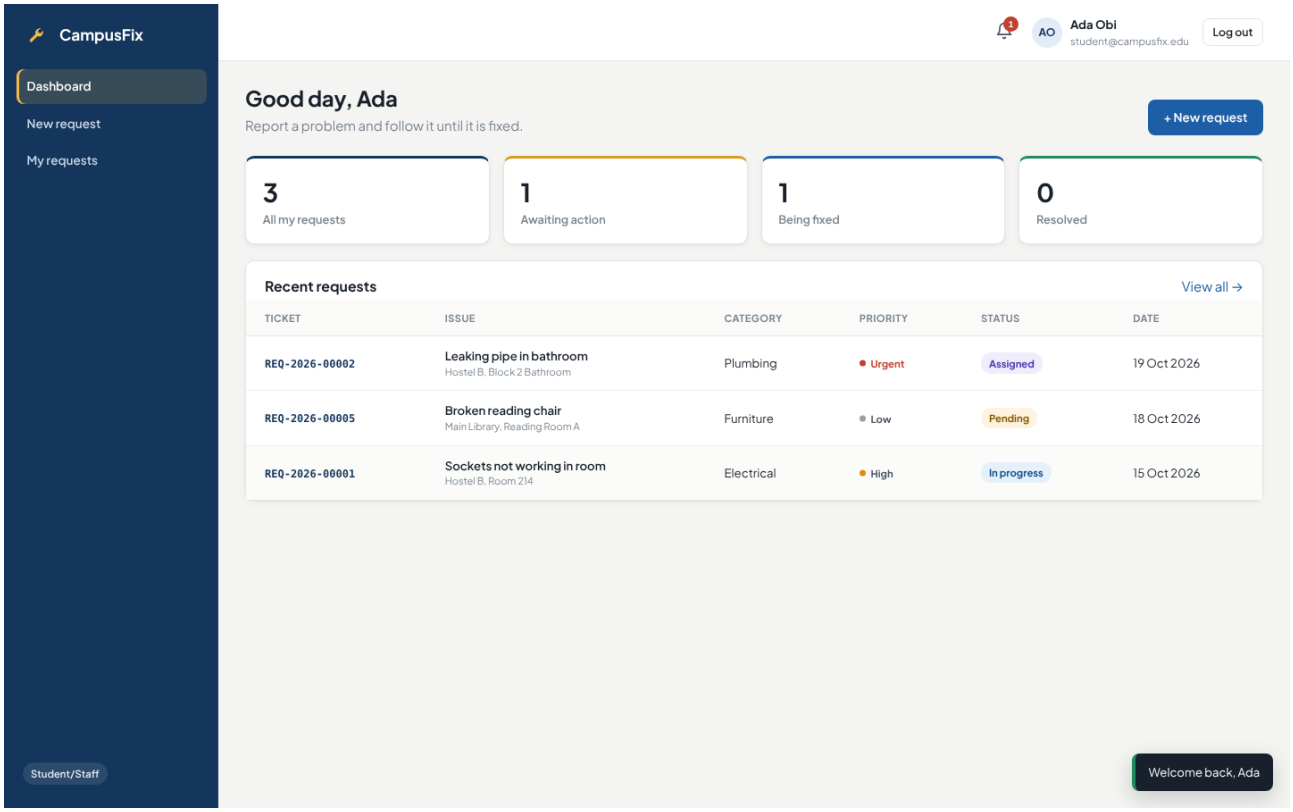


Figure 9. Student/Staff dashboard

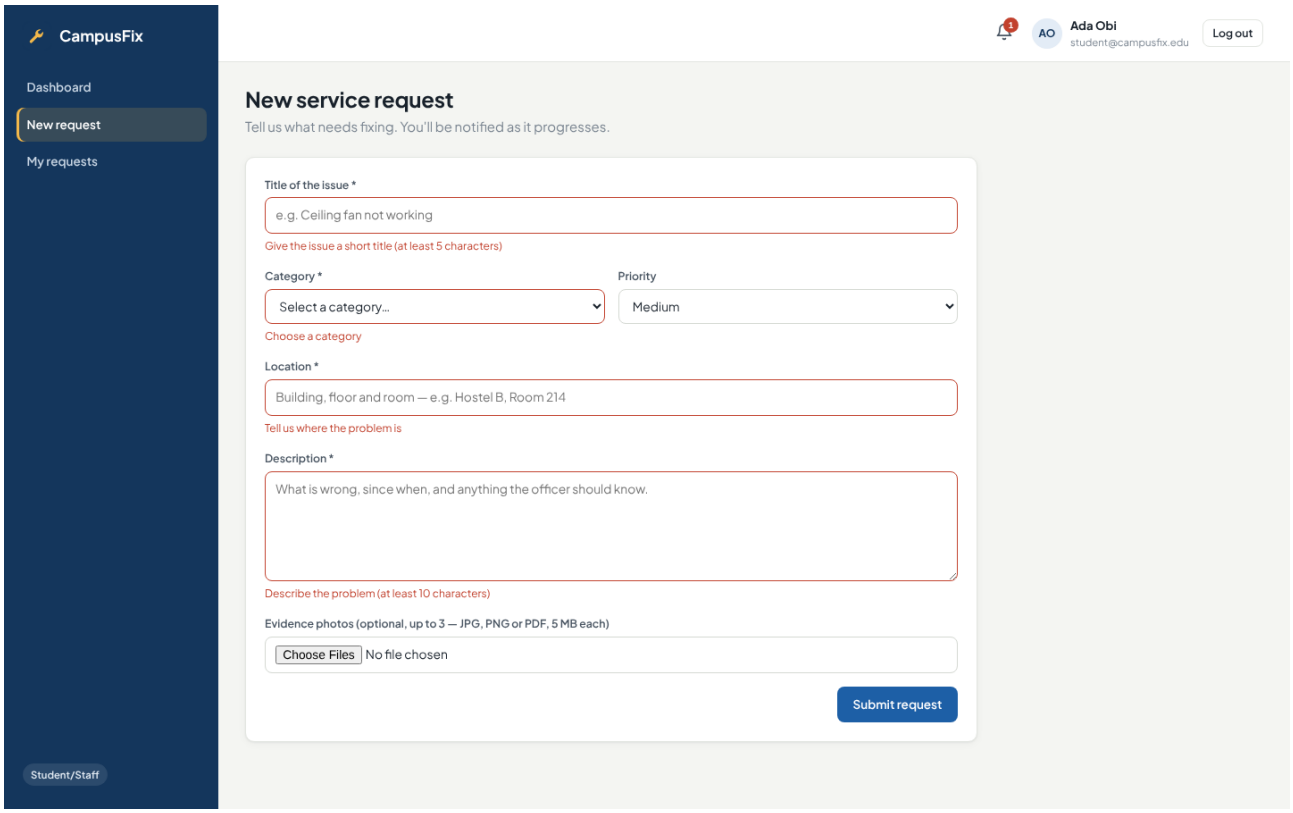


Figure 10. Service request form — validation messages

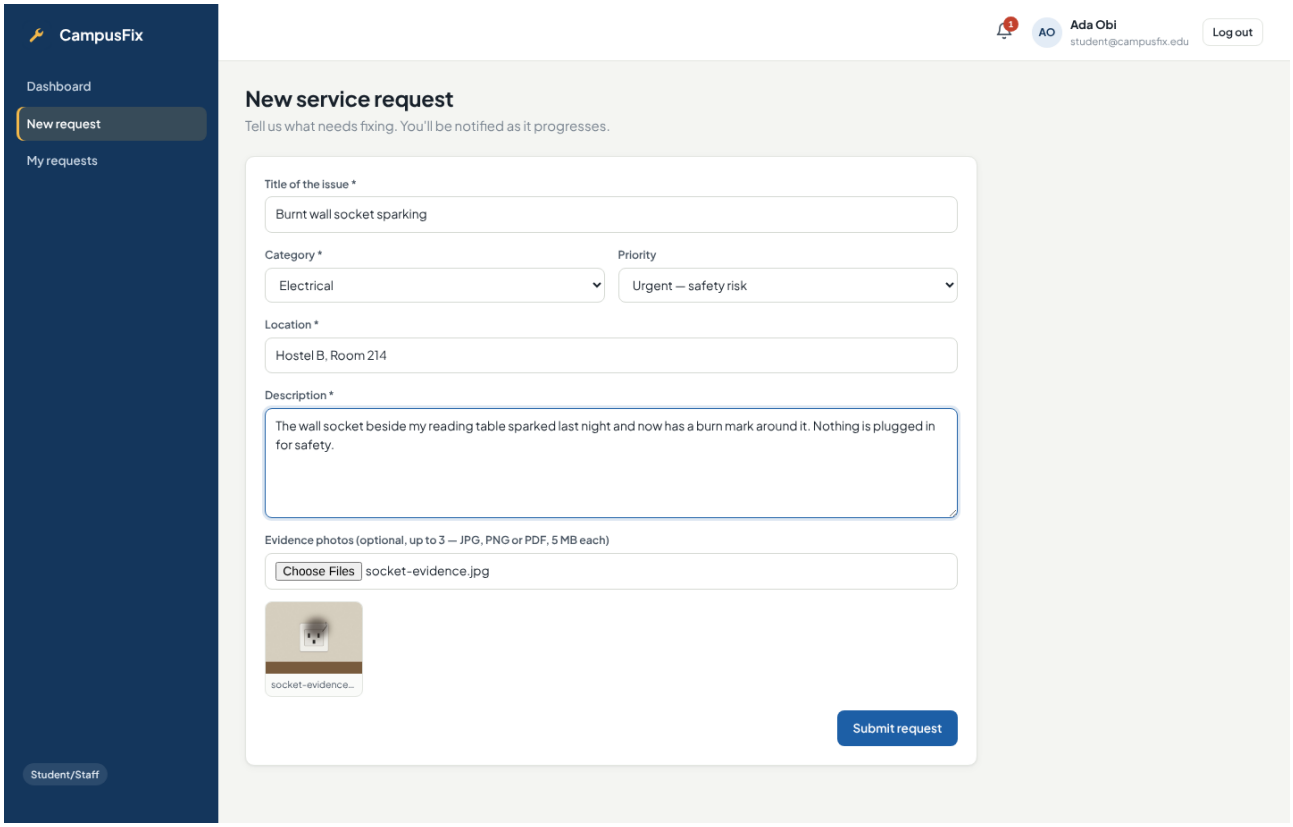


Figure 11. Service request form completed with an evidence photo

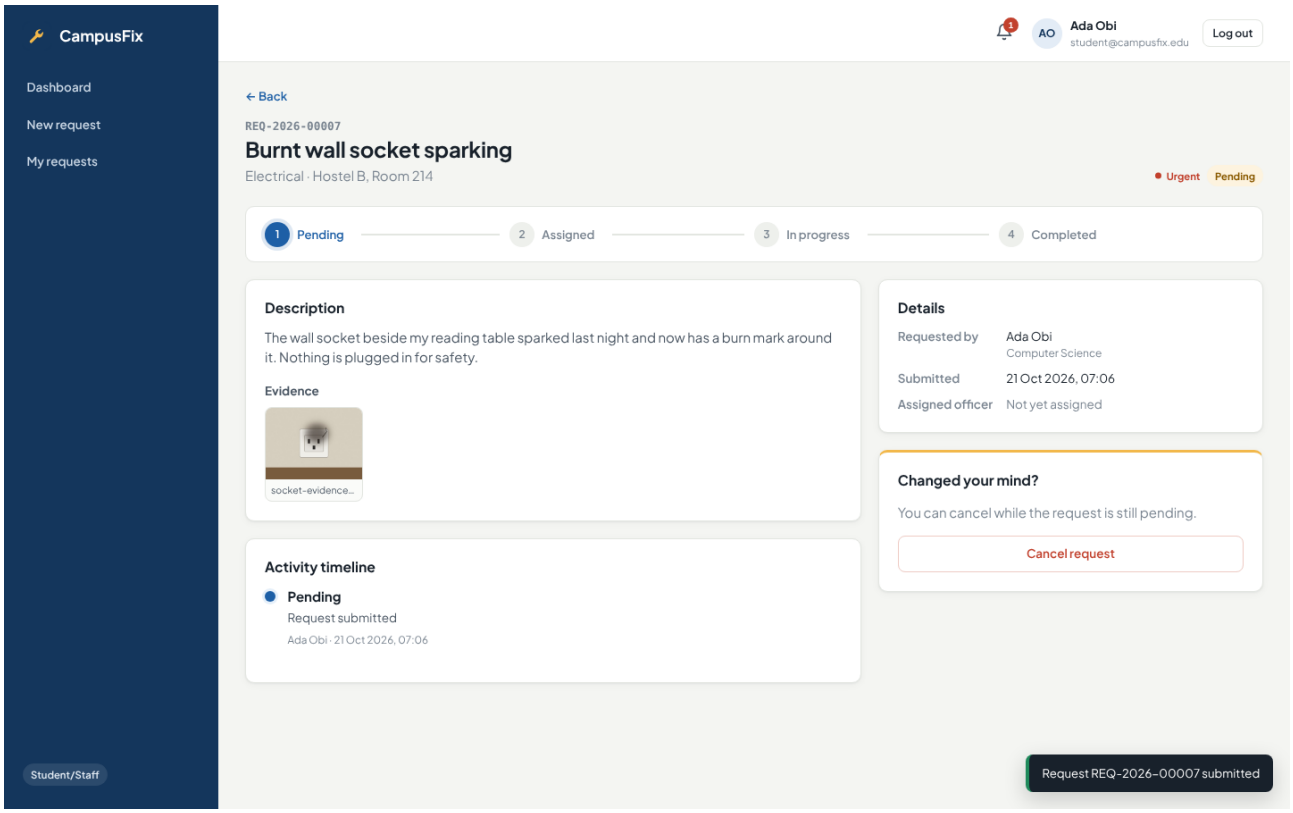


Figure 12. Request submitted — tracking page with ticket number, evidence and timeline

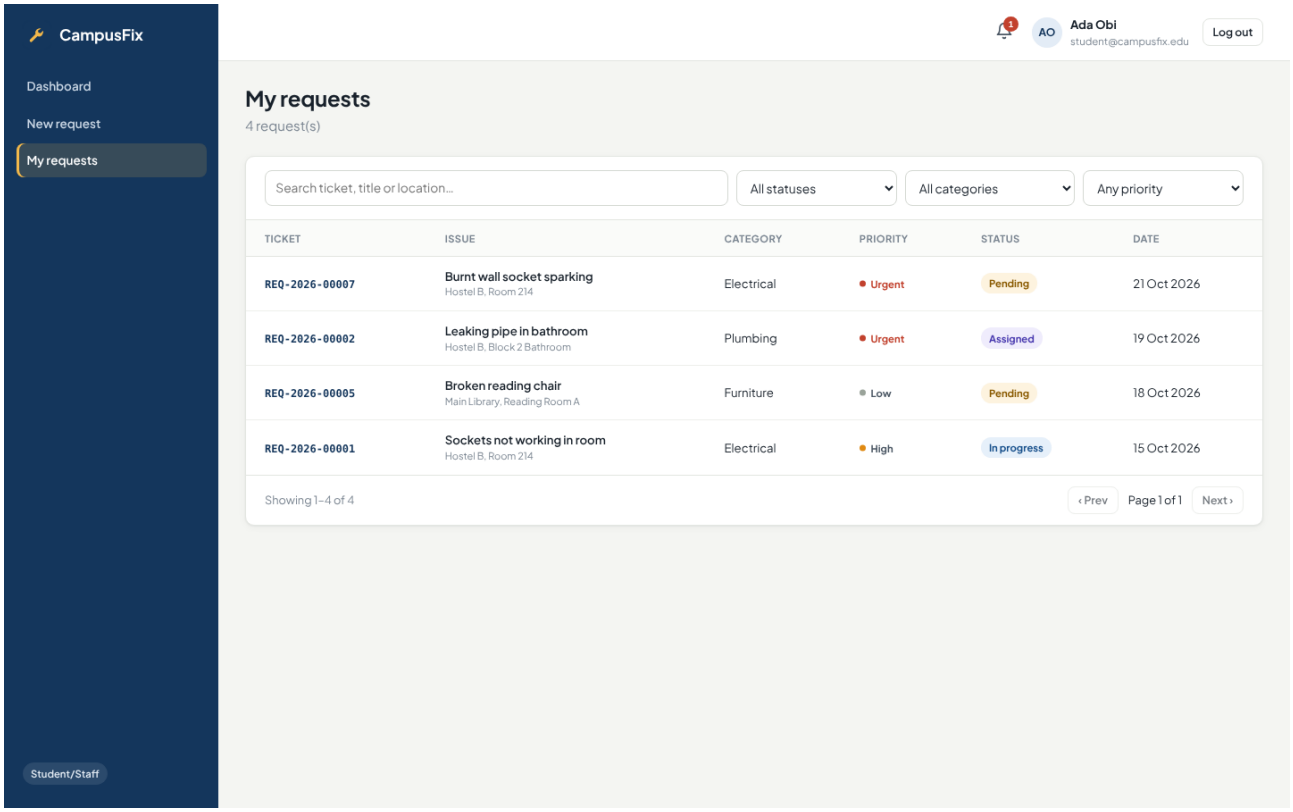


Figure 13. Student's "My requests" list

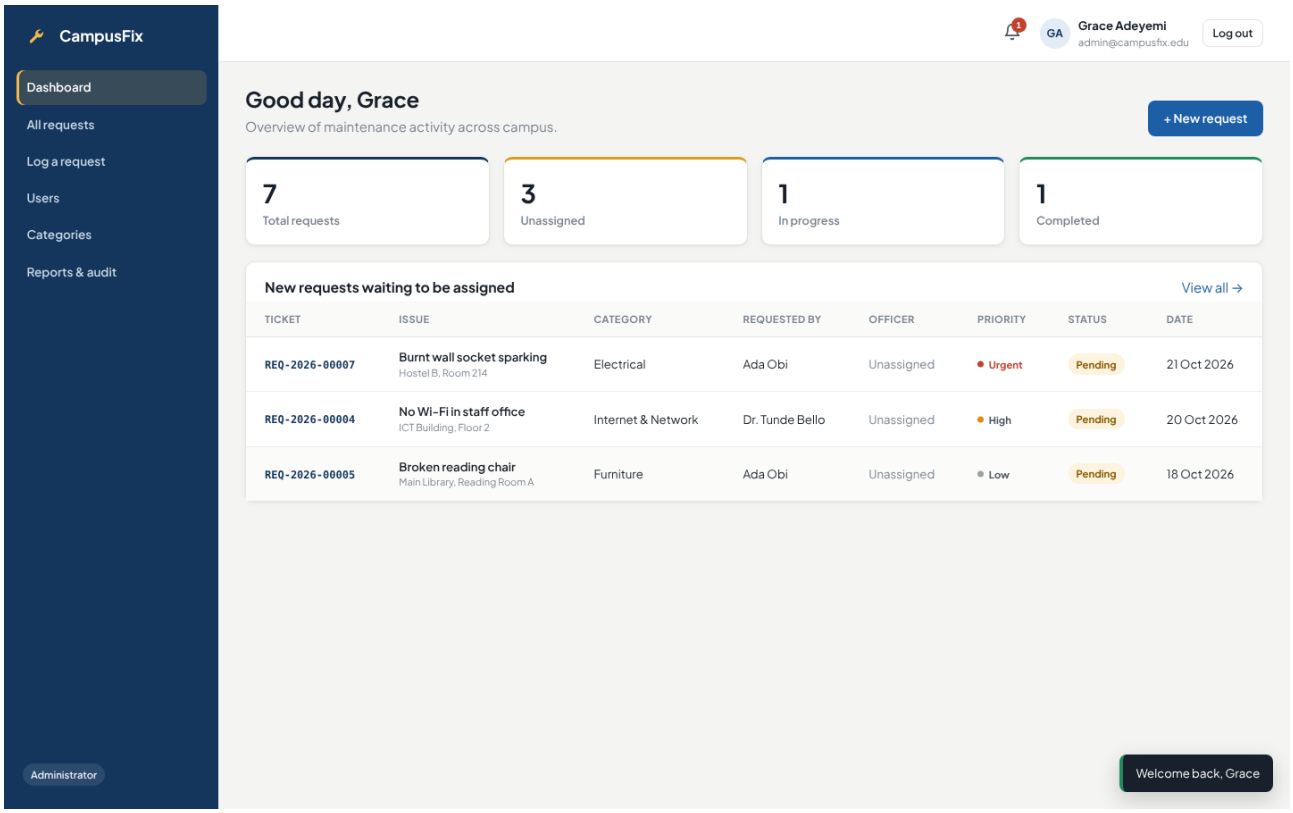


Figure 14. Administrator dashboard — unassigned queue

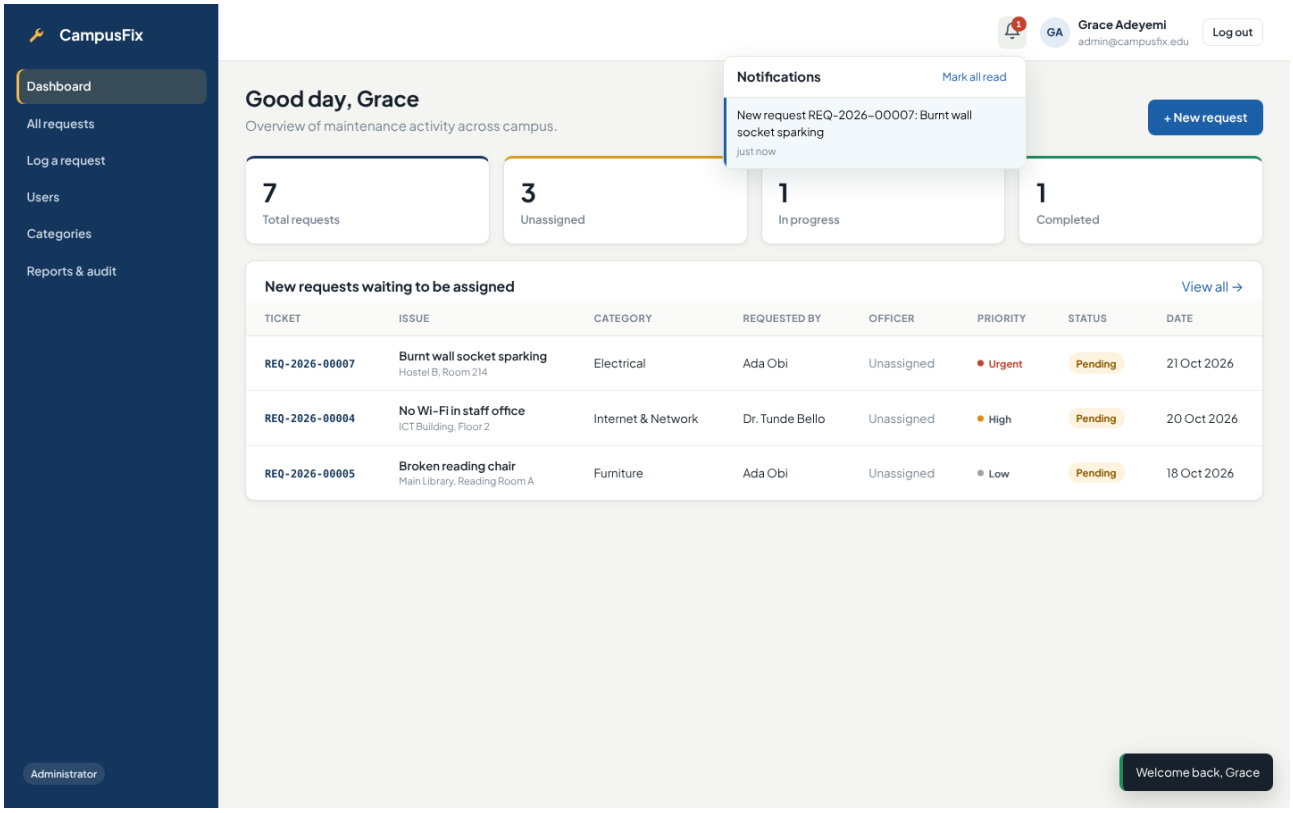


Figure 15. In-app notification for the administrator about the new request

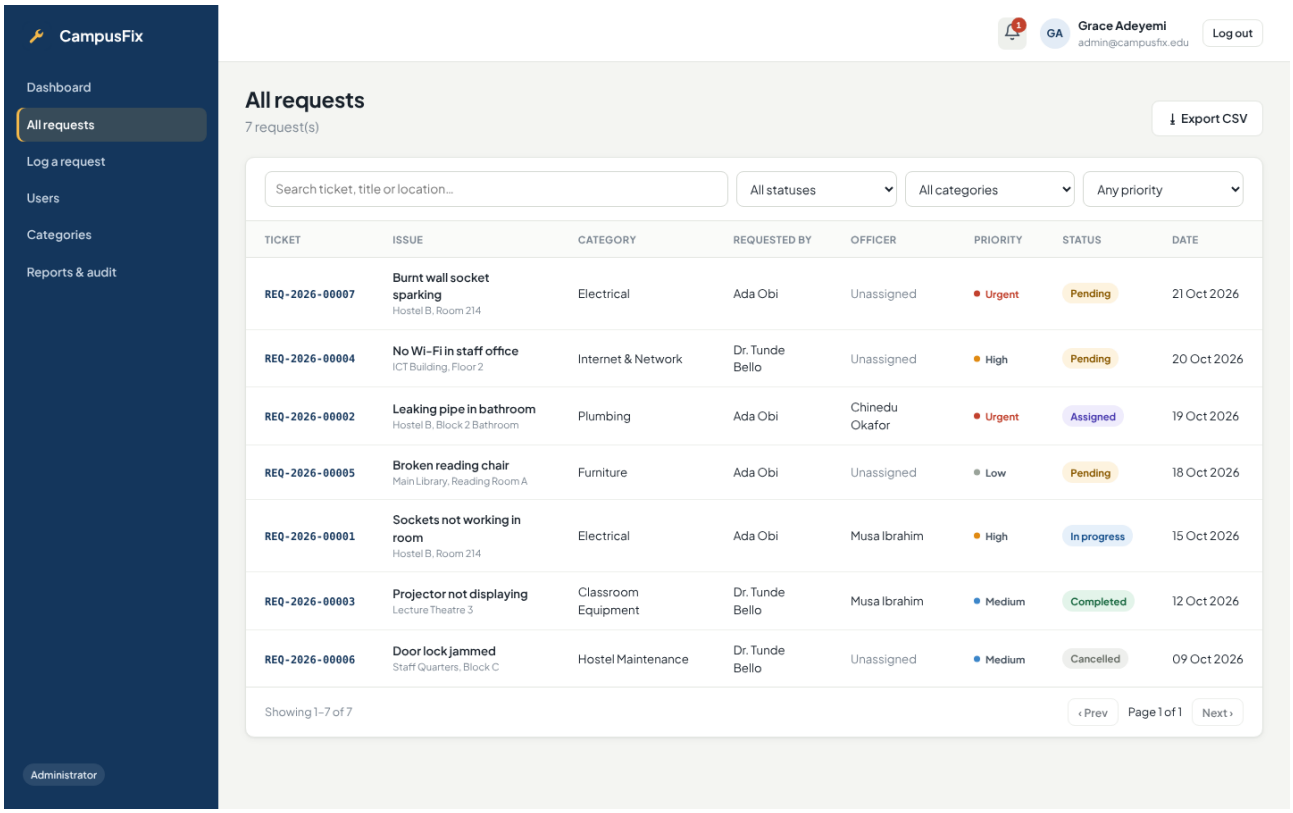


Figure 16. Admin request management — all requests with pagination

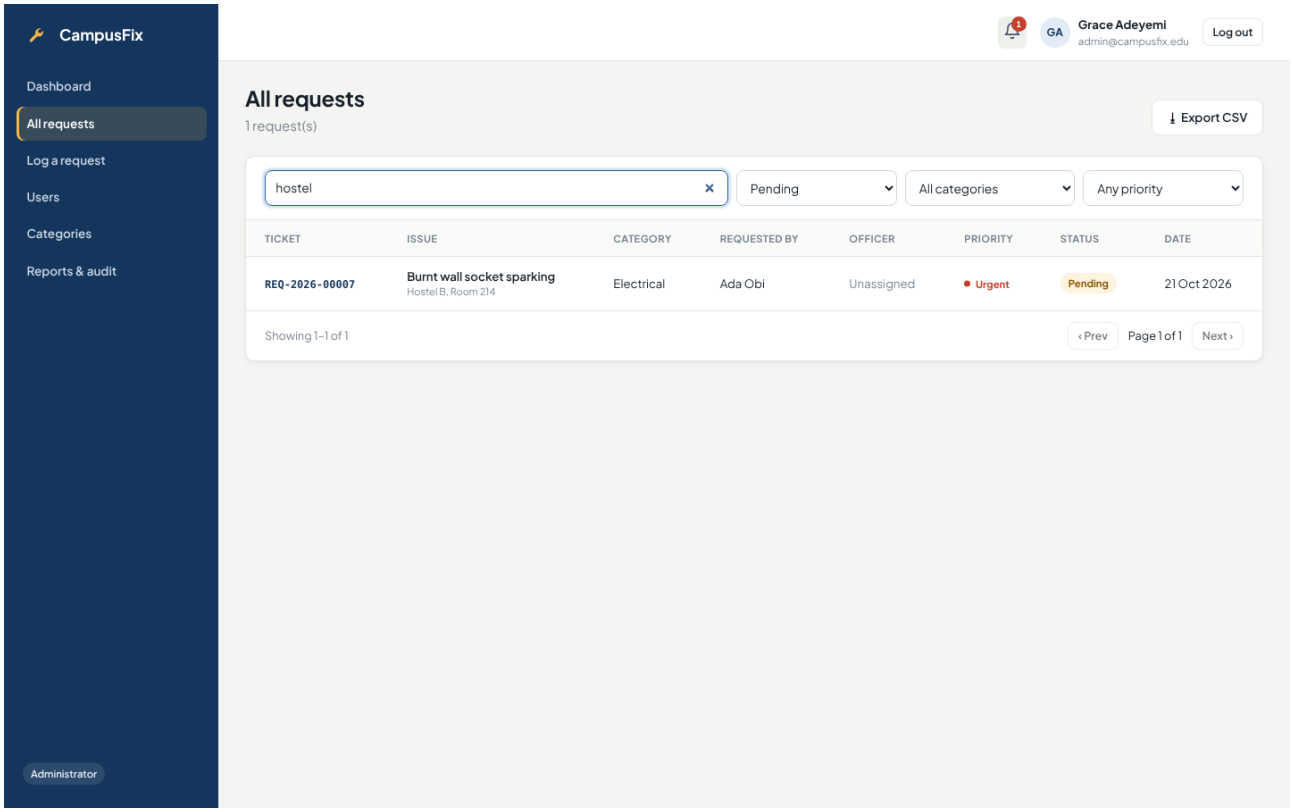


Figure 17. Search ("hostel") combined with a status filter

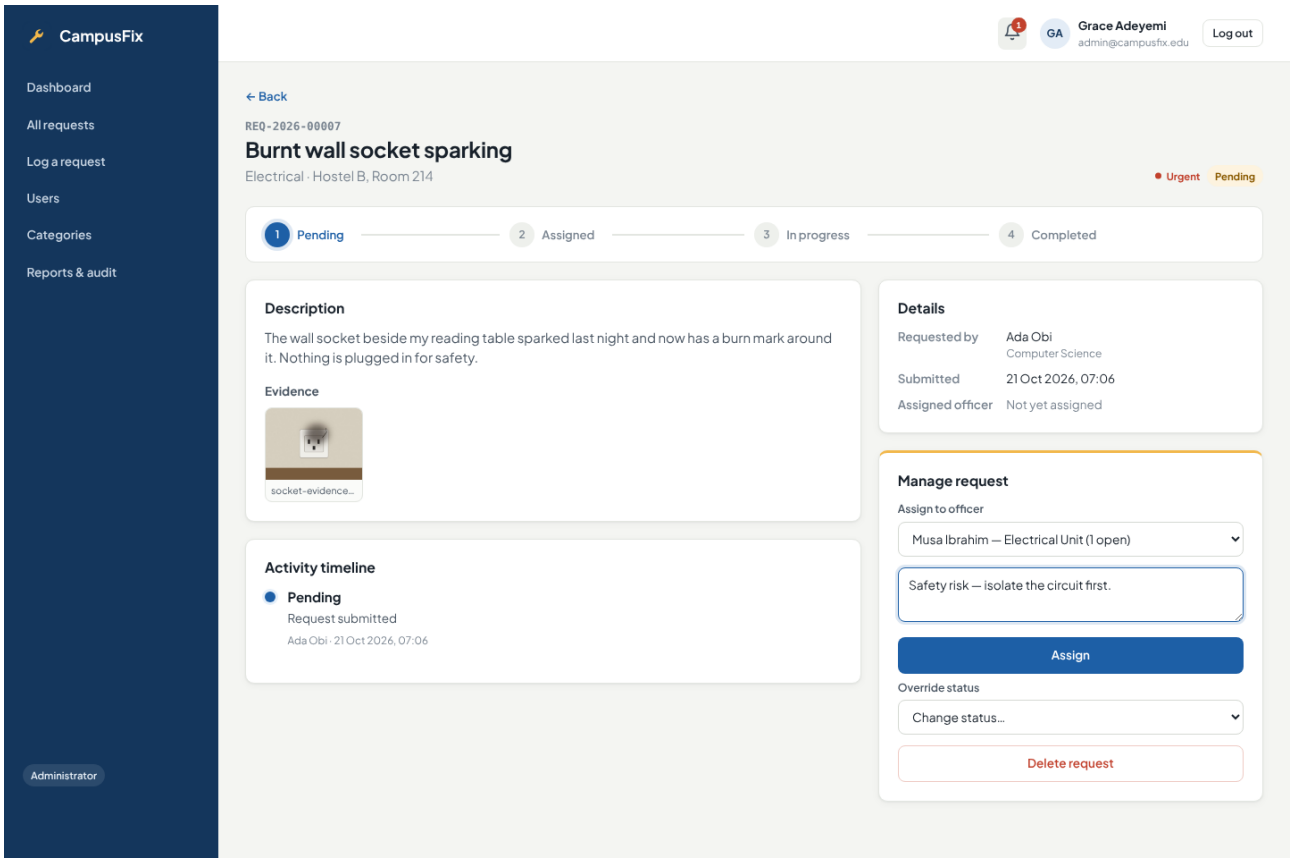


Figure 18. Admin assigning a maintenance officer with an instruction

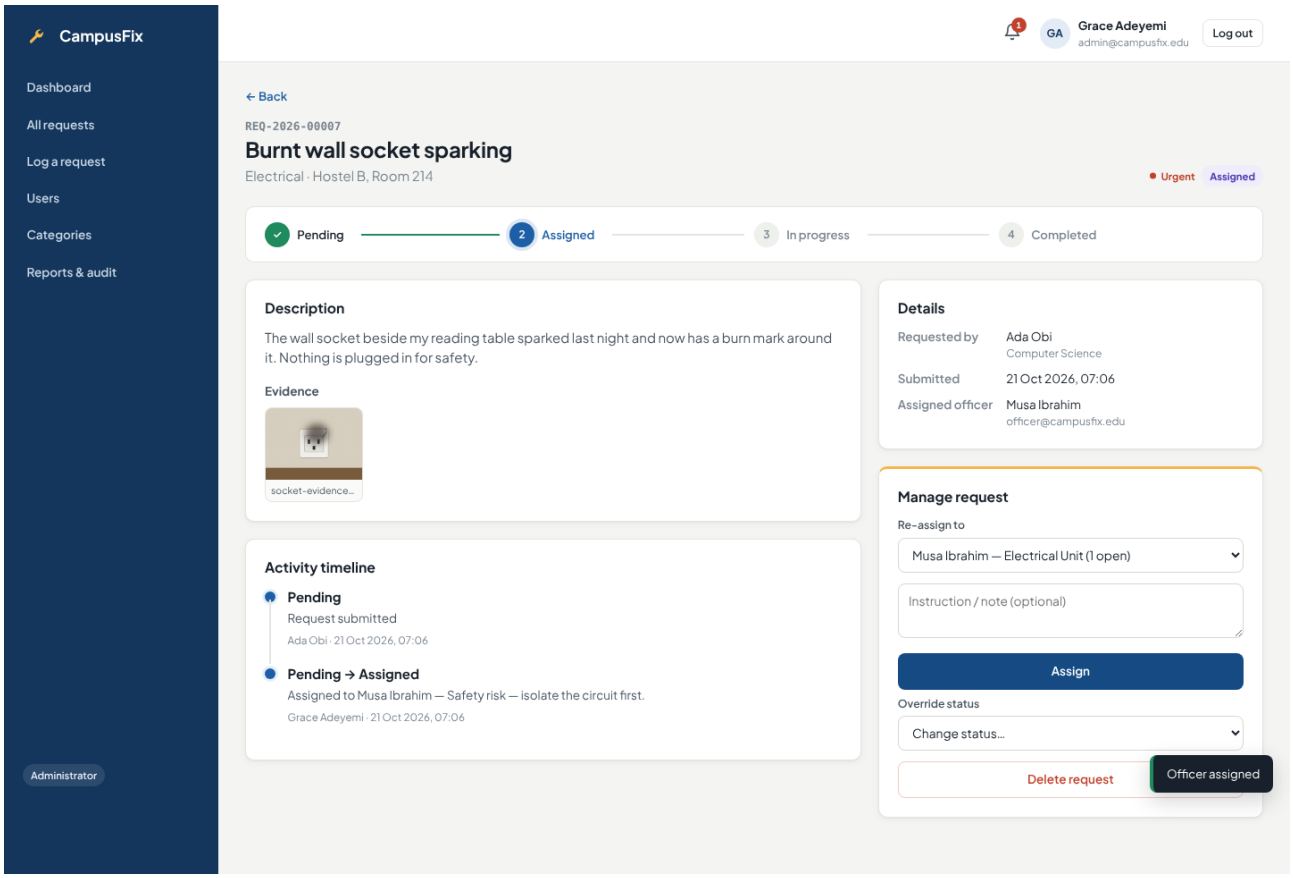


Figure 19. Assignment confirmed; status and timeline updated

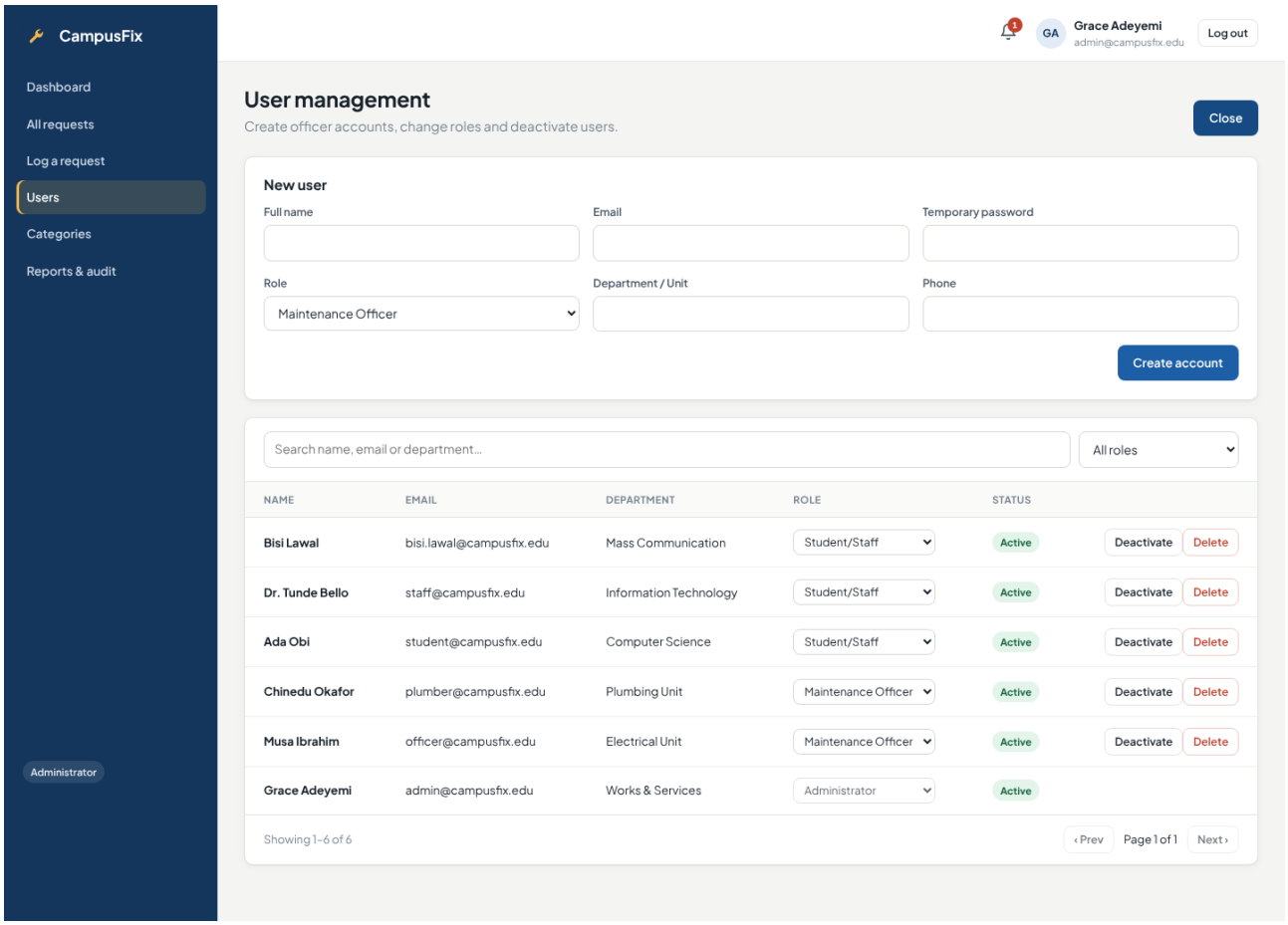


Figure 20. User management — create officer accounts, change roles, deactivate

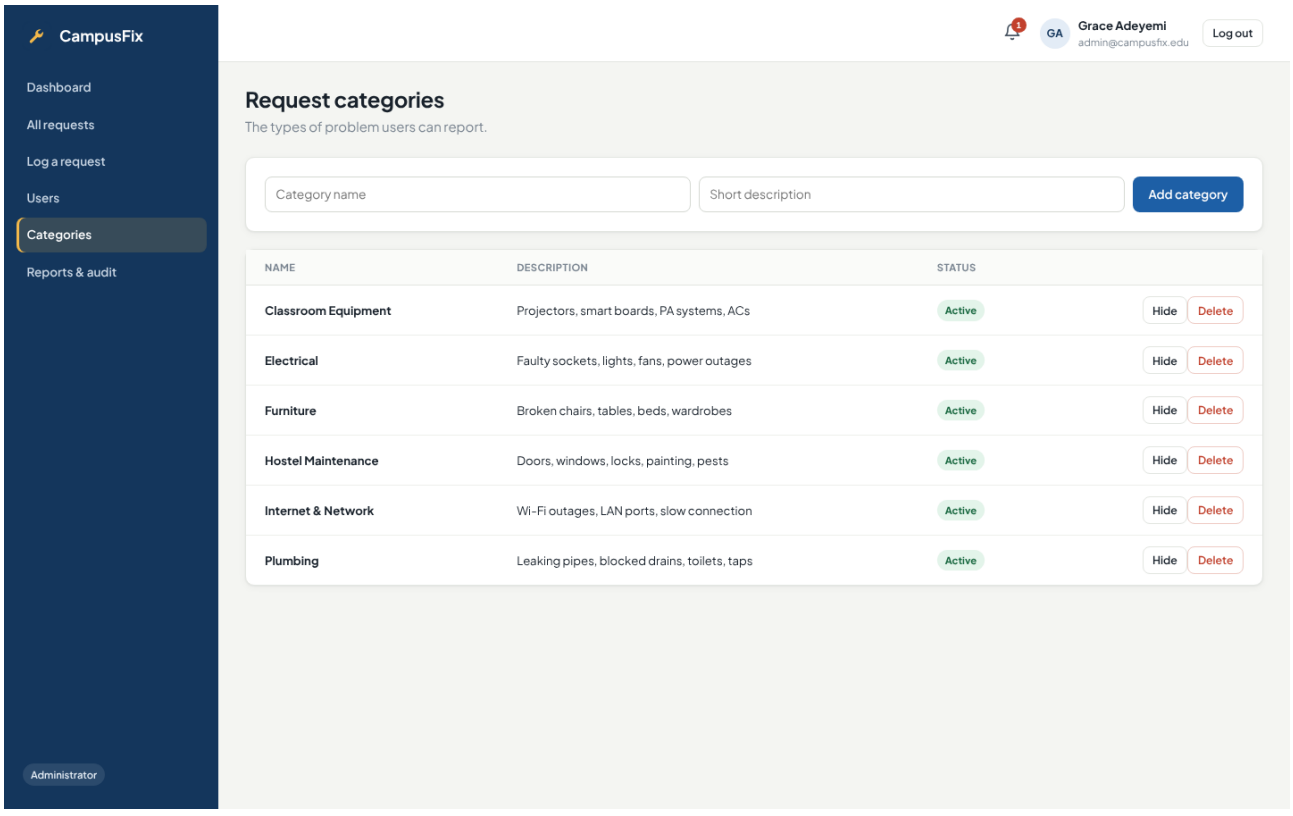


Figure 21. Request category management

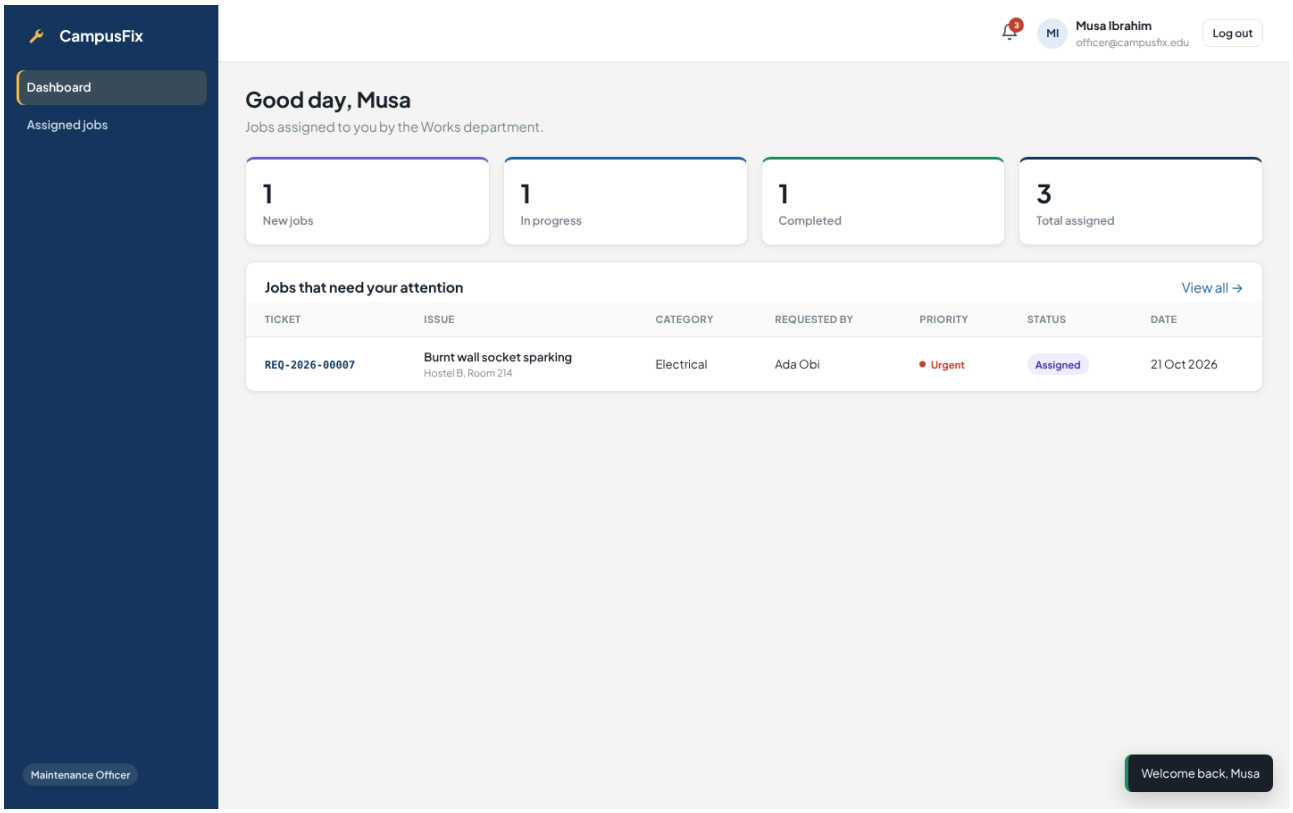


Figure 22. Maintenance Officer dashboard

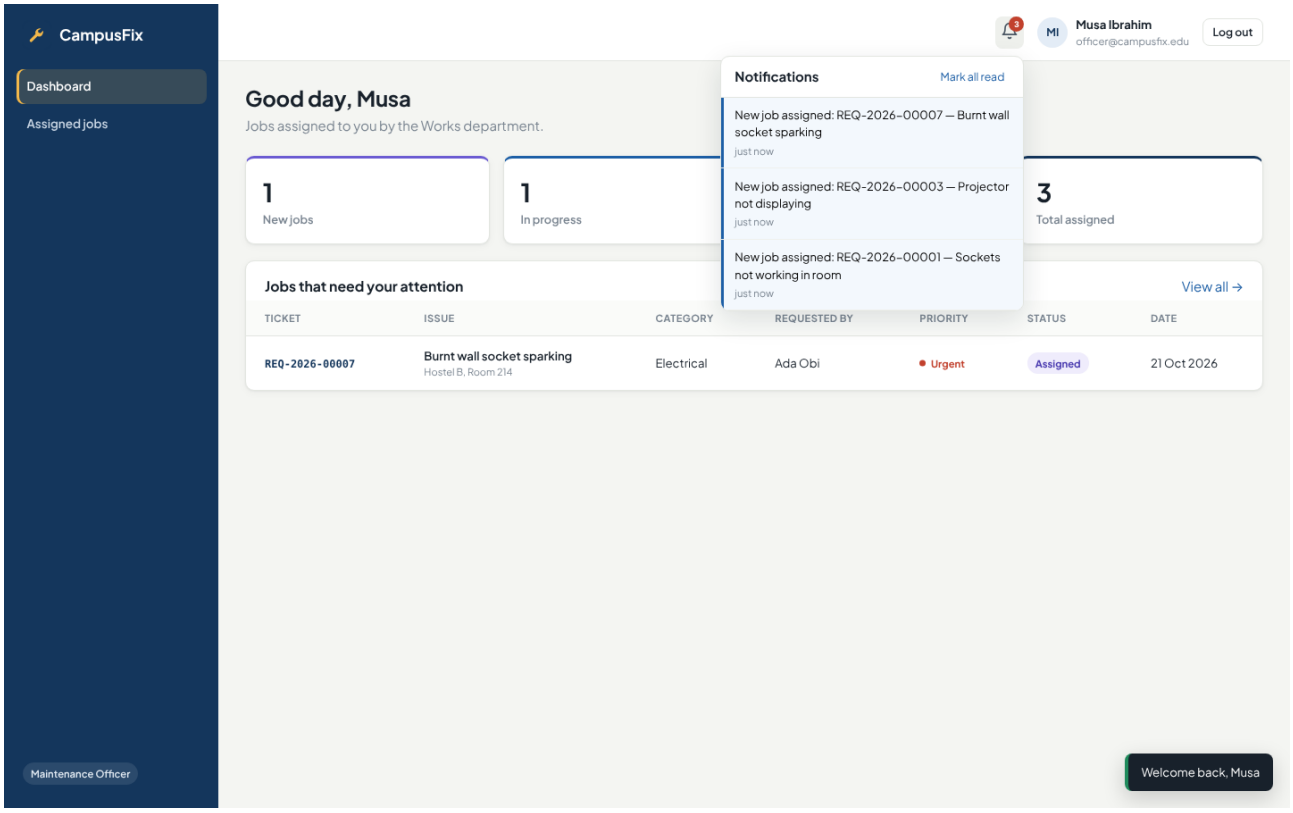


Figure 23. Officer notified of the new job

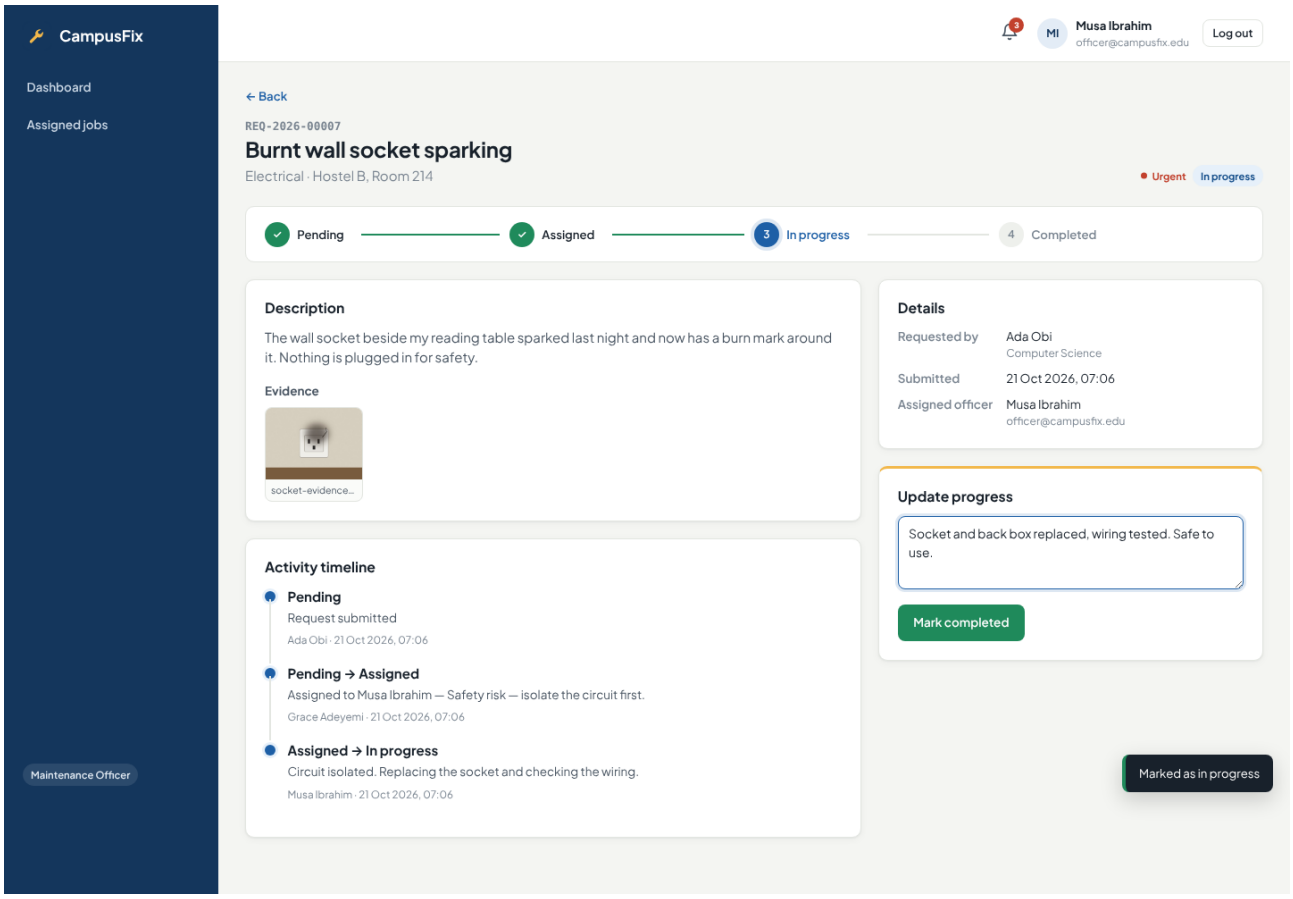


Figure 24. Officer started the job and is adding a completion note

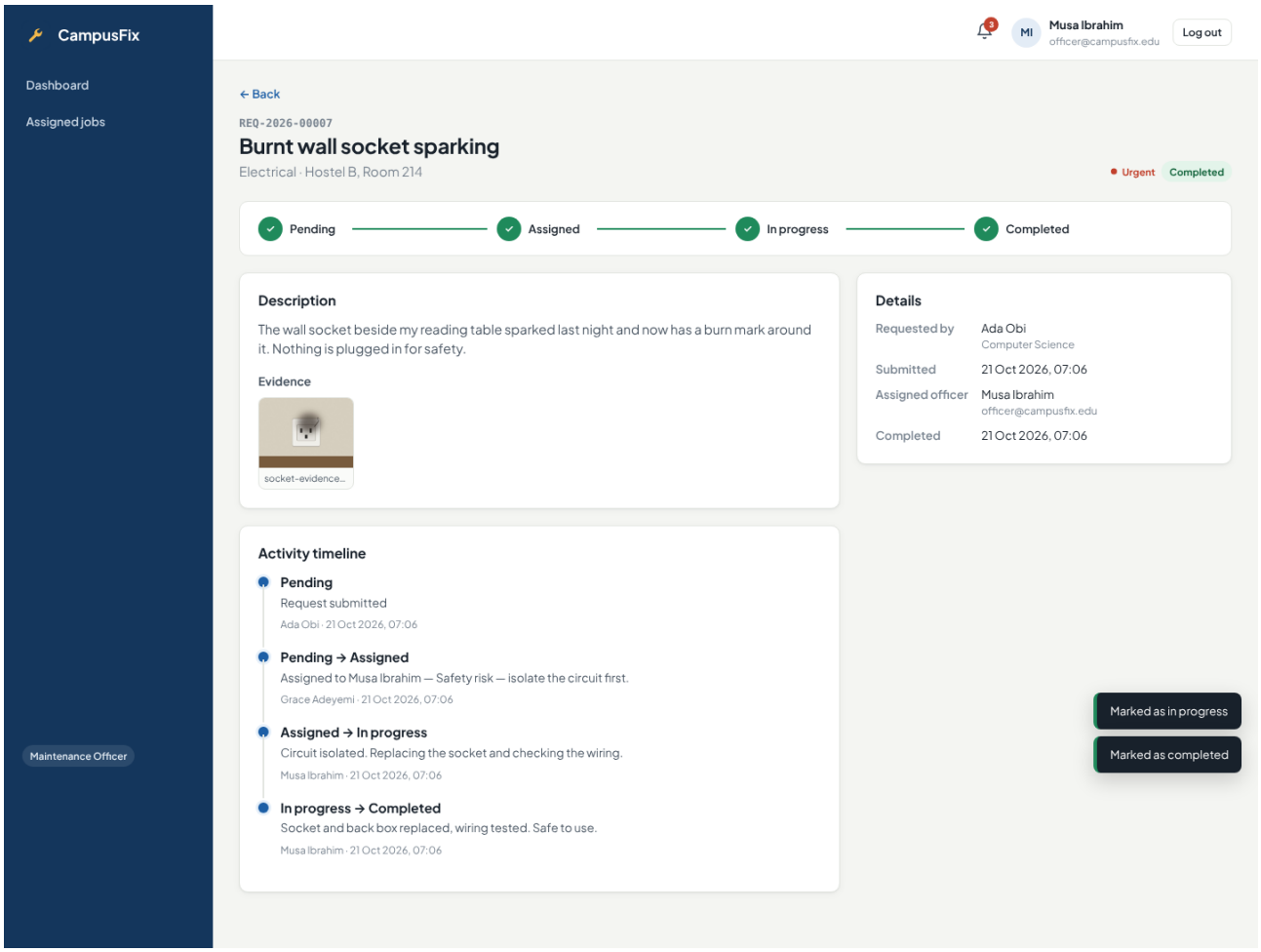


Figure 25. Officer marked the job completed

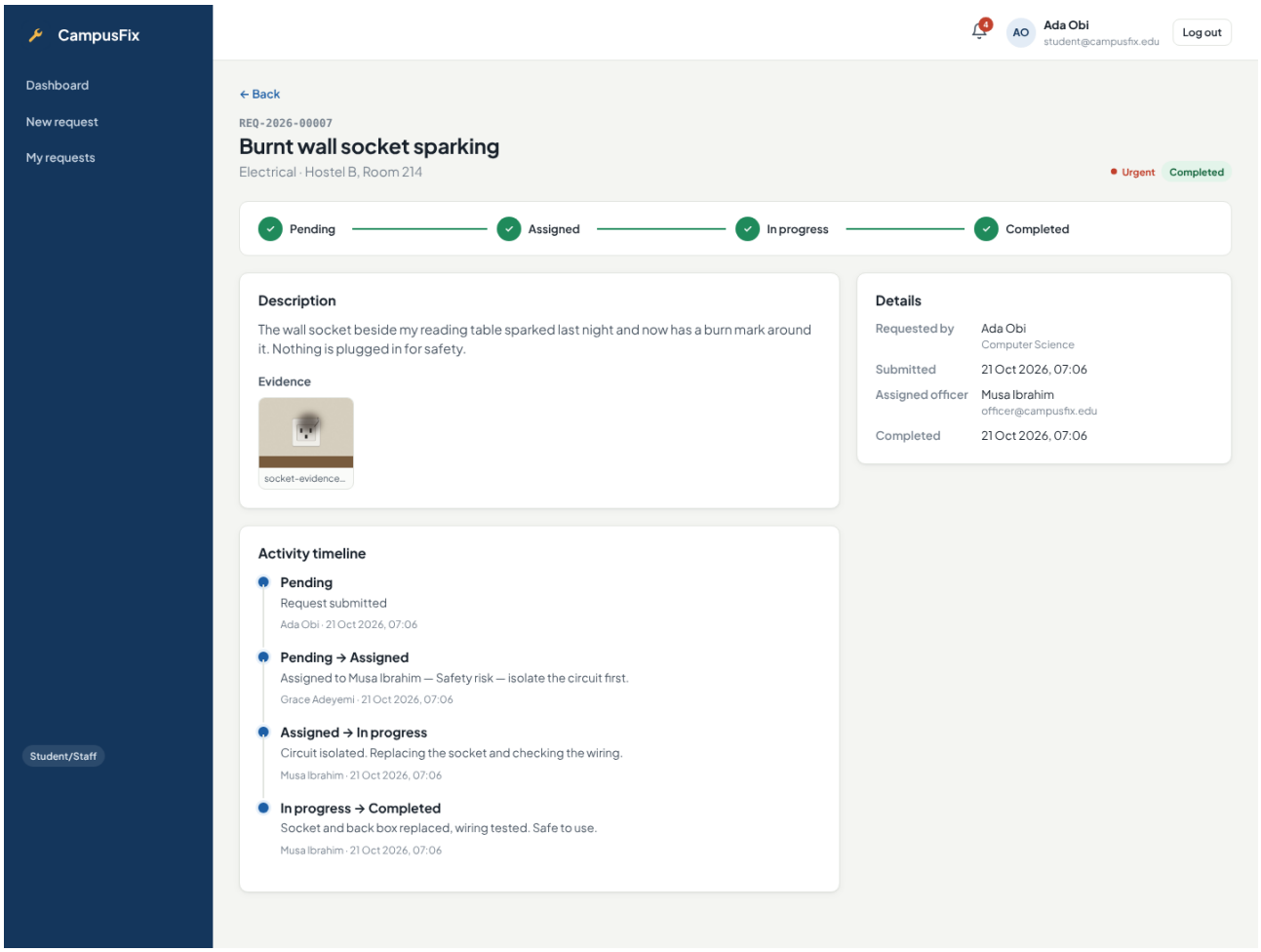


Figure 26. Requester's tracking page showing the full, attributed timeline

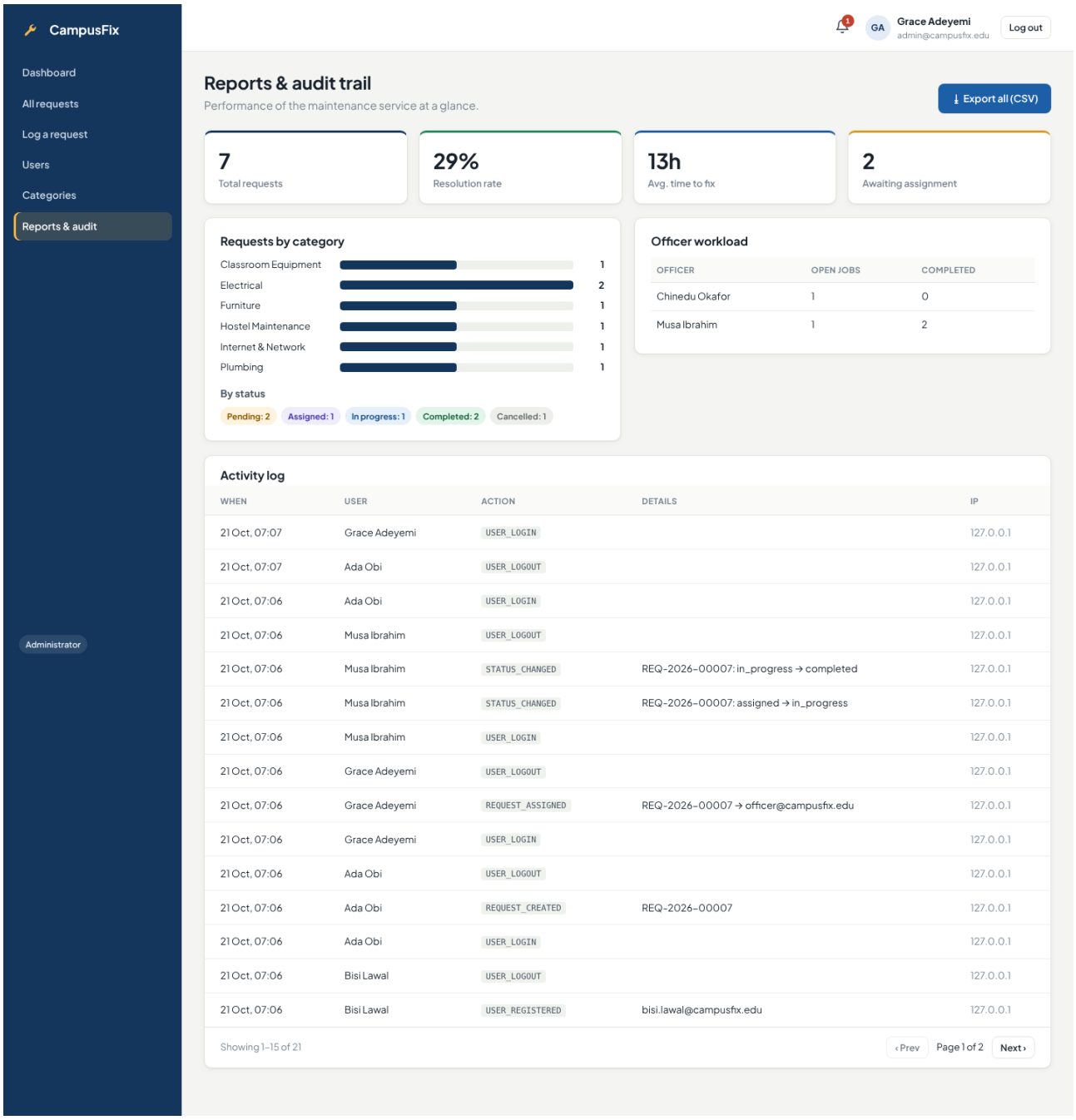


Figure 27. Reports: totals, resolution rate, average fix time, workload and audit trail



Figure 28. Health-check endpoint confirming the database connection

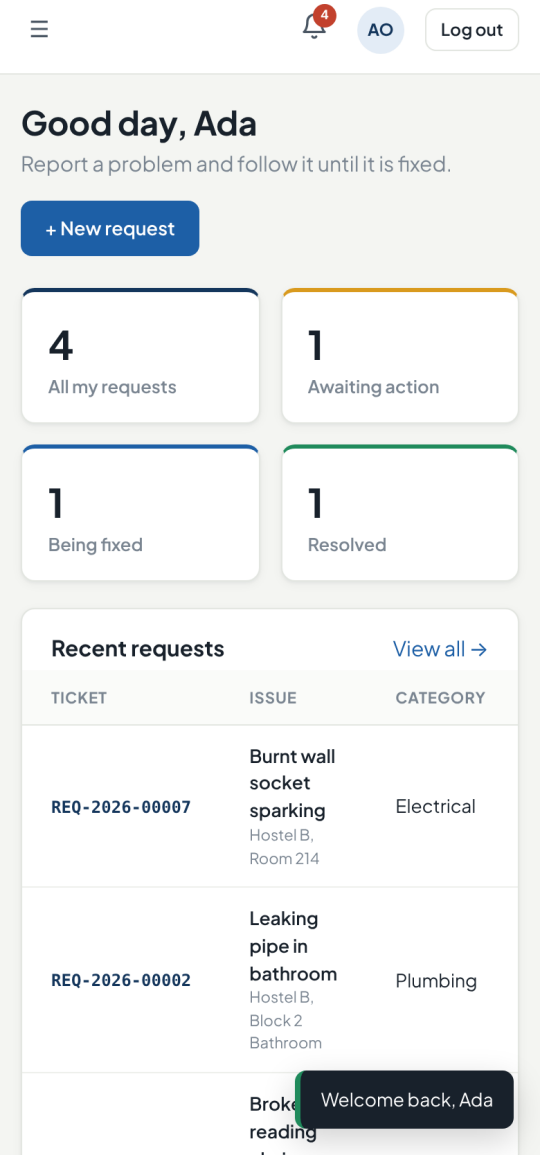


Figure 29. Responsive layout on a phone-sized screen

9. Testing Evidence

Testing was done at two levels, and both suites pass in full.

Suite	Tools	What is covered	Result
Backend API	Jest, Supertest, test MongoDB database	Registration, login, validation (422), duplicates (409), invalid/missing tokens (401), deactivated accounts, request CRUD with file upload, role scoping, search/filter/pagination, RBAC (403), assignment, officer status workflow, timeline, notifications, reports, CSV export, audit log, deletion (404 afterwards)	25 / 25 passed
Frontend components	Vitest, React Testing Library, jsdom	Login and registration validation, server error display, ProtectedRoute redirects by role, RequestForm validation and submission, file limits, StatusBadge, Pagination, RequestTable	16 / 16 passed

Commands: `cd server && npm test` and `cd client && npm test`. The API tests seed a separate `campusfix_test` database before running and drop it afterwards, so they never

touch real data. In addition, the complete user journey in Section 8 was executed end-to-end in a headless Chromium browser, including a check that a student who types an admin URL is redirected to their own dashboard.

Backend test run (npm test)

```
  console.error
      1 | // Shared helpers: connect to a throw-away test database and log in demo users
        |                    ^
      3 | process.env.NODE_ENV = 'test';
      4 | process.env.JWT_SECRET = process.env.JWT_SECRET || 'test-secret';
      5 |
      at ie (node_modules/dotenv/dist/index.cjs:7:155)
      at Object.ae [as configDotenv] (node_modules/dotenv/dist/index.cjs:7:1128)
      at Object.le [as config] (node_modules/dotenv/dist/index.cjs:7:1237)
      at Object.config (tests/setup.js:2:19)
      at Object.require (tests/requests.test.js:3:50)
  console.error
      1 | // Populates the database with roles, categories, demo accounts and sample requests.
      2 | // Run with:  npm run seed
        |                    ^
      4 | const mongoose = require('mongoose');
      5 | const Role = require('./models/Role');
      6 | const User = require('./models/User');
      at ie (node_modules/dotenv/dist/index.cjs:7:155)
      at Object.ae [as configDotenv] (node_modules/dotenv/dist/index.cjs:7:1128)
      at Object.le [as config] (node_modules/dotenv/dist/index.cjs:7:1237)
      at Object.config (src/seed.js:3:19)
      at Object.require (tests/setup.js:9:17)
      at Object.require (tests/requests.test.js:3:50)
  console.error
      1 | // Shared helpers: connect to a throw-away test database and log in demo users
        |                    ^
      3 | process.env.NODE_ENV = 'test';
      4 | process.env.JWT_SECRET = process.env.JWT_SECRET || 'test-secret';
      5 |
      at ie (node_modules/dotenv/dist/index.cjs:7:155)
      at Object.ae [as configDotenv] (node_modules/dotenv/dist/index.cjs:7:1128)
      at Object.le [as config] (node_modules/dotenv/dist/index.cjs:7:1237)
      at Object.config (tests/setup.js:2:19)
      at Object.require (tests/auth.test.js:1:50)
  console.error
      1 | // Populates the database with roles, categories, demo accounts and sample requests.
      2 | // Run with:  npm run seed
        |                    ^
      4 | const mongoose = require('mongoose');
      5 | const Role = require('./models/Role');
      6 | const User = require('./models/User');
      at ie (node_modules/dotenv/dist/index.cjs:7:155)
      at Object.ae [as configDotenv] (node_modules/dotenv/dist/index.cjs:7:1128)
      at Object.le [as config] (node_modules/dotenv/dist/index.cjs:7:1237)
      at Object.config (src/seed.js:3:19)
      at Object.require (tests/setup.js:9:17)
      at Object.require (tests/auth.test.js:1:50)
Test Suites: 2 passed, 2 total
Tests:       25 passed, 25 total
Snapshots:   0 total
Time:        9.711 s, estimated 10 s
Ran all test suites.
```

Frontend test run (npm test)

```
 RUN  v5.0.5 /home/claude/campusfix/client
 ✓ src/__tests__/RequestForm.test.jsx > RequestForm > shows validation messages and does not submit an empty form 230m
 ✓ src/__tests__/RequestForm.test.jsx > RequestForm > lists categories from the API in the dropdown 34ms
 ✓ src/__tests__/RequestForm.test.jsx > RequestForm > submits the form data when valid 441ms
 ✓ src/__tests__/RequestForm.test.jsx > RequestForm > validateRequest rejects more than 3 files and oversized files 1m
 ✓ src/__tests__/Auth.test.jsx > Login page > validates email and password before calling the API 213ms
 ✓ src/__tests__/Auth.test.jsx > Login page > shows the server error message on failed login 205ms
 ✓ src/__tests__/Auth.test.jsx > Login page > validateLogin and validateRegister rules 1ms
 ✓ src/__tests__/Auth.test.jsx > ProtectedRoute (role-based access) > hides the page from a visitor who is not logged
 ✓ src/__tests__/Auth.test.jsx > ProtectedRoute (role-based access) > hides an admin page from a student 3ms
 ✓ src/__tests__/Auth.test.jsx > ProtectedRoute (role-based access) > shows the page to an admin 2ms
 ✓ src/__tests__/Components.test.jsx > StatusBadge > renders pending as "Pending" with matching style 31ms
 ✓ src/__tests__/Components.test.jsx > StatusBadge > renders in_progress as "In progress" with matching style 3ms
 ✓ src/__tests__/Components.test.jsx > StatusBadge > renders completed as "Completed" with matching style 4ms
 ✓ src/__tests__/Components.test.jsx > Pagination > shows the range and disables Prev on the first page 155ms
 ✓ src/__tests__/Components.test.jsx > RequestTable > renders rows with a link to the tracking page 61ms
 ✓ src/__tests__/Components.test.jsx > RequestTable > shows the empty message when there are no rows 3ms
```

```
 Test Files  3 passed (3)
      Tests  16 passed (16)
   Start at  07:06:19
   Duration  4.92s (environment 52%, tests 31%, setup 8%, import 6%, transform 1%, worker 1%)
```

CSV export output (first rows)

```
"Ticket","Title","Category","Location","Priority","Status","Requester","Requester Email","Assigned To","Created","Comp
"REQ-2026-00007","Burnt wall socket sparking","Electrical","Hostel B, Room 214","urgent","completed","Ada Obi","studen
"REQ-2026-00004","No Wi-Fi in staff office","Internet & Network","ICT Building, Floor 2","high","pending","Dr. Tunde B
"REQ-2026-00002","Leaking pipe in bathroom","Plumbing","Hostel B, Block 2 Bathroom","urgent","assigned","Ada Obi","stu
"REQ-2026-00005","Broken reading chair","Furniture","Main Library, Reading Room A","low","pending","Ada Obi","student@
```

10. Deployment Information

Item	Value
Live URL	[https://your-app.onrender.com] ← replace after deploying
API documentation	[https://your-app.onrender.com/api/docs]
Health check	[https://your-app.onrender.com/api/health] → "database": "connected"
Hosting	Render — one Node web service (render.yaml blueprint)
Database	MongoDB Atlas, free M0 cluster, database "campusfix"
Build / start	npm run build / npm start
Environment variables	NODE_ENV=production, MONGO_URI, JWT_SECRET, JWT_EXPIRES_IN=1d
Demo logins (password Password123)	admin@campusfix.edu · officer@campusfix.edu · student@campusfix.edu

Deployment steps

- Create a MongoDB Atlas M0 cluster, add a database user, allow network access from 0.0.0.0/0 and copy the connection string with `/campusfix` as the database name.
- Push the project to GitHub (the `.env` file is git-ignored, so secrets are never committed).
- On Render choose New → Blueprint and select the repository; Render reads `render.yaml`, generates a random JWT_SECRET, and asks for MONGO_URI.
- The build installs both packages and compiles the React app; in production Express serves the compiled files and the API from the same URL, so no CORS configuration is needed.
- Open the Render Shell once and run `npm run seed` to create roles, categories and demo accounts.
- Visit `/api/health`: a response of `"database": "connected"` proves the deployed app reaches Atlas. Then log in with a demo account.

[Insert a screenshot of the live site and of the Render dashboard showing a successful deploy here.]

11. Challenges Encountered and Solutions

Challenge	Solution
Keeping each role to its own data (a student must never see another student's request).	A single scopeFor(user) function builds the MongoDB filter for every list, count and export, and loadRequestForUser() checks ownership/assignment before any detail, edit or status change. Tests assert 403 responses.
Real-time updates without exposing other users' events.	The Socket.IO handshake verifies the JWT; each user joins a private room "user:<id>" so notifications go only to their recipient. Pages re-fetch through the normal, permission-checked API instead of trusting socket data.
Ticket numbers duplicating after deletions.	Generating the number from a document count repeated numbers once a request was deleted. The pre-save hook now takes the highest existing ticket number and adds one.
Mongoose 9 upgrade broke the password-hashing hook ("next is not a function").	Async middleware in Mongoose 9 no longer receives next(); the hooks were rewritten as plain async functions that return.
Validation messages from three different sources (React, express-validator, Mongoose).	A central error handler converts every error into one JSON shape with a field list; the frontend has one errorMessage() helper to display it.
Showing evidence photos safely.	Multer restricts uploads to images/PDF, 5 MB each and three files, and renames files to remove unsafe characters. Note: Render's free disk is temporary, so for long-term storage the upload middleware can be pointed at Cloudinary or S3.
A test for protected routes froze the test runner.	The redirect target was rendered by the same component, creating an infinite redirect loop. The test now renders real routes, which also makes it a better model of the app.

12. Conclusion

CampusFix meets the brief: it replaces phone calls, paper forms and WhatsApp messages with a single, accountable platform. Students and staff report faults with evidence and follow them live; officers see exactly the jobs assigned to them and record their progress; administrators manage users, categories and assignments, and can measure the service through reports, CSV export and a complete audit trail.

The project implements every core frontend, backend and database requirement and eight of the nine advanced features (JWT authentication, RBAC, file upload, notifications, search/filter/pagination, real-time updates, audit trail, Swagger documentation and CSV export), backed by 41 passing automated tests. Possible next steps are e-mail/SMS notifications, cloud storage for photos, PDF reports, service-level targets per priority, and a requester satisfaction rating after completion.

Appendix A. Complete Code Base

Every source file of the project is listed below, grouped by folder. Generated folders (`node_modules`, `client/dist`), lock files and the private `.env` file are excluded.

README.md

```
# CampusFix — University Maintenance Request System

Full-stack MERN web application (MongoDB, Express, React, Node.js) for MIT 8333 — Advanced Web Application Development.

Students and staff submit maintenance requests (electricity, furniture, plumbing, internet, classroom equipment, hostel issues) with photo evidence. Administrators assign them to maintenance officers, officers update progress, and everyone follows the request live until it is completed.

## Features

| Requirement | Where |
|---|---|
| Registration & login | `client/src/pages/Login.jsx`, `Register.jsx`, `server/src/routes/auth.js` |
| Role-based dashboards (Student/Staff, Officer, Admin) | `client/src/pages/Dashboard.jsx`, `components/Layout.jsx` |
| Request form, tracking page, admin management | `NewRequest.jsx`, `RequestDetail.jsx`, `Requests.jsx`, `pages/admin/*` |
| JWT authentication + bcrypt password hashing | `server/src/middleware/auth.js`, `models/User.js` |
| Role-based access control (API + UI) | `authorize()` middleware, `ProtectedRoute.jsx` |
| File/image upload (evidence) | `server/src/middleware/upload.js` (Multer) |
| In-app notifications | `routes/notifications.js`, bell in `Layout.jsx` |
| Real-time status updates | Socket.IO — `server/src/utils/socket.js`, `client/src/context/SocketContext.jsx` |
| Search, filter, pagination | `GET /api/requests?q=&status=&category=&priority=&page=` |
| Audit trail / activity log | `models/AuditLog.js`, `GET /api/reports/audit` |
| API documentation (Swagger) | `/api/docs` |
| CSV export | `GET /api/reports/export` |

## Run it locally

You need **Node.js 18+** and **MongoDB** (install MongoDB Community Server locally, or create a free cluster on MongoDB Atlas).

```bash
# 1. Backend
cd server
cp .env.example .env          # then edit MONGO_URI and JWT_SECRET
npm install
npm run seed                  # roles, categories and demo accounts
npm run dev                   # API on http://localhost:5000  (docs: /api/docs)

# 2. Frontend (new terminal)
cd client
npm install
npm run dev                   # app on http://localhost:5173
```

Demo accounts (password `Password123`)

Role	Email
Administrator	admin@campusfix.edu
Maintenance Officer	officer@campusfix.edu, plumber@campusfix.edu
Student / Staff	student@campusfix.edu, staff@campusfix.edu

Tests

```bash
cd server && npm test   # 25 API tests (Jest + Supertest) — needs MongoDB running
cd client && npm test   # 16 component tests (Vitest + React Testing Library)
```

The API tests use a separate database (`campusfix_test`, or set `MONGO_URI_TEST`) and drop it when finished.

Deploy (MongoDB Atlas + Render)

1. **Database:** create a free M0 cluster on MongoDB Atlas → Database Access: add a user → Network Access: allow `0.0.0.0/0` → copy the connection string and add `/campusfix` before the `?`.
2. **Code:** push this folder to a GitHub repository.
3. **Render:** New → Blueprint → pick the repo (uses `render.yaml`), or New → Web Service with
   - Build command: `npm run build`
```

```
    - Start command: `npm start`
    - Environment: `NODE_ENV=production`, `MONGO_URI=<Atlas string>`, `JWT_SECRET=<long random string>`
4. After the first deploy, open the Render **Shell** and run `npm run seed` once.
5. Check `https://<your-app>.onrender.com/api/health` — it should say `"database": "connected"`.

Express serves the React build in production, so the whole app runs at one URL. Note: Render's free disk is te
↪ mporary, so uploaded photos are lost on redeploy; for permanent storage switch Multer to Cloudinary or S3.
```

package.json

```
{
  "name": "campusfix",
  "version": "1.0.0",
  "private": true,
  "description": "CampusFix - university maintenance request system (MERN). MIT 8333 project.",
  "scripts": {
    "install:all": "npm install --prefix server && npm install --prefix client",
    "build": "npm run install:all && npm run build --prefix client",
    "start": "npm start --prefix server",
    "seed": "npm run seed --prefix server",
    "test": "npm test --prefix server && npm test --prefix client"
  },
  "engines": {
    "node": ">=18"
  }
}
```

render.yaml

```
# Render "Blueprint": one web service that runs the API and serves the React build.
# Set MONGO_URI (MongoDB Atlas) in the Render dashboard — it is not stored here.
services:
  - type: web
    name: campusfix
    runtime: node
    plan: free
    buildCommand: npm run build
    startCommand: npm start
    healthCheckPath: /api/health
    envVars:
      - key: NODE_ENV
        value: production
      - key: JWT_SECRET
        generateValue: true
      - key: JWT_EXPIRES_IN
        value: 1d
      - key: MONGO_URI
        sync: false
```

server/.env.example

```
# Copy this file to .env and fill in your own values
NODE_ENV=development
PORT=5000

# Local MongoDB:  mongodb://127.0.0.1:27017/campusfix
# MongoDB Atlas:  mongodb+srv://<user>:<password>@<cluster>.mongodb.net/campusfix
MONGO_URI=mongodb://127.0.0.1:27017/campusfix

# Use a long random string in production
JWT_SECRET=change-me-to-a-long-random-string
JWT_EXPIRES_IN=1d

# Where the React app runs (for CORS and Socket.IO)
CLIENT_URL=http://localhost:5173
```

server/.gitignore

```
node_modules
.env
uploads/*
!uploads/.gitkeep
```

server/package.json

```
{
  "name": "campusfix-server",
  "version": "1.0.0",
  "description": "CampusFix API - university maintenance request system",
  "main": "src/server.js",
  "directories": {
    "test": "tests"
  },
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js",
    "seed": "node src/seed.js",
    "test": "jest --runInBand --forceExit"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^18.0.8",
    "express": "^5.3.0",
    "express-rate-limit": "^8.7.1",
    "express-validator": "^7.3.2",
    "helmet": "^8.3.0",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.11.2",
    "morgan": "^1.12.1",
    "multer": "^2.4.0",
    "socket.io": "^4.8.4",
    "swagger-ui-express": "^5.0.1"
  },
  "devDependencies": {
    "jest": "^30.5.2",
    "nodemon": "^3.1.14",
    "supertest": "^7.3.1"
  },
  "jest": {
    "testEnvironment": "node",
    "testTimeout": 30000
  },
  "engines": {
    "node": ">=18"
  }
}
```

server/src/app.js

```
const path = require('path');
const fs = require('fs');
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const mongoose = require('mongoose');
const swaggerUi = require('swagger-ui-express');
const openapi = require('./docs/openapi');
const { notFound, errorHandler } = require('./middleware/error');

const app = express();

app.use(helmet({ contentSecurityPolicy: false, crossOriginResourcePolicy: { policy: 'cross-origin' } }));
app.use(cors({ origin: process.env.CLIENT_URL || '*' }));
app.use(express.json({ limit: '1mb' }));
if (process.env.NODE_ENV === 'development') app.use(morgan('dev'));

// Uploaded evidence photos
app.use('/uploads', express.static(path.join(__dirname, '..', 'uploads')));

// API documentation
app.use('/api/docs', swaggerUi.serve, swaggerUi.setup(openapi, { customSiteTitle: 'CampusFix API Docs' }));
app.get('/api/docs.json', (_req, res) => res.json(openapi));

// Health check — also proves the deployed app can reach the database
app.get('/api/health', (_req, res) => {
  const dbUp = mongoose.connection.readyState === 1;
  res.status(dbUp ? 200 : 503).json({ success: dbUp, api: 'up', database: dbUp ? 'connected' : 'disconnected',
  ↪ time: new Date() });
});

app.use('/api/auth', require('./routes/auth'));
app.use('/api/requests', require('./routes/requests'));
```

```
app.use('/api/users', require('./routes/users'));
app.use('/api/categories', require('./routes/categories'));
app.use('/api/notifications', require('./routes/notifications'));
app.use('/api/reports', require('./routes/reports'));

// In production, Express also serves the built React app (one deployment)
const clientDist = path.join(__dirname, '..', '..', 'client', 'dist');
if (process.env.NODE_ENV === 'production' && fs.existsSync(clientDist)) {
  app.use(express.static(clientDist));
  app.get(/^\/(?!api|uploads).*/, (_req, res) => res.sendFile(path.join(clientDist, 'index.html')));
}

app.use(notFound);
app.use(errorHandler);

module.exports = app;
```

server/src/config/db.js

```
const mongoose = require('mongoose');

async function connectDB(uri) {
  mongoose.set('strictQuery', true);
  await mongoose.connect(uri);
  console.log(`MongoDB connected: ${mongoose.connection.host}/${mongoose.connection.name}`);
}

module.exports = connectDB;
```

server/src/docs/openapi.js

```
// OpenAPI 3 specification — served as interactive Swagger UI at /api/docs
const bearer = [{ bearerAuth: [] }];
const idParam = { name: 'id', in: 'path', required: true, schema: { type: 'string' } };
const ok = (description) => ({ description });
const json = (schema) => ({ required: true, content: { 'application/json': { schema } } });

module.exports = {
  openapi: '3.0.3',
  info: {
    title: 'CampusFix — University Maintenance Request API',
    version: '1.0.0',
    description:
      'REST API for submitting, assigning and tracking university maintenance requests. ' +
      'Log in with POST /api/auth/login, copy the token, click "Authorize" and paste it.',
  },
  servers: [{ url: '/' }],
  components: {
    securitySchemes: { bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' } },
    schemas: {
      Credentials: {
        type: 'object', required: ['email', 'password'],
        properties: { email: { type: 'string', example: 'student@campusfix.edu' }, password: { type: 'string',
 example: 'Password123' } },
      },
      Register: {
        type: 'object', required: ['name', 'email', 'password'],
        properties: {
          name: { type: 'string', example: 'Ada Obi' }, email: { type: 'string', example: 'ada@campusfix.edu'
},
          password: { type: 'string', example: 'Password123' }, department: { type: 'string', example: 'Comput
er Science' },
          phone: { type: 'string', example: '08030000000' },
        },
      },
      ServiceRequestInput: {
        type: 'object', required: ['title', 'description', 'category', 'location'],
        properties: {
          title: { type: 'string', example: 'Ceiling fan not working' },
          description: { type: 'string', example: 'The fan in Lecture Hall 2 makes noise and stops after a few
 minutes.' },
          category: { type: 'string', description: 'Category ObjectId' },
          location: { type: 'string', example: 'Lecture Hall 2' },
          priority: { type: 'string', enum: ['low', 'medium', 'high', 'urgent'] },
          attachments: { type: 'array', items: { type: 'string', format: 'binary' } },
        },
      },
      Error: {
        type: 'object',
        properties: {
          success: { type: 'boolean', example: false }, message: { type: 'string' },
          errors: { type: 'array', items: { type: 'object', properties: { field: { type: 'string' }, message:
{ type: 'string' } } } },
```

```
        },
      },
    },
  },
  tags: [
    { name: 'Auth' }, { name: 'Service Requests' }, { name: 'Users (Admin)' },
    { name: 'Categories' }, { name: 'Notifications' }, { name: 'Reports (Admin)' },
  ],
  paths: {
    '/api/auth/register': { post: { tags: ['Auth'], summary: 'Register a Student/Staff account', requestBody: json({ $ref: '#/components/schemas/Register' }), responses: { 201: ok('Created — returns token and user'), 409: ok('Email already registered'), 422: ok('Validation failed') } } },
    '/api/auth/login': { post: { tags: ['Auth'], summary: 'Log in and receive a JWT', requestBody: json({ $ref: '#/components/schemas/Credentials' }), responses: { 200: ok('Token and user'), 401: ok('Invalid email or password') } } },
    '/api/auth/me': { get: { tags: ['Auth'], summary: 'Current user', security: bearer, responses: { 200: ok('User'), 401: ok('Not authenticated') } } },
    '/api/auth/logout': { post: { tags: ['Auth'], summary: 'Log out (recorded in audit log)', security: bearer, responses: { 200: ok('Logged out') } } },

    '/api/requests': {
      get: {
        tags: ['Service Requests'], summary: 'List requests (search, filter, paginate; scoped by role)', security: bearer,
        parameters: ['q', 'status', 'category', 'priority', 'page', 'limit'].map((n) => ({ name: n, in: 'query', schema: { type: 'string' } })),
        responses: { 200: ok('Paged list') },
      },
      post: {
        tags: ['Service Requests'], summary: 'Submit a request (Student/Staff or Admin)', security: bearer,
        requestBody: { required: true, content: { 'multipart/form-data': { schema: { $ref: '#/components/schemas/ServiceRequestInput' } } } },
        responses: { 201: ok('Created'), 403: ok('Wrong role'), 422: ok('Validation failed') },
      },
    },
    '/api/requests/stats': { get: { tags: ['Service Requests'], summary: 'Counts per status for the dashboard', security: bearer, responses: { 200: ok('Stats') } } },
    '/api/requests/{id}': {
      get: { tags: ['Service Requests'], summary: 'Request detail with timeline', security: bearer, parameters: [idParam], responses: { 200: ok('Request'), 403: ok('No access'), 404: ok('Not found') } },
      put: { tags: ['Service Requests'], summary: 'Edit (owner while pending, or admin)', security: bearer, parameters: [idParam], requestBody: json({ type: 'object', properties: { title: { type: 'string' }, description: { type: 'string' }, location: { type: 'string' }, priority: { type: 'string' } } }), responses: { 200: ok('Updated'), 403: ok('Not allowed') } },
      delete: { tags: ['Service Requests'], summary: 'Delete (admin)', security: bearer, parameters: [idParam], responses: { 200: ok('Deleted') } },
    },
    '/api/requests/{id}/status': { patch: { tags: ['Service Requests'], summary: 'Update status (officer: in_progress/completed; owner: cancel; admin: any)', security: bearer, parameters: [idParam], requestBody: json({ type: 'object', required: ['status'], properties: { status: { type: 'string', enum: ['pending', 'assigned', 'in_progress', 'completed', 'cancelled'] }, comment: { type: 'string' } } }), responses: { 200: ok('Status changed'), 403: ok('Not allowed') } } },
    '/api/requests/{id}/assign': { patch: { tags: ['Service Requests'], summary: 'Assign to a maintenance officer (admin)', security: bearer, parameters: [idParam], requestBody: json({ type: 'object', required: ['officerId'], properties: { officerId: { type: 'string' }, note: { type: 'string' } } }), responses: { 200: ok('Assigned'), 422: ok('Not an officer') } } },

    '/api/users': {
      get: { tags: ['Users (Admin)'], summary: 'List users', security: bearer, parameters: ['q', 'role', 'page', 'limit'].map((n) => ({ name: n, in: 'query', schema: { type: 'string' } })), responses: { 200: ok('Paged list') } },
      post: { tags: ['Users (Admin)'], summary: 'Create a user of any role', security: bearer, requestBody: json({ allOf: [{ $ref: '#/components/schemas/Register' }, { type: 'object', properties: { role: { type: 'string', enum: ['user', 'officer', 'admin'] } } }] }), responses: { 201: ok('Created') } },
    },
    '/api/users/officers': { get: { tags: ['Users (Admin)'], summary: 'Active officers with open-job counts', security: bearer, responses: { 200: ok('Officers') } } },
    '/api/users/{id}': {
      put: { tags: ['Users (Admin)'], summary: 'Update user / role / active flag', security: bearer, parameters: [idParam], requestBody: json({ type: 'object', properties: { name: { type: 'string' }, role: { type: 'string' }, isActive: { type: 'boolean' } } }), responses: { 200: ok('Updated') } },
      delete: { tags: ['Users (Admin)'], summary: 'Delete user (only if no requests on record)', security: bearer, parameters: [idParam], responses: { 200: ok('Deleted'), 409: ok('Has requests') } },
    },

    '/api/categories': {
      get: { tags: ['Categories'], summary: 'List categories', security: bearer, responses: { 200: ok('Categories') } },
      post: { tags: ['Categories'], summary: 'Create (admin)', security: bearer, requestBody: json({ type: 'object', properties: { name: { type: 'string' }, description: { type: 'string' } } }), responses: { 201: ok('Created') } },
    },
    '/api/categories/{id}': {
```

```
      put: { tags: ['Categories'], summary: 'Update (admin)', security: bearer, parameters: [idParam], responses: { 200: ok('Updated') } },
      delete: { tags: ['Categories'], summary: 'Delete (admin, unused only)', security: bearer, parameters: [idParam], responses: { 200: ok('Deleted') } },
    },

    '/api/notifications': { get: { tags: ['Notifications'], summary: 'My notifications + unread count', security: bearer, responses: { 200: ok('List') } } },
    '/api/notifications/read-all': { patch: { tags: ['Notifications'], summary: 'Mark all read', security: bearer, responses: { 200: ok('Done') } } },
    '/api/notifications/{id}/read': { patch: { tags: ['Notifications'], summary: 'Mark one read', security: bearer, parameters: [idParam], responses: { 200: ok('Done') } } },

    '/api/reports/summary': { get: { tags: ['Reports (Admin)'], summary: 'Totals by status, category, officer; average resolution time', security: bearer, responses: { 200: ok('Summary') } } },
    '/api/reports/export': { get: { tags: ['Reports (Admin)'], summary: 'Export requests as CSV (accepts the same filters as the list)', security: bearer, parameters: ['q', 'status', 'category', 'priority'].map((n) => ({ name: n, in: 'query', schema: { type: 'string' } })), responses: { 200: { description: 'CSV file', content: { 'text/csv': {} } } } } },
    '/api/reports/audit': { get: { tags: ['Reports (Admin)'], summary: 'Audit trail / activity log', security: bearer, parameters: [{ name: 'page', in: 'query', schema: { type: 'integer' } }, { name: 'action', in: 'query', schema: { type: 'string' } }], responses: { 200: ok('Paged log') } } },
    '/api/health': { get: { tags: ['Auth'], summary: 'Health check (API + database)', responses: { 200: ok('OK') } } },
  },
};
```

server/src/middleware/auth.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const ApiError = require('../utils/ApiError');

// Verifies the JWT in the "Authorization: Bearer <token>" header
async function protect(req, _res, next) {
  try {
    const header = req.headers.authorization || '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : null;
    if (!token) throw new ApiError(401, 'Not authenticated. Please log in.');

    const payload = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(payload.id).populate('role');
    if (!user || !user.isActive) throw new ApiError(401, 'Account not found or deactivated.');

    req.user = user;
    next();
  } catch (err) {
    if (err.name === 'JsonWebTokenError' || err.name === 'TokenExpiredError') {
      return next(new ApiError(401, 'Session expired or invalid. Please log in again.'));
    }
    next(err);
  }
}

// Role-based access control: authorize('admin'), authorize('officer', 'admin') ...
function authorize(...roles) {
  return (req, _res, next) => {
    if (!req.user || !roles.includes(req.user.role.name)) {
      return next(new ApiError(403, 'You do not have permission to perform this action.'));
    }
    next();
  };
}

module.exports = { protect, authorize };
```

server/src/middleware/error.js

```
const ApiError = require('../utils/ApiError');

function notFound(req, _res, next) {
  next(new ApiError(404, `Route not found: ${req.method} ${req.originalUrl}`));
}

// Central error handler: every error leaves the API in the same JSON shape
// eslint-disable-next-line no-unused-vars
function errorHandler(err, _req, res, _next) {
  let status = err.statusCode || 500;
  let message = err.message || 'Server error';
  let errors = err.errors;

  if (err.name === 'ValidationError') {        // Mongoose schema validation
    status = 422;
```

```
    message = 'Validation failed';
    errors = Object.values(err.errors).map((e) => ({ field: e.path, message: e.message }));
  } else if (err.name === 'CastError') {       // Bad ObjectId
    status = 400;
    message = `Invalid ${err.path}`;
    errors = undefined;
  } else if (err.code === 11000) {             // Duplicate key
    status = 409;
    message = `${Object.keys(err.keyValue || {}).join(', ') || 'Value'} already exists`;
  } else if (err.name === 'MulterError') {
    status = 400;
  }

  if (status >= 500 && process.env.NODE_ENV !== 'test') console.error(err);
  res.status(status).json({ success: false, message, ...(errors && { errors }) });
}

module.exports = { notFound, errorHandler };
```

server/src/middleware/upload.js

```
const multer = require('multer');
const path = require('path');
const ApiError = require('../utils/ApiError');

const storage = multer.diskStorage({
  destination: (_req, _file, cb) => cb(null, path.join(__dirname, '..', '..', 'uploads')),
  filename: (_req, file, cb) => {
    const safe = file.originalname.replace(/[^a-zA-Z0-9.\-_]/g, '_');
    cb(null, `${Date.now()}-${safe}`);
  },
});

// Only images (evidence photos) and PDFs, max 5 MB each, max 3 files
const upload = multer({
  storage,
  limits: { fileSize: 5 * 1024 * 1024, files: 3 },
  fileFilter: (_req, file, cb) => {
    const ok = /^(image\/(jpeg|png|webp|gif)|application\/pdf)$/.test(file.mimetype);
    cb(ok ? null : new ApiError(400, 'Only JPG, PNG, WEBP, GIF or PDF files are allowed'), ok);
  },
});

module.exports = upload;
```

server/src/middleware/validate.js

```
const { validationResult } = require('express-validator');
const ApiError = require('../utils/ApiError');

// Runs after express-validator rules and returns a 422 with field errors
function validate(req, _res, next) {
  const result = validationResult(req);
  if (result.isEmpty()) return next();
  const errors = result.array().map((e) => ({ field: e.path, message: e.msg }));
  next(new ApiError(422, 'Validation failed', errors));
}

module.exports = validate;
```

server/src/models/Assignment.js

```
const mongoose = require('mongoose');

// Keeps the history of who was assigned which request, by whom and when
const assignmentSchema = new mongoose.Schema(
  {
    request: { type: mongoose.Schema.Types.ObjectId, ref: 'ServiceRequest', required: true },
    officer: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    assignedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    note: { type: String, trim: true },
    active: { type: Boolean, default: true }, // false once re-assigned
  },
  { timestamps: true }
);

module.exports = mongoose.model('Assignment', assignmentSchema);
```

server/src/models/AuditLog.js

```
const mongoose = require('mongoose');

// System-wide activity trail (logins, creates, updates, deletes, assignments)
const auditLogSchema = new mongoose.Schema(
  {
    actor: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    action: { type: String, required: true }, // e.g. REQUEST_CREATED
    entity: { type: String },                 // e.g. ServiceRequest
    entityId: { type: mongoose.Schema.Types.ObjectId },
    details: { type: String },
    ip: { type: String },
  },
  { timestamps: true }
);

module.exports = mongoose.model('AuditLog', auditLogSchema);
```

server/src/models/Category.js

```
const mongoose = require('mongoose');

const categorySchema = new mongoose.Schema(
  {
    name: { type: String, required: true, unique: true, trim: true },
    description: { type: String, trim: true },
    isActive: { type: Boolean, default: true },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Category', categorySchema);
```

server/src/models/Notification.js

```
const mongoose = require('mongoose');

const notificationSchema = new mongoose.Schema(
  {
    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    message: { type: String, required: true },
    request: { type: mongoose.Schema.Types.ObjectId, ref: 'ServiceRequest' },
    read: { type: Boolean, default: false },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Notification', notificationSchema);
```

server/src/models/Role.js

```
const mongoose = require('mongoose');

// Roles are stored as their own collection so permissions can be extended later
const roleSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      unique: true,
      enum: ['user', 'officer', 'admin'],
    },
    label: { type: String, required: true }, // e.g. "Student/Staff"
    permissions: [{ type: String }],
  },
  { timestamps: true }
);

module.exports = mongoose.model('Role', roleSchema);
```

server/src/models/ServiceRequest.js

```
const mongoose = require('mongoose');

const STATUSES = ['pending', 'assigned', 'in_progress', 'completed', 'cancelled'];
const PRIORITIES = ['low', 'medium', 'high', 'urgent'];

const serviceRequestSchema = new mongoose.Schema(
  {
    ticketNo: { type: String, unique: true },
    title: { type: String, required: true, trim: true, maxlength: 120 },
    description: { type: String, required: true, trim: true, maxlength: 2000 },
```

```
    category: { type: mongoose.Schema.Types.ObjectId, ref: 'Category', required: true },
    location: { type: String, required: true, trim: true }, // e.g. "Hostel B, Room 12"
    priority: { type: String, enum: PRIORITIES, default: 'medium' },
    status: { type: String, enum: STATUSES, default: 'pending' },
    requester: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    assignedTo: { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
    attachments: [{ filename: String, url: String, mimetype: String }],
    completedAt: Date,
  },
  { timestamps: true }
);

// Generate a human-friendly ticket number, e.g. REQ-2026-00042
serviceRequestSchema.pre('save', async function makeTicket() {
  if (this.ticketNo) return;
  // Use the highest existing number (not a count) so deletions never cause duplicates
  const last = await this.constructor.findOne({ ticketNo: { $exists: true } }).sort({ ticketNo: -1 }).select('
  ↪ ticketNo');
  const lastNo = last ? parseInt(last.ticketNo.split('-').pop(), 10) : 0;
  this.ticketNo = `REQ-${new Date().getFullYear()}-${String(lastNo + 1).padStart(5, '0')}`;
});

module.exports = mongoose.model('ServiceRequest', serviceRequestSchema);
module.exports.STATUSES = STATUSES;
module.exports.PRIORITIES = PRIORITIES;
```

server/src/models/StatusLog.js

```
const mongoose = require('mongoose');

// One entry every time a request changes status (powers the tracking timeline)
const statusLogSchema = new mongoose.Schema(
  {
    request: { type: mongoose.Schema.Types.ObjectId, ref: 'ServiceRequest', required: true },
    fromStatus: { type: String },
    toStatus: { type: String, required: true },
    comment: { type: String, trim: true },
    changedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  },
  { timestamps: true }
);

module.exports = mongoose.model('StatusLog', statusLogSchema);
```

server/src/models/User.js

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    password: { type: String, required: true, minlength: 6, select: false },
    role: { type: mongoose.Schema.Types.ObjectId, ref: 'Role', required: true },
    department: { type: String, trim: true },
    phone: { type: String, trim: true },
    isActive: { type: Boolean, default: true },
  },
  { timestamps: true }
);

// Hash the password before saving (only when it has changed)
userSchema.pre('save', async function hashPassword() {
  if (!this.isModified('password')) return;
  this.password = await bcrypt.hash(this.password, 12);
});

userSchema.methods.comparePassword = function comparePassword(plain) {
  return bcrypt.compare(plain, this.password);
};

// Never send the password hash to the client
userSchema.set('toJSON', {
  transform: (_doc, ret) => {
    delete ret.password;
    delete ret.__v;
    return ret;
  },
});

module.exports = mongoose.model('User', userSchema);
```

server/src/routes/auth.js

```
const router = require('express').Router();
const jwt = require('jsonwebtoken');
const { body } = require('express-validator');
const rateLimit = require('express-rate-limit');
const User = require('../models/User');
const Role = require('../models/Role');
const validate = require('../middleware/validate');
const { protect } = require('../middleware/auth');
const asyncHandler = require('../utils/asyncHandler');
const ApiError = require('../utils/ApiError');
const { audit } = require('../utils/activity');

const signToken = (user) =>
  jwt.sign({ id: user._id, role: user.role.name }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN || '1d',
  });

// Slow down brute-force login attempts (20 tries per 15 minutes per IP)
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: process.env.NODE_ENV === 'test' ? 1000 : 20,
  message: { success: false, message: 'Too many login attempts. Try again in 15 minutes.' },
});

// POST /api/auth/register  — self-registration is always as Student/Staff
router.post(
  '/register',
  [
    body('name').trim().notEmpty().withMessage('Name is required'),
    body('email').isEmail().withMessage('A valid email is required').normalizeEmail(),
    body('password')
      .isLength({ min: 8 }).withMessage('Password must be at least 8 characters')
      .matches(/\d/).withMessage('Password must contain a number'),
    body('department').optional().trim(),
    body('phone').optional().trim(),
  ],
  validate,
  asyncHandler(async (req, res) => {
    const { name, email, password, department, phone } = req.body;
    if (await User.exists({ email })) throw new ApiError(409, 'Email is already registered');

    const role = await Role.findOne({ name: 'user' });
    const user = await User.create({ name, email, password, department, phone, role: role._id });
    await user.populate('role');

    req.user = user;
    await audit(req, 'USER_REGISTERED', 'User', user._id, email);
    res.status(201).json({ success: true, token: signToken(user), user });
  })
);

// POST /api/auth/login
router.post(
  '/login',
  loginLimiter,
  [
    body('email').isEmail().withMessage('A valid email is required').normalizeEmail(),
    body('password').notEmpty().withMessage('Password is required'),
  ],
  validate,
  asyncHandler(async (req, res) => {
    const user = await User.findOne({ email: req.body.email }).select('+password').populate('role');
    if (!user || !(await user.comparePassword(req.body.password))) {
      throw new ApiError(401, 'Invalid email or password');
    }
    if (!user.isActive) throw new ApiError(403, 'Your account has been deactivated');

    req.user = user;
    await audit(req, 'USER_LOGIN', 'User', user._id);
    res.json({ success: true, token: signToken(user), user });
  })
);

// GET /api/auth/me — returns the logged-in user (used to restore a session)
router.get('/me', protect, (req, res) => res.json({ success: true, user: req.user }));

// POST /api/auth/logout — JWT is stateless; the client discards the token
router.post(
  '/logout',
  protect,
  asyncHandler(async (req, res) => {
```

```
    await audit(req, 'USER_LOGOUT', 'User', req.user._id);
    res.json({ success: true, message: 'Logged out' });
  })
);

module.exports = router;
```

server/src/routes/categories.js

```
const router = require('express').Router();
const { body } = require('express-validator');
const Category = require('../models/Category');
const ServiceRequest = require('../models/ServiceRequest');
const { protect, authorize } = require('../middleware/auth');
const validate = require('../middleware/validate');
const asyncHandler = require('../utils/asyncHandler');
const ApiError = require('../utils/ApiError');
const { audit } = require('../utils/activity');

router.use(protect);

// GET /api/categories — any logged-in user (fills the request form dropdown)
router.get('/', asyncHandler(async (req, res) => {
  const filter = req.user.role.name === 'admin' ? {} : { isActive: true };
  res.json({ success: true, data: await Category.find(filter).sort({ name: 1 }) });
}));

router.post(
  '/',
  authorize('admin'),
  [body('name').trim().notEmpty().withMessage('Name is required'), body('description').optional().trim()],
  validate,
  asyncHandler(async (req, res) => {
    const cat = await Category.create(req.body);
    await audit(req, 'CATEGORY_CREATED', 'Category', cat._id, cat.name);
    res.status(201).json({ success: true, message: 'Category created', data: cat });
  })
);

router.put(
  '/:id',
  authorize('admin'),
  [body('name').optional().trim().notEmpty(), body('isActive').optional().isBoolean().toBoolean()],
  validate,
  asyncHandler(async (req, res) => {
    const cat = await Category.findByIdAndUpdate(req.params.id, req.body, { new: true, runValidators: true });
    if (!cat) throw new ApiError(404, 'Category not found');
    await audit(req, 'CATEGORY_UPDATED', 'Category', cat._id, cat.name);
    res.json({ success: true, message: 'Category updated', data: cat });
  })
);

router.delete(
  '/:id',
  authorize('admin'),
  asyncHandler(async (req, res) => {
    if (await ServiceRequest.exists({ category: req.params.id })) {
      throw new ApiError(409, 'Category is in use — deactivate it instead');
    }
    const cat = await Category.findByIdAndDelete(req.params.id);
    if (!cat) throw new ApiError(404, 'Category not found');
    await audit(req, 'CATEGORY_DELETED', 'Category', cat._id, cat.name);
    res.json({ success: true, message: 'Category deleted' });
  })
);

module.exports = router;
```

server/src/routes/notifications.js

```
const router = require('express').Router();
const Notification = require('../models/Notification');
const { protect } = require('../middleware/auth');
const asyncHandler = require('../utils/asyncHandler');

router.use(protect);

// GET /api/notifications — latest 20 for the logged-in user + unread count
router.get('/', asyncHandler(async (req, res) => {
  const [items, unread] = await Promise.all([
    Notification.find({ user: req.user._id }).sort({ createdAt: -1 }).limit(20),
    Notification.countDocuments({ user: req.user._id, read: false }),
  ]);
```

```
  res.json({ success: true, data: items, unread });
}));

// PATCH /api/notifications/read-all
router.patch('/read-all', asyncHandler(async (req, res) => {
  await Notification.updateMany({ user: req.user._id, read: false }, { read: true });
  res.json({ success: true, message: 'All notifications marked as read' });
}));

// PATCH /api/notifications/:id/read
router.patch('/:id/read', asyncHandler(async (req, res) => {
  await Notification.updateOne({ _id: req.params.id, user: req.user._id }, { read: true });
  res.json({ success: true });
}));

module.exports = router;
```

server/src/routes/reports.js

```
const router = require('express').Router();
const ServiceRequest = require('../models/ServiceRequest');
const { STATUSES } = require('../models/ServiceRequest');
const Category = require('../models/Category');
const User = require('../models/User');
const Role = require('../models/Role');
const AuditLog = require('../models/AuditLog');
const { protect, authorize } = require('../middleware/auth');
const asyncHandler = require('../utils/asyncHandler');
const { buildFilter } = require('./requests');
const { audit } = require('../utils/activity');

router.use(protect, authorize('admin'));

// GET /api/reports/summary — numbers for the admin reports page
router.get('/summary', asyncHandler(async (_req, res) => {
  const statusCounts = await Promise.all(STATUSES.map((s) => ServiceRequest.countDocuments({ status: s })));
  const byStatus = Object.fromEntries(STATUSES.map((s, i) => [s, statusCounts[i]]));

  const categories = await Category.find().sort({ name: 1 });
  const byCategory = await Promise.all(
    categories.map(async (c) => ({ name: c.name, count: await ServiceRequest.countDocuments({ category: c._id
↪ }) }))
  );

  const officerRole = await Role.findOne({ name: 'officer' });
  const officers = await User.find({ role: officerRole._id }).sort({ name: 1 });
  const byOfficer = await Promise.all(
    officers.map(async (o) => ({
      name: o.name,
      open: await ServiceRequest.countDocuments({ assignedTo: o._id, status: { $in: ['assigned', 'in_progress'
↪ ] } }),
      completed: await ServiceRequest.countDocuments({ assignedTo: o._id, status: 'completed' }),
    }))
  );

  // Average time from submission to completion, in hours
  const done = await ServiceRequest.find({ status: 'completed', completedAt: { $exists: true } }).select('crea
↪ tedAt completedAt');
  const avgResolutionHours = done.length
    ? +(done.reduce((sum, r) => sum + (r.completedAt - r.createdAt), 0) / done.length / 36e5).toFixed(1)
    : null;

  res.json({
    success: true,
    data: { total: statusCounts.reduce((a, b) => a + b, 0), byStatus, byCategory, byOfficer, avgResolutionHour
↪ s },
  });
}));

// GET /api/reports/export?format=csv&status=&category=&q= — download requests as CSV
router.get('/export', asyncHandler(async (req, res) => {
  const rows = await ServiceRequest.find(buildFilter(req.user, req.query))
    .sort({ createdAt: -1 })
    .populate('category', 'name')
    .populate('requester', 'name email')
    .populate('assignedTo', 'name');

  const esc = (v) => `"${String(v ?? '').replace(/"/g, '""')}"`;
  const header = ['Ticket', 'Title', 'Category', 'Location', 'Priority', 'Status', 'Requester', 'Requester Ema
↪ il', 'Assigned To', 'Created', 'Completed'];
  const lines = rows.map((r) => [
    r.ticketNo, r.title, r.category?.name, r.location, r.priority, r.status,
    r.requester?.name, r.requester?.email, r.assignedTo?.name || '',
```

```
    r.createdAt.toISOString(), r.completedAt ? r.completedAt.toISOString() : '',
  ].map(esc).join(','));

  await audit(req, 'REPORT_EXPORTED', 'ServiceRequest', undefined, `${rows.length} rows`);
  res.setHeader('Content-Type', 'text/csv');
  res.setHeader('Content-Disposition', `attachment; filename="service-requests-${Date.now()}.csv"`);
  res.send([header.map(esc).join(','), ...lines].join('\n'));
}));

// GET /api/reports/audit?page= — the activity log
router.get('/audit', asyncHandler(async (req, res) => {
  const page = Math.max(parseInt(req.query.page, 10) || 1, 1);
  const limit = 15;
  const filter = req.query.action ? { action: req.query.action } : {};
  const [items, total] = await Promise.all([
    AuditLog.find(filter).sort({ createdAt: -1 }).skip((page - 1) * limit).limit(limit).populate('actor', 'nam
↪ e email'),
    AuditLog.countDocuments(filter),
  ]);
  res.json({ success: true, data: items, pagination: { page, limit, total, pages: Math.ceil(total / limit) ||
↪ 1 } });
}));

module.exports = router;
```

server/src/routes/requests.js

```
const router = require('express').Router();
const { body, query } = require('express-validator');
const ServiceRequest = require('../models/ServiceRequest');
const { STATUSES, PRIORITIES } = require('../models/ServiceRequest');
const StatusLog = require('../models/StatusLog');
const Assignment = require('../models/Assignment');
const Category = require('../models/Category');
const User = require('../models/User');
const { protect, authorize } = require('../middleware/auth');
const validate = require('../middleware/validate');
const upload = require('../middleware/upload');
const asyncHandler = require('../utils/asyncHandler');
const ApiError = require('../utils/ApiError');
const { audit, notify, broadcastRequest } = require('../utils/activity');

router.use(protect);

const escapeRegex = (s) => s.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

// Which requests may this user see? (role-based data scoping)
function scopeFor(user) {
  if (user.role.name === 'admin') return {};
  if (user.role.name === 'officer') return { assignedTo: user._id };
  return { requester: user._id };
}

// Build a MongoDB filter from ?q=&status=&category=&priority=
function buildFilter(user, qs) {
  const filter = scopeFor(user);
  if (qs.status) filter.status = qs.status;
  if (qs.category) filter.category = qs.category;
  if (qs.priority) filter.priority = qs.priority;
  if (qs.q) {
    const rx = new RegExp(escapeRegex(qs.q), 'i');
    filter.$or = [{ title: rx }, { ticketNo: rx }, { location: rx }, { description: rx }];
  }
  return filter;
}

async function loadRequestForUser(id, user) {
  const request = await ServiceRequest.findById(id)
    .populate('category', 'name')
    .populate('requester', 'name email department phone')
    .populate('assignedTo', 'name email phone');
  if (!request) throw new ApiError(404, 'Service request not found');

  const role = user.role.name;
  const isOwner = String(request.requester._id) === String(user._id);
  const isAssignee = request.assignedTo && String(request.assignedTo._id) === String(user._id);
  if (role !== 'admin' && !isOwner && !isAssignee) {
    throw new ApiError(403, 'You do not have access to this request');
  }
  return request;
}

const listValidators = [
```

```
  query('page').optional().isInt({ min: 1 }).toInt(),
  query('limit').optional().isInt({ min: 1, max: 100 }).toInt(),
  query('status').optional().isIn(STATUSES).withMessage('Invalid status'),
  query('priority').optional().isIn(PRIORITIES).withMessage('Invalid priority'),
  query('category').optional().isMongoId().withMessage('Invalid category'),
];

// GET /api/requests — search, filter, paginate (scoped by role)
router.get(
  '/',
  listValidators,
  validate,
  asyncHandler(async (req, res) => {
    const page = req.query.page || 1;
    const limit = req.query.limit || 10;
    const filter = buildFilter(req.user, req.query);

    const [items, total] = await Promise.all([
      ServiceRequest.find(filter)
        .sort({ createdAt: -1 })
        .skip((page - 1) * limit)
        .limit(limit)
        .populate('category', 'name')
        .populate('requester', 'name email')
        .populate('assignedTo', 'name email'),
      ServiceRequest.countDocuments(filter),
    ]);

    res.json({ success: true, data: items, pagination: { page, limit, total, pages: Math.ceil(total / limit) | 1 } });
  })
);

// GET /api/requests/stats — counts per status for the dashboard cards
router.get(
  '/stats',
  asyncHandler(async (req, res) => {
    const scope = scopeFor(req.user);
    const counts = await Promise.all(STATUSES.map((s) => ServiceRequest.countDocuments({ ...scope, status: s })));
    const byStatus = Object.fromEntries(STATUSES.map((s, i) => [s, counts[i]]));
    const total = counts.reduce((a, b) => a + b, 0);
    res.json({ success: true, data: { total, byStatus } });
  })
);

// POST /api/requests — submit a new service request (with optional evidence files)
router.post(
  '/',
  authorize('user', 'admin'),
  upload.array('attachments', 3),
  [
    body('title').trim().isLength({ min: 5, max: 120 }).withMessage('Title must be 5–120 characters'),
    body('description').trim().isLength({ min: 10 }).withMessage('Describe the problem (at least 10 characters)'),
    body('category').isMongoId().withMessage('Choose a category'),
    body('location').trim().notEmpty().withMessage('Location is required'),
    body('priority').optional().isIn(PRIORITIES).withMessage('Invalid priority'),
  ],
  validate,
  asyncHandler(async (req, res) => {
    if (!(await Category.exists({ _id: req.body.category, isActive: true }))) {
      throw new ApiError(422, 'Selected category does not exist');
    }
    const attachments = (req.files || []).map((f) => ({
      filename: f.originalname,
      url: `/uploads/${f.filename}`,
      mimetype: f.mimetype,
    }));

    const request = await ServiceRequest.create({
      title: req.body.title,
      description: req.body.description,
      category: req.body.category,
      location: req.body.location,
      priority: req.body.priority || 'medium',
      requester: req.user._id,
      attachments,
    });

    await StatusLog.create({ request: request._id, toStatus: 'pending', comment: 'Request submitted', changedBy: req.user._id });
    await audit(req, 'REQUEST_CREATED', 'ServiceRequest', request._id, request.ticketNo);
```

```
136
137     // Tell every admin there is a new request to triage
138     const admins = await User.find({ isActive: true }).populate('role');
139     await Promise.all(
140       admins.filter((u) => u.role.name === 'admin')
141         .map((a) => notify(a._id, `New request ${request.ticketNo}: ${request.title}`, request._id))
142     );
143
144     res.status(201).json({ success: true, message: 'Request submitted successfully', data: request });
145   })
146 );
147
148 // GET /api/requests/:id — full detail with timeline and assignment history
149 router.get(
150   '/:id',
151   asyncHandler(async (req, res) => {
152     const request = await loadRequestForUser(req.params.id, req.user);
153     const [timeline, assignments] = await Promise.all([
154       StatusLog.find({ request: request._id }).sort({ createdAt: 1 }).populate('changedBy', 'name'),
155       Assignment.find({ request: request._id }).sort({ createdAt: -1 })
156         .populate('officer', 'name').populate('assignedBy', 'name'),
157     ]);
158     res.json({ success: true, data: { ...request.toObject(), timeline, assignments } });
159   })
160 );
161
162 // PUT /api/requests/:id — edit details (owner while still pending, or admin)
163 router.put(
164   '/:id',
165   [
166     body('title').optional().trim().isLength({ min: 5, max: 120 }).withMessage('Title must be 5–120 characters
    ↪ '),
167     body('description').optional().trim().isLength({ min: 10 }).withMessage('Description too short'),
168     body('category').optional().isMongoId().withMessage('Invalid category'),
169     body('location').optional().trim().notEmpty().withMessage('Location cannot be empty'),
170     body('priority').optional().isIn(PRIORITIES).withMessage('Invalid priority'),
171   ],
172   validate,
173   asyncHandler(async (req, res) => {
174     const request = await loadRequestForUser(req.params.id, req.user);
175     const isAdmin = req.user.role.name === 'admin';
176     const isOwner = String(request.requester._id) === String(req.user._id);
177     if (!isAdmin && !(isOwner && request.status === 'pending')) {
178       throw new ApiError(403, 'Only pending requests can be edited by their owner');
179     }
180
181     ['title', 'description', 'category', 'location', 'priority'].forEach((f) => {
182       if (req.body[f] !== undefined) request[f] = req.body[f];
183     });
184     await request.save();
185     await audit(req, 'REQUEST_UPDATED', 'ServiceRequest', request._id, request.ticketNo);
186     res.json({ success: true, message: 'Request updated', data: request });
187   })
188 );
189
190 // PATCH /api/requests/:id/status — progress updates
191 //   officer: in_progress | completed (only on their own assignments)
192 //   owner:   cancelled (only while pending)
193 //   admin:   any status
194 router.patch(
195   '/:id/status',
196   [
197     body('status').isIn(STATUSES).withMessage('Invalid status'),
198     body('comment').optional().trim().isLength({ max: 500 }),
199   ],
200   validate,
201   asyncHandler(async (req, res) => {
202     const request = await loadRequestForUser(req.params.id, req.user);
203     const role = req.user.role.name;
204     const next = req.body.status;
205     const prev = request.status;
206
207     if (prev === next) throw new ApiError(400, `Request is already ${next}`);
208     if (['completed', 'cancelled'].includes(prev) && role !== 'admin') {
209       throw new ApiError(400, 'This request is closed');
210     }
211     if (role === 'officer' && !['in_progress', 'completed'].includes(next)) {
212       throw new ApiError(403, 'Officers can only mark requests as in progress or completed');
213     }
214     if (role === 'user' && !(next === 'cancelled' && prev === 'pending')) {
215       throw new ApiError(403, 'You can only cancel a request that is still pending');
216     }
217
```

```
    request.status = next;
    request.completedAt = next === 'completed' ? new Date() : undefined;
    await request.save();

    await StatusLog.create({ request: request._id, fromStatus: prev, toStatus: next, comment: req.body.comment
↪ , changedBy: req.user._id });
    await audit(req, 'STATUS_CHANGED', 'ServiceRequest', request._id, `${request.ticketNo}: ${prev} → ${next}`
↪ );

    const label = next.replace('_', ' ');
    if (String(request.requester._id) !== String(req.user._id)) {
      await notify(request.requester._id, `Your request ${request.ticketNo} is now ${label}`, request._id);
    }
    broadcastRequest(request);

    res.json({ success: true, message: `Status changed to ${label}`, data: request });
  })
);

// PATCH /api/requests/:id/assign — admin assigns (or re-assigns) a maintenance officer
router.patch(
  '/:id/assign',
  authorize('admin'),
  [body('officerId').isMongoId().withMessage('Choose an officer'), body('note').optional().trim()],
  validate,
  asyncHandler(async (req, res) => {
    const request = await loadRequestForUser(req.params.id, req.user);
    if (['completed', 'cancelled'].includes(request.status)) throw new ApiError(400, 'Cannot assign a closed r
↪ equest');

    const officer = await User.findById(req.body.officerId).populate('role');
    if (!officer || officer.role.name !== 'officer' || !officer.isActive) {
      throw new ApiError(422, 'Selected user is not an active maintenance officer');
    }

    await Assignment.updateMany({ request: request._id, active: true }, { active: false });
    await Assignment.create({ request: request._id, officer: officer._id, assignedBy: req.user._id, note: req.
↪ body.note });

    const prev = request.status;
    request.assignedTo = officer._id;
    if (prev === 'pending') request.status = 'assigned';
    await request.save();

    await StatusLog.create({
      request: request._id, fromStatus: prev, toStatus: request.status,
      comment: `Assigned to ${officer.name}${req.body.note ? ` — ${req.body.note}` : ''}`,
      changedBy: req.user._id,
    });
    await audit(req, 'REQUEST_ASSIGNED', 'ServiceRequest', request._id, `${request.ticketNo} → ${officer.email
↪ }`);
    await notify(officer._id, `New job assigned: ${request.ticketNo} — ${request.title}`, request._id);
    await notify(request.requester._id, `Your request ${request.ticketNo} has been assigned to an officer`, re
↪ quest._id);
    broadcastRequest(request);

    await request.populate('assignedTo', 'name email phone');
    res.json({ success: true, message: `Assigned to ${officer.name}`, data: request });
  })
);

// DELETE /api/requests/:id — admin only
router.delete(
  '/:id',
  authorize('admin'),
  asyncHandler(async (req, res) => {
    const request = await ServiceRequest.findById(req.params.id);
    if (!request) throw new ApiError(404, 'Service request not found');
    await Promise.all([
      StatusLog.deleteMany({ request: request._id }),
      Assignment.deleteMany({ request: request._id }),
      request.deleteOne(),
    ]);
    await audit(req, 'REQUEST_DELETED', 'ServiceRequest', request._id, request.ticketNo);
    res.json({ success: true, message: 'Request deleted' });
  })
);

module.exports = router;
module.exports.buildFilter = buildFilter;
```

server/src/routes/users.js

```
const router = require('express').Router();
const { body, query } = require('express-validator');
const User = require('../models/User');
const Role = require('../models/Role');
const ServiceRequest = require('../models/ServiceRequest');
const { protect, authorize } = require('../middleware/auth');
const validate = require('../middleware/validate');
const asyncHandler = require('../utils/asyncHandler');
const ApiError = require('../utils/ApiError');
const { audit } = require('../utils/activity');

router.use(protect, authorize('admin'));

const escapeRegex = (s) => s.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

// GET /api/users?q=&role=&page=&limit=
router.get(
  '/',
  [query('page').optional().isInt({ min: 1 }).toInt(), query('limit').optional().isInt({ min: 1, max: 100 }).toInt()],
  validate,
  asyncHandler(async (req, res) => {
    const page = req.query.page || 1;
    const limit = req.query.limit || 10;
    const filter = {};
    if (req.query.role) {
      const role = await Role.findOne({ name: req.query.role });
      filter.role = role ? role._id : null;
    }
    if (req.query.q) {
      const rx = new RegExp(escapeRegex(req.query.q), 'i');
      filter.$or = [{ name: rx }, { email: rx }, { department: rx }];
    }
    const [items, total] = await Promise.all([
      User.find(filter).sort({ createdAt: -1 }).skip((page - 1) * limit).limit(limit).populate('role'),
      User.countDocuments(filter),
    ]);
    res.json({ success: true, data: items, pagination: { page, limit, total, pages: Math.ceil(total / limit) || 1 } });
  })
);

// GET /api/users/officers — active officers with their open workload (for the assign dropdown)
router.get(
  '/officers',
  asyncHandler(async (_req, res) => {
    const role = await Role.findOne({ name: 'officer' });
    const officers = await User.find({ role: role._id, isActive: true }).sort({ name: 1 });
    const loads = await Promise.all(
      officers.map((o) => ServiceRequest.countDocuments({ assignedTo: o._id, status: { $in: ['assigned', 'in_progress'] } }))
    );
    res.json({ success: true, data: officers.map((o, i) => ({ ...o.toJSON(), openJobs: loads[i] })) });
  })
);

// GET /api/users/roles
router.get('/roles', asyncHandler(async (_req, res) => {
  res.json({ success: true, data: await Role.find().sort({ name: 1 }) });
}));

// POST /api/users — admin creates any type of account (e.g. maintenance officers)
router.post(
  '/',
  [
    body('name').trim().notEmpty().withMessage('Name is required'),
    body('email').isEmail().withMessage('A valid email is required').normalizeEmail(),
    body('password').isLength({ min: 8 }).withMessage('Password must be at least 8 characters'),
    body('role').isIn(['user', 'officer', 'admin']).withMessage('Invalid role'),
  ],
  validate,
  asyncHandler(async (req, res) => {
    if (await User.exists({ email: req.body.email })) throw new ApiError(409, 'Email is already registered');
    const role = await Role.findOne({ name: req.body.role });
    const user = await User.create({ ...req.body, role: role._id });
    await user.populate('role');
    await audit(req, 'USER_CREATED', 'User', user._id, `${user.email} (${role.name})`);
    res.status(201).json({ success: true, message: 'User created', data: user });
  })
);

```

```
// PUT /api/users/:id — change details, role or activate/deactivate
router.put(
  '/:id',
  [
    body('name').optional().trim().notEmpty(),
    body('role').optional().isIn(['user', 'officer', 'admin']).withMessage('Invalid role'),
    body('isActive').optional().isBoolean().toBoolean(),
  ],
  validate,
  asyncHandler(async (req, res) => {
    const user = await User.findById(req.params.id);
    if (!user) throw new ApiError(404, 'User not found');
    if (String(user._id) === String(req.user._id) && (req.body.isActive === false || (req.body.role && req.body.role !== 'admin'))) {
      throw new ApiError(400, 'You cannot deactivate or demote your own account');
    }
    ['name', 'department', 'phone', 'isActive'].forEach((f) => {
      if (req.body[f] !== undefined) user[f] = req.body[f];
    });
    if (req.body.role) user.role = (await Role.findOne({ name: req.body.role }))._id;
    await user.save();
    await user.populate('role');
    await audit(req, 'USER_UPDATED', 'User', user._id, user.email);
    res.json({ success: true, message: 'User updated', data: user });
  })
);

// DELETE /api/users/:id
router.delete(
  '/:id',
  asyncHandler(async (req, res) => {
    if (String(req.params.id) === String(req.user._id)) throw new ApiError(400, 'You cannot delete your own account');
    const user = await User.findById(req.params.id);
    if (!user) throw new ApiError(404, 'User not found');
    if (await ServiceRequest.exists({ $or: [{ requester: user._id }, { assignedTo: user._id }] })) {
      throw new ApiError(409, 'User has service requests on record — deactivate the account instead');
    }
    await user.deleteOne();
    await audit(req, 'USER_DELETED', 'User', user._id, user.email);
    res.json({ success: true, message: 'User deleted' });
  })
);

module.exports = router;
```

server/src/seed.js

```
// Populates the database with roles, categories, demo accounts and sample requests.
// Run with:  npm run seed
require('dotenv').config();
const mongoose = require('mongoose');
const Role = require('./models/Role');
const User = require('./models/User');
const Category = require('./models/Category');
const ServiceRequest = require('./models/ServiceRequest');
const Assignment = require('./models/Assignment');
const StatusLog = require('./models/StatusLog');
const Notification = require('./models/Notification');
const AuditLog = require('./models/AuditLog');

const ROLES = [
  { name: 'user', label: 'Student/Staff', permissions: ['request:create', 'request:read:own', 'request:cancel:own'] },
  { name: 'officer', label: 'Maintenance Officer', permissions: ['request:read:assigned', 'request:update-status'] },
  { name: 'admin', label: 'Administrator', permissions: ['*'] },
];

const CATEGORIES = [
  ['Electrical', 'Faulty sockets, lights, fans, power outages'],
  ['Furniture', 'Broken chairs, tables, beds, wardrobes'],
  ['Plumbing', 'Leaking pipes, blocked drains, toilets, taps'],
  ['Internet & Network', 'Wi-Fi outages, LAN ports, slow connection'],
  ['Classroom Equipment', 'Projectors, smart boards, PA systems, ACs'],
  ['Hostel Maintenance', 'Doors, windows, locks, painting, pests'],
];

async function seedReference() {
  for (const r of ROLES) await Role.updateOne({ name: r.name }, r, { upsert: true });
  for (const [name, description] of CATEGORIES) await Category.updateOne({ name }, { name, description }, { upsert: true });
}
```

```
33
34  async function run({ withDemo = true } = {}) {
35    await seedReference();
36    if (!withDemo) return;
37
38    await Promise.all([User, ServiceRequest, Assignment, StatusLog, Notification, AuditLog].map((m) => m.deleteM
  ↪ any({})));
39    const roles = Object.fromEntries((await Role.find()).map((r) => [r.name, r._id]));
40    const cats = Object.fromEntries((await Category.find()).map((c) => [c.name, c._id]));
41
42    const mk = (name, email, role, department) =>
43      User.create({ name, email, password: 'Password123', role: roles[role], department });
44    const admin = await mk('Grace Adeyemi', 'admin@campusfix.edu', 'admin', 'Works & Services');
45    const officer1 = await mk('Musa Ibrahim', 'officer@campusfix.edu', 'officer', 'Electrical Unit');
46    const officer2 = await mk('Chinedu Okafor', 'plumber@campusfix.edu', 'officer', 'Plumbing Unit');
47    const student = await mk('Ada Obi', 'student@campusfix.edu', 'user', 'Computer Science');
48    const staff = await mk('Dr. Tunde Bello', 'staff@campusfix.edu', 'user', 'Information Technology');
49
50    // [requester, category, title, description, location, priority, status, officer, daysAgo]
51    const samples = [
52      [student, 'Electrical', 'Sockets not working in room', 'None of the wall sockets in my room are working si
  ↪ nce yesterday evening.', 'Hostel B, Room 214', 'high', 'in_progress', officer1, 6],
53      [student, 'Plumbing', 'Leaking pipe in bathroom', 'Water is leaking from the pipe under the sink and flood
  ↪ ing the floor.', 'Hostel B, Block 2 Bathroom', 'urgent', 'assigned', officer2, 2],
54      [staff, 'Classroom Equipment', 'Projector not displaying', 'The projector in LT 3 powers on but shows no i
  ↪ nput from the HDMI cable.', 'Lecture Theatre 3', 'medium', 'completed', officer1, 9],
55      [staff, 'Internet & Network', 'No Wi-Fi in staff office', 'The Wi-Fi network is not visible in the IT depa
  ↪ rtment offices on floor 2.', 'ICT Building, Floor 2', 'high', 'pending', null, 1],
56      [student, 'Furniture', 'Broken reading chair', 'One of the chairs in the library reading area has a broken
  ↪  leg.', 'Main Library, Reading Room A', 'low', 'pending', null, 3],
57      [staff, 'Hostel Maintenance', 'Door lock jammed', 'The door lock to the staff quarters store room is jamme
  ↪ d.', 'Staff Quarters, Block C', 'medium', 'cancelled', null, 12],
58    ];
59
60    for (const [who, cat, title, description, location, priority, status, officer, daysAgo] of samples) {
61      const createdAt = new Date(Date.now() - daysAgo * 86400e3);
62      const req = await ServiceRequest.create({
63        title, description, location, priority, category: cats[cat], requester: who._id,
64        status, assignedTo: officer ? officer._id : null,
65        completedAt: status === 'completed' ? new Date(createdAt.getTime() + 26 * 3600e3) : undefined,
66      });
67      // back-date the sample so dashboards show realistic dates and fix times
68      await ServiceRequest.collection.updateOne({ _id: req._id }, { $set: { createdAt } });
69      await StatusLog.create({ request: req._id, toStatus: 'pending', comment: 'Request submitted', changedBy: w
  ↪ ho._id });
70      if (officer) {
71        await Assignment.create({ request: req._id, officer: officer._id, assignedBy: admin._id, note: 'Please a
  ↪ ttend promptly' });
72        await StatusLog.create({ request: req._id, fromStatus: 'pending', toStatus: 'assigned', comment: `Assign
  ↪ ed to ${officer.name}`, changedBy: admin._id });
73        await Notification.create({ user: officer._id, message: `New job assigned: ${req.ticketNo} — ${title}`,
  ↪ request: req._id });
74      }
75      if (['in_progress', 'completed'].includes(status)) {
76        await StatusLog.create({ request: req._id, fromStatus: 'assigned', toStatus: 'in_progress', comment: 'In
  ↪ spected — working on it', changedBy: officer._id });
77      }
78      if (status === 'completed') {
79        await StatusLog.create({ request: req._id, fromStatus: 'in_progress', toStatus: 'completed', comment: 'R
  ↪ eplaced faulty HDMI switch', changedBy: officer._id });
80        await Notification.create({ user: who._id, message: `Your request ${req.ticketNo} is now completed`, req
  ↪ uest: req._id });
81      }
82      if (status === 'cancelled') {
83        await StatusLog.create({ request: req._id, fromStatus: 'pending', toStatus: 'cancelled', comment: 'Resol
  ↪ ved on my own', changedBy: who._id });
84      }
85      await AuditLog.create({ actor: who._id, action: 'REQUEST_CREATED', entity: 'ServiceRequest', entityId: req
  ↪ ._id, details: req.ticketNo });
86    }
87    await Notification.create({ user: student._id, message: 'Your request has been assigned to an officer', read
  ↪ : false });
88  }
89
90  module.exports = { run, seedReference };
91
92  if (require.main === module) {
93    mongoose.connect(process.env.MONGO_URI)
94      .then(() => run())
95      .then(() => {
96        console.log('Seed complete. Demo logins (password: Password123):');
97        console.log('  admin@campusfix.edu | officer@campusfix.edu | plumber@campusfix.edu | student@campusfix.e
  ↪ du | staff@campusfix.edu');
```

```
      return mongoose.disconnect();
    })
    .catch((e) => { console.error(e); process.exit(1); });
}
```

server/src/server.js

```
require('dotenv').config();
const http = require('http');
const app = require('./app');
const connectDB = require('./config/db');
const { initSocket } = require('./utils/socket');

const PORT = process.env.PORT || 5000;

(async () => {
  if (!process.env.JWT_SECRET) throw new Error('JWT_SECRET is not set (see .env.example)');
  await connectDB(process.env.MONGO_URI);

  const server = http.createServer(app);
  initSocket(server, process.env.CLIENT_URL || '*');
  server.listen(PORT, () => console.log(`CampusFix API running on http://localhost:${PORT}  (docs: /api/docs)`
↳ ));
})().catch((err) => {
  console.error('Failed to start:', err.message);
  process.exit(1);
});
```

server/src/utils/ApiError.js

```
class ApiError extends Error {
  constructor(statusCode, message, errors) {
    super(message);
    this.statusCode = statusCode;
    this.errors = errors;
  }
}

module.exports = ApiError;
```

server/src/utils/activity.js

```
const AuditLog = require('../models/AuditLog');
const Notification = require('../models/Notification');
const { getIO } = require('./socket');

// Write an entry to the audit trail. Never let logging break the main request.
async function audit(req, action, entity, entityId, details) {
  try {
    await AuditLog.create({
      actor: req.user ? req.user._id : undefined,
      action,
      entity,
      entityId,
      details,
      ip: req.ip,
    });
  } catch (e) {
    console.error('Audit log failed:', e.message);
  }
}

// Save an in-app notification and push it live to the user over Socket.IO
async function notify(userId, message, requestId) {
  if (!userId) return;
  const note = await Notification.create({ user: userId, message, request: requestId });
  const io = getIO();
  if (io) io.to(`user:${userId}`).emit('notification', note);
  return note;
}

// Broadcast a status change to everyone watching that request
function broadcastRequest(request) {
  const io = getIO();
  if (io) io.emit('request:updated', { id: String(request._id), status: request.status });
}

module.exports = { audit, notify, broadcastRequest };
```

server/src/utils/asyncHandler.js

```
// Wraps async route handlers so rejected promises reach the error middleware
module.exports = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
```

server/src/utils/socket.js

```
const jwt = require('jsonwebtoken');
const { Server } = require('socket.io');

let io = null;

// Real-time layer: each logged-in browser joins a private room "user:<id>"
function initSocket(httpServer, corsOrigin) {
  io = new Server(httpServer, { cors: { origin: corsOrigin } });

  io.use((socket, next) => {
    try {
      const payload = jwt.verify(socket.handshake.auth.token, process.env.JWT_SECRET);
      socket.userId = payload.id;
      next();
    } catch {
      next(new Error('Unauthorized'));
    }
  });

  io.on('connection', (socket) => {
    socket.join(`user:${socket.userId}`);
  });

  return io;
}

const getIO = () => io;

module.exports = { initSocket, getIO };
```

server/tests/auth.test.js

```
const { app, request, startDb, stopDb, login } = require('./setup');

beforeAll(startDb);
afterAll(stopDb);

describe('Authentication API', () => {
  test('registers a new Student/Staff user and returns a JWT', async () => {
    const res = await request(app).post('/api/auth/register').send({
      name: 'Test Student', email: 'new.student@campusfix.edu', password: 'Secret123', department: 'Physics',
    });
    expect(res.status).toBe(201);
    expect(res.body.token).toBeDefined();
    expect(res.body.user.role.name).toBe('user');
    expect(res.body.user.password).toBeUndefined(); // hash never leaves the server
  });

  test('rejects a duplicate email with 409', async () => {
    const res = await request(app).post('/api/auth/register').send({
      name: 'Dup', email: 'student@campusfix.edu', password: 'Secret123',
    });
    expect(res.status).toBe(409);
  });

  test('validates registration input with 422 and field errors', async () => {
    const res = await request(app).post('/api/auth/register').send({ name: '', email: 'bad', password: 'short' });
    expect(res.status).toBe(422);
    const fields = res.body.errors.map((e) => e.field);
    expect(fields).toEqual(expect.arrayContaining(['name', 'email', 'password']));
  });

  test('logs in with correct credentials', async () => {
    const res = await request(app).post('/api/auth/login').send({ email: 'admin@campusfix.edu', password: 'Password123' });
    expect(res.status).toBe(200);
    expect(res.body.user.role.name).toBe('admin');
  });

  test('refuses a wrong password with 401', async () => {
    const res = await request(app).post('/api/auth/login').send({ email: 'admin@campusfix.edu', password: 'wrongpass1' });
    expect(res.status).toBe(401);
  });

```

```
  test('GET /me requires a valid token', async () => {
    expect((await request(app).get('/api/auth/me')).status).toBe(401);
    expect((await request(app).get('/api/auth/me').set('Authorization', 'Bearer not-a-token')).status).toBe(401);

    const token = await login('student@campusfix.edu');
    const res = await request(app).get('/api/auth/me').set('Authorization', `Bearer ${token}`);
    expect(res.status).toBe(200);
    expect(res.body.user.email).toBe('student@campusfix.edu');
  });

  test('a deactivated account cannot log in', async () => {
    const admin = await login('admin@campusfix.edu');
    const list = await request(app).get('/api/users?q=staff@').set('Authorization', `Bearer ${admin}`);
    const staffId = list.body.data[0]._id;
    await request(app).put(`/api/users/${staffId}`).set('Authorization', `Bearer ${admin}`).send({ isActive: false });

    const res = await request(app).post('/api/auth/login').send({ email: 'staff@campusfix.edu', password: 'Password123' });
    expect(res.status).toBe(403);
  });
});
```

server/tests/requests.test.js

```
const path = require('path');
const fs = require('fs');
const { app, request, startDb, stopDb, login } = require('./setup');

let student, officer, plumber, admin, categoryId, newId, officerId;
const auth = (t) => ({ Authorization: `Bearer ${t}` });

beforeAll(async () => {
  await startDb();
  [student, officer, plumber, admin] = await Promise.all([
    login('student@campusfix.edu'), login('officer@campusfix.edu'),
    login('plumber@campusfix.edu'), login('admin@campusfix.edu'),
  ]);
  const cats = await request(app).get('/api/categories').set(auth(student));
  categoryId = cats.body.data.find((c) => c.name === 'Electrical')._id;
  const officers = await request(app).get('/api/users/officers').set(auth(admin));
  officerId = officers.body.data.find((o) => o.email === 'officer@campusfix.edu')._id;
});
afterAll(stopDb);

describe('Service request CRUD', () => {
  test('student submits a request with an evidence photo', async () => {
    const img = path.join(__dirname, 'fixture.png');
    fs.writeFileSync(img, Buffer.from('89504E470D0A1A0A0000000D4948445200000001000000010806000000001F15C489000000D4944415478DA6300010000050001A5F645400000000049454E44AE426082', 'hex'));

    const res = await request(app).post('/api/requests').set(auth(student))
      .field('title', 'Bulb blown in corridor')
      .field('description', 'The corridor light outside room 210 has been off for two days.')
      .field('category', categoryId)
      .field('location', 'Hostel B, 2nd floor corridor')
      .field('priority', 'medium')
      .attach('attachments', img);

    expect(res.status).toBe(201);
    expect(res.body.data.ticketNo).toMatch(/^REQ-\d{4}-\d{5}$/);
    expect(res.body.data.status).toBe('pending');
    expect(res.body.data.attachments).toHaveLength(1);
    newId = res.body.data._id;
    fs.unlinkSync(img);
  });

  test('rejects an invalid request with 422', async () => {
    const res = await request(app).post('/api/requests').set(auth(student)).send({ title: 'x', category: 'nope' });
    expect(res.status).toBe(422);
  });

  test('officers cannot submit requests (RBAC → 403)', async () => {
    const res = await request(app).post('/api/requests').set(auth(officer)).send({});
    expect(res.status).toBe(403);
  });

  test('student sees only their own requests, with pagination info', async () => {
    const res = await request(app).get('/api/requests?limit=2').set(auth(student));
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBeLessThanOrEqual(2);
```

```
    expect(res.body.pagination.total).toBe(4); // 3 seeded + 1 created above
    res.body.data.forEach((r) => expect(r.requester.email).toBe('student@campusfix.edu'));
  });

  test('search and filter narrow the results', async () => {
    const res = await request(app).get('/api/requests?q=corridor&status=pending').set(auth(student));
    expect(res.body.data).toHaveLength(1);
    expect(res.body.data[0]._id).toBe(newId);
  });

  test('owner can edit while pending', async () => {
    const res = await request(app).put(`/api/requests/${newId}`).set(auth(student)).send({ priority: 'high' });
    expect(res.status).toBe(200);
    expect(res.body.data.priority).toBe('high');
  });

  test('another officer cannot read a request not assigned to them', async () => {
    const res = await request(app).get(`/api/requests/${newId}`).set(auth(plumber));
    expect(res.status).toBe(403);
  });
});

describe('Assignment and status workflow', () => {
  test('only admin can assign', async () => {
    const res = await request(app).patch(`/api/requests/${newId}/assign`).set(auth(student)).send({ officerId });
    expect(res.status).toBe(403);
  });

  test('admin assigns the request to an officer', async () => {
    const res = await request(app).patch(`/api/requests/${newId}/assign`).set(auth(admin)).send({ officerId, note: 'Urgent' });
    expect(res.status).toBe(200);
    expect(res.body.data.status).toBe('assigned');
    expect(res.body.data.assignedTo._id).toBe(officerId);
  });

  test('officer sees the job and marks it in progress then completed', async () => {
    const list = await request(app).get('/api/requests').set(auth(officer));
    expect(list.body.data.some((r) => r._id === newId)).toBe(true);

    let res = await request(app).patch(`/api/requests/${newId}/status`).set(auth(officer)).send({ status: 'in_progress', comment: 'On my way' });
    expect(res.status).toBe(200);
    res = await request(app).patch(`/api/requests/${newId}/status`).set(auth(officer)).send({ status: 'completed', comment: 'Bulb replaced' });
    expect(res.status).toBe(200);
    expect(res.body.data.completedAt).toBeDefined();
  });

  test('officer cannot cancel a job', async () => {
    const res = await request(app).patch(`/api/requests/${newId}/status`).set(auth(officer)).send({ status: 'cancelled' });
    expect([400, 403]).toContain(res.status);
  });

  test('timeline records every status change', async () => {
    const res = await request(app).get(`/api/requests/${newId}`).set(auth(student));
    expect(res.body.data.timeline.map((t) => t.toStatus)).toEqual(['pending', 'assigned', 'in_progress', 'completed']);
  });

  test('requester was notified', async () => {
    const res = await request(app).get('/api/notifications').set(auth(student));
    expect(res.body.data.some((n) => /completed/.test(n.message))).toBe(true);
  });
});

describe('Admin reports, export and audit log', () => {
  test('summary counts by status and category', async () => {
    const res = await request(app).get('/api/reports/summary').set(auth(admin));
    expect(res.status).toBe(200);
    expect(res.body.data.total).toBe(7);
    expect(res.body.data.byStatus.completed).toBe(2);
  });

  test('CSV export returns a file', async () => {
    const res = await request(app).get('/api/reports/export?status=completed').set(auth(admin));
    expect(res.status).toBe(200);
    expect(res.headers['content-type']).toMatch(/text\/csv/);
    expect(res.text.split('\n')).toHaveLength(3); // header + 2 rows
  });
```

```

  test('reports are admin-only', async () => {
    expect((await request(app).get('/api/reports/summary').set(auth(student))).status).toBe(403);
  });

  test('audit log captured the assignment', async () => {
    const res = await request(app).get('/api/reports/audit?action=REQUEST_ASSIGNED').set(auth(admin));
    expect(res.body.data.length).toBeGreaterThan(0);
  });

  test('admin deletes a request; unknown ids return 404', async () => {
    expect((await request(app).delete(`/api/requests/${newId}`).set(auth(admin))).status).toBe(200);
    expect((await request(app).get(`/api/requests/${newId}`).set(auth(admin))).status).toBe(404);
  });
});
```

server/tests/setup.js

```
// Shared helpers: connect to a throw-away test database and log in demo users
require('dotenv').config();
process.env.NODE_ENV = 'test';
process.env.JWT_SECRET = process.env.JWT_SECRET || 'test-secret';

const mongoose = require('mongoose');
const request = require('supertest');
const app = require('../src/app');
const { run } = require('../src/seed');

const TEST_URI = process.env.MONGO_URI_TEST || 'mongodb://127.0.0.1:27017/campusfix_test';

async function startDb() {
  await mongoose.connect(TEST_URI);
  await run({ withDemo: true }); // fresh roles, categories, demo users, sample requests
}

async function stopDb() {
  await mongoose.connection.dropDatabase();
  await mongoose.disconnect();
}

async function login(email, password = 'Password123') {
  const res = await request(app).post('/api/auth/login').send({ email, password });
  return res.body.token;
}

module.exports = { app, request, startDb, stopDb, login };
```

client/.gitignore

```
node_modules
dist
```

client/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" href="/favicon.svg" type="image/svg+xml" />
    <title>CampusFix — Maintenance Requests</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

client/package.json

```
{
  "name": "campusfix-client",
  "version": "1.0.0",
  "description": "CampusFix React frontend",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "test": "vitest run"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
```

```
    "@fontsource/plus-jakarta-sans": "^5.3.0",
    "axios": "^1.20.0",
    "react": "^19.3.1",
    "react-dom": "^19.3.1",
    "react-router-dom": "^7.18.4",
    "socket.io-client": "^4.8.4"
  },
  "devDependencies": {
    "@testing-library/jest-dom": "^7.0.1",
    "@testing-library/react": "^16.3.3",
    "@testing-library/user-event": "^14.6.8",
    "@vitejs/plugin-react": "^6.1.2",
    "jsdom": "^29.1.1",
    "vite": "^8.3.4",
    "vitest": "^5.0.5"
  },
  "private": true,
  "type": "module"
}
```

client/vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    // In development, forward API, uploads and websocket traffic to Express
    proxy: {
      '/api': 'http://localhost:5000',
      '/uploads': 'http://localhost:5000',
      '/socket.io': { target: 'http://localhost:5000', ws: true },
    },
  },
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: './src/__tests__/setupTests.js',
  },
});
```

client/public/favicon.svg

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 32 32"><rect width="32" height="32" rx="8" fill="#14365c"
↪ /><path d="M20.5 8.5a5 5 0 0 0-6.3 6.3l-6 6a1.6 1.6 0 0 0 2.3 2.3l6-6a5 5 0 0 0 6.3-6.3l-2.9 2.9-2.3-.6-.6-2.3
↪ z" fill="#f2b84b"/></svg>
```

client/src/App.jsx

```
import { Navigate, Route, Routes } from 'react-router-dom';
import { useAuth } from './context/AuthContext';
import ProtectedRoute from './components/ProtectedRoute';
import Layout from './components/Layout';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';
import NewRequest from './pages/NewRequest';
import Requests from './pages/Requests';
import RequestDetail from './pages/RequestDetail';
import Users from './pages/admin/Users';
import Categories from './pages/admin/Categories';
import Reports from './pages/admin/Reports';

export default function App() {
  const { user } = useAuth();
  return (
    <Routes>
      <Route path="/login" element={user ? <Navigate to="/dashboard" /> : <Login />} />
      <Route path="/register" element={user ? <Navigate to="/dashboard" /> : <Register />} />

      <Route element={<ProtectedRoute><Layout /></ProtectedRoute>}>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/requests" element={<Requests />} />
        <Route path="/requests/new" element={<ProtectedRoute roles={['user', 'admin']}><NewRequest /></Protect
↪ edRoute>} />
        <Route path="/requests/:id" element={<RequestDetail />} />
        <Route path="/admin/users" element={<ProtectedRoute roles={['admin']}><Users /></ProtectedRoute>} />
        <Route path="/admin/categories" element={<ProtectedRoute roles={['admin']}><Categories /></ProtectedRo
↪ ute>} />
        <Route path="/admin/reports" element={<ProtectedRoute roles={['admin']}><Reports /></ProtectedRoute>}
↪ />
```

```
      </Route>

      <Route path="*" element={<Navigate to={user ? '/dashboard' : '/login'} replace />} />
    </Routes>
  );
}
```

client/src/__tests__/Auth.test.jsx

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { MemoryRouter, Route, Routes } from 'react-router-dom';
import { describe, expect, test, vi } from 'vitest';
import Login, { validateLogin } from '../pages/Login';
import { validateRegister } from '../pages/Register';
import ProtectedRoute from '../components/ProtectedRoute';

// Replace the real auth hook with a controllable fake
const authState = { user: null, loading: false, login: vi.fn() };
vi.mock('../context/AuthContext', () => ({ useAuth: () => authState }));
vi.mock('../context/ToastContext', () => ({ useToast: () => vi.fn() }));

describe('Login page', () => {
  test('validates email and password before calling the API', async () => {
    render(<MemoryRouter><Login /></MemoryRouter>);
    await userEvent.click(screen.getByRole('button', { name: /sign in/i }));
    expect(screen.getByText('Email is required')).toBeInTheDocument();
    expect(screen.getByText('Password is required')).toBeInTheDocument();
    expect(authState.login).not.toHaveBeenCalled();
  });

  test('shows the server error message on failed login', async () => {
    authState.login = vi.fn().mockRejectedValue({ response: { data: { message: 'Invalid email or password' } } });
    render(<MemoryRouter><Login /></MemoryRouter>);
    await userEvent.type(screen.getByLabelText(/email/i), 'student@campusfix.edu');
    await userEvent.type(screen.getByLabelText(/password/i), 'wrong-pass1');
    await userEvent.click(screen.getByRole('button', { name: /sign in/i }));
    expect(await screen.findByRole('alert')).toHaveTextContent('Invalid email or password');
  });

  test('validateLogin and validateRegister rules', () => {
    expect(validateLogin({ email: 'bad', password: 'x' }).email).toMatch(/valid email/);
    const errs = validateRegister({ name: '', email: 'a@b.co', password: 'abcdefgh', confirm: 'zzz' });
    expect(errs.name).toBeDefined();
    expect(errs.password).toMatch(/number/);
    expect(errs.confirm).toMatch(/do not match/);
  });
});

describe('ProtectedRoute (role-based access)', () => {
  const renderAt = (roles) =>
    render(
      <MemoryRouter initialEntries={['/admin']}>
        <Routes>
          <Route path="/admin" element={<ProtectedRoute roles={roles}><p>secret admin page</p></ProtectedRoute>} />
          <Route path="/login" element={<p>login page</p>} />
          <Route path="/dashboard" element={<p>dashboard page</p>} />
        </Routes>
      </MemoryRouter>
    );

  test('hides the page from a visitor who is not logged in', () => {
    authState.user = null;
    renderAt(['admin']);
    expect(screen.queryByText('secret admin page')).not.toBeInTheDocument();
    expect(screen.getByText('login page')).toBeInTheDocument();
  });

  test('hides an admin page from a student', () => {
    authState.user = { name: 'Ada', role: { name: 'user' } };
    renderAt(['admin']);
    expect(screen.queryByText('secret admin page')).not.toBeInTheDocument();
    expect(screen.getByText('dashboard page')).toBeInTheDocument();
  });

  test('shows the page to an admin', () => {
    authState.user = { name: 'Grace', role: { name: 'admin' } };
    renderAt(['admin']);
    expect(screen.getByText('secret admin page')).toBeInTheDocument();
  });
});
```

client/src/__tests__/Components.test.jsx

```jsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { MemoryRouter } from 'react-router-dom';
import { describe, expect, test, vi } from 'vitest';
import StatusBadge from '../components/StatusBadge';
import Pagination from '../components/Pagination';
import RequestTable from '../components/RequestTable';

describe('StatusBadge', () => {
  test.each([
    ['pending', 'Pending'],
    ['in_progress', 'In progress'],
    ['completed', 'Completed'],
  ])('renders %s as "%s" with matching style', (status, label) => {
    render(<StatusBadge status={status} />);
    const el = screen.getByText(label);
    expect(el).toHaveClass(`badge-${status}`);
  });
});

describe('Pagination', () => {
  test('shows the range and disables Prev on the first page', async () => {
    const onChange = vi.fn();
    render(<Pagination pagination={{ page: 1, pages: 3, total: 25, limit: 10 }} onChange={onChange} />);
    expect(screen.getByText('Showing 1–10 of 25')).toBeInTheDocument();
    expect(screen.getByRole('button', { name: /prev/i })).toBeDisabled();
    await userEvent.click(screen.getByRole('button', { name: /next/i }));
    expect(onChange).toHaveBeenCalledWith(2);
  });
});

describe('RequestTable', () => {
  const rows = [{
    _id: 'r1', ticketNo: 'REQ-2026-00001', title: 'Fan not working', location: 'LT 2',
    category: { name: 'Electrical' }, priority: 'high', status: 'assigned',
    requester: { name: 'Ada Obi' }, assignedTo: { name: 'Musa Ibrahim' }, createdAt: '2026-10-01T10:00:00Z',
  }];

  test('renders rows with a link to the tracking page', () => {
    render(<MemoryRouter><RequestTable items={rows} showRequester showOfficer /></MemoryRouter>);
    expect(screen.getByRole('link', { name: 'REQ-2026-00001' })).toHaveAttribute('href', '/requests/r1');
    expect(screen.getByText('Musa Ibrahim')).toBeInTheDocument();
    expect(screen.getByText('Assigned')).toBeInTheDocument();
  });

  test('shows the empty message when there are no rows', () => {
    render(<MemoryRouter><RequestTable items={[]} empty="Nothing here" /></MemoryRouter>);
    expect(screen.getByText('Nothing here')).toBeInTheDocument();
  });
});
```

client/src/__tests__/RequestForm.test.jsx

```jsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { describe, expect, test, vi } from 'vitest';
import RequestForm, { validateRequest } from '../components/RequestForm';

const categories = [
  { _id: 'c1', name: 'Electrical' },
  { _id: 'c2', name: 'Plumbing' },
];

describe('RequestForm', () => {
  test('shows validation messages and does not submit an empty form', async () => {
    const onSubmit = vi.fn();
    render(<RequestForm categories={categories} onSubmit={onSubmit} />);
    await userEvent.click(screen.getByRole('button', { name: /submit request/i }));

    expect(screen.getByText(/at least 5 characters/i)).toBeInTheDocument();
    expect(screen.getByText(/choose a category/i)).toBeInTheDocument();
    expect(screen.getByText(/tell us where/i)).toBeInTheDocument();
    expect(onSubmit).not.toHaveBeenCalled();
  });

  test('lists categories from the API in the dropdown', () => {
    render(<RequestForm categories={categories} onSubmit={vi.fn()} />);
    expect(screen.getByRole('option', { name: 'Electrical' })).toBeInTheDocument();
    expect(screen.getByRole('option', { name: 'Plumbing' })).toBeInTheDocument();
  });
```

```
29    test('submits the form data when valid', async () => {
30      const onSubmit = vi.fn();
31      render(<RequestForm categories={categories} onSubmit={onSubmit} />);
32      await userEvent.type(screen.getByLabelText(/title of the issue/i), 'Leaking pipe in toilet');
33      await userEvent.selectOptions(screen.getByLabelText(/category/i), 'c2');
34      await userEvent.type(screen.getByLabelText(/location/i), 'Hostel A, Block 3');
35      await userEvent.type(screen.getByLabelText(/description/i), 'Water is leaking from the pipe behind the toi
  ↪ let.');
36      await userEvent.click(screen.getByRole('button', { name: /submit request/i }));
37
38      expect(onSubmit).toHaveBeenCalledTimes(1);
39      const [form, files] = onSubmit.mock.calls[0];
40      expect(form).toMatchObject({ title: 'Leaking pipe in toilet', category: 'c2', location: 'Hostel A, Block 3
  ↪ ', priority: 'medium' });
41      expect(files).toEqual([]);
42    });
43
44    test('validateRequest rejects more than 3 files and oversized files', () => {
45      const ok = { title: 'Broken window', category: 'c1', location: 'Room 4', description: 'The window glass is
  ↪  cracked.' };
46      expect(validateRequest(ok, [{}, {}, {}, {}]).files).toMatch(/at most 3/);
47      expect(validateRequest(ok, [{ size: 6 * 1024 * 1024 }]).files).toMatch(/5 MB/);
48      expect(validateRequest(ok, [])).toEqual({});
49    });
50  });
```

client/src/__tests__/setupTests.js

```
1  import '@testing-library/jest-dom/vitest';
2  import { cleanup } from '@testing-library/react';
3  import { afterEach } from 'vitest';
4
5  afterEach(cleanup);
```

client/src/api.js

```
1  import axios from 'axios';
2
3  // All API calls go through this instance. The JWT is attached automatically.
4  const api = axios.create({ baseURL: import.meta.env.VITE_API_URL || '/api' });
5
6  api.interceptors.request.use((config) => {
7    const token = localStorage.getItem('token');
8    if (token) config.headers.Authorization = `Bearer ${token}`;
9    return config;
10 });
11
12 // If the session has expired, clear it and send the user back to login
13 api.interceptors.response.use(
14   (res) => res,
15   (err) => {
16     if (err.response?.status === 401 && localStorage.getItem('token')) {
17       localStorage.removeItem('token');
18       window.location.href = '/login?expired=1';
19     }
20     return Promise.reject(err);
21   }
22 );
23
24 // Turn any API error into a readable message for the UI
25 export function errorMessage(err) {
26   const data = err?.response?.data;
27   if (data?.errors?.length) return data.errors.map((e) => e.message).join('. ');
28   return data?.message || err?.message || 'Something went wrong';
29 }
30
31 export default api;
```

client/src/components/AuthShell.jsx

```
1  import Logo from './Logo';
2
3  // Two-column frame shared by the login and registration pages
4  export default function AuthShell({ title, subtitle, children }) {
5    return (
6      <div className="auth">
7        <section className="auth-panel">
8          <div className="brand brand-light"><Logo size={40} /><span>CampusFix</span></div>
9          <div className="auth-pitch">
10           <h1>One place for every campus maintenance request.</h1>
11           <p>Report a fault in under a minute, attach a photo, and follow it from submission to completion — n
  ↪ o more phone calls, paper forms or lost WhatsApp messages.</p>
12           <ul>
```

```
            <li>Electricity, plumbing, furniture &amp;amp; hostel issues</li>
            <li>Live status updates and notifications</li>
            <li>Every job tracked and accountable</li>
          </ul>
        </div>
        <small>University Works &amp;amp; Services Department</small>
      </section>
      <section className="auth-form">
        <div className="auth-card">
          <h2>{title}</h2>
          <p className="muted">{subtitle}</p>
          {children}
        </div>
      </section>
    </div>
  );
}
```

client/src/components/Layout.jsx

```
import { useCallback, useEffect, useRef, useState } from 'react';
import { Link, NavLink, Outlet, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import api from '../api';
import Logo from './Logo';

const NAV = {
  user: [
    ['/dashboard', 'Dashboard'],
    ['/requests/new', 'New request'],
    ['/requests', 'My requests'],
  ],
  officer: [
    ['/dashboard', 'Dashboard'],
    ['/requests', 'Assigned jobs'],
  ],
  admin: [
    ['/dashboard', 'Dashboard'],
    ['/requests', 'All requests'],
    ['/requests/new', 'Log a request'],
    ['/admin/users', 'Users'],
    ['/admin/categories', 'Categories'],
    ['/admin/reports', 'Reports & audit'],
  ],
};

function timeAgo(date) {
  const s = Math.floor((Date.now() - new Date(date)) / 1000);
  if (s < 60) return 'just now';
  if (s < 3600) return `${Math.floor(s / 60)}m ago`;
  if (s < 86400) return `${Math.floor(s / 3600)}h ago`;
  return `${Math.floor(s / 86400)}d ago`;
}

function NotificationBell() {
  const [open, setOpen] = useState(false);
  const [data, setData] = useState({ items: [], unread: 0 });
  const ref = useRef(null);
  const navigate = useNavigate();

  const load = useCallback(() => {
    api.get('/notifications').then((r) => setData({ items: r.data.data, unread: r.data.unread })).catch(() =>
↪ {});
  }, []);

  useEffect(() => {
    load();
    window.addEventListener('notifications:changed', load);
    const close = (e) => ref.current && !ref.current.contains(e.target) && setOpen(false);
    document.addEventListener('mousedown', close);
    return () => {
      window.removeEventListener('notifications:changed', load);
      document.removeEventListener('mousedown', close);
    };
  }, [load]);

  const openItem = async (n) => {
    if (!n.read) await api.patch(`/notifications/${n._id}/read`);
    setOpen(false);
    load();
    if (n.request) navigate(`/requests/${n.request}`);
  };

```

```
  63    const readAll = async () => {
  64      await api.patch('/notifications/read-all');
  65      load();
  66    };
  67
  68    return (
  69      <div className="bell" ref={ref}>
  70        <button type="button" className="icon-btn" aria-label="Notifications" onClick={() => setOpen(!open)}>
  71          <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" stro
  ↪ keLinecap="round" strokeLinejoin="round"><path d="M6 8a6 6 0 0 1 12 0c0 7 3 9 3 9H3s3-2 3-9" /><path d="M10.3
  ↪ 21a1.94 1.94 0 0 0 3.4 0" /></svg>
  72          {data.unread > 0 && <span className="bell-count">{data.unread}</span>}
  73        </button>
  74        {open && (
  75          <div className="bell-menu">
  76            <div className="bell-head">
  77              <strong>Notifications</strong>
  78              {data.unread > 0 && <button type="button" className="link-btn" onClick={readAll}>Mark all read</bu
  ↪ tton>}
  79            </div>
  80            {data.items.length === 0 && <p className="muted pad">You are all caught up.</p>}
  81            {data.items.map((n) => (
  82              <button type="button" key={n._id} className={`bell-item ${n.read ? '' : 'unread'}`} onClick={() =>
  ↪  openItem(n)}>
  83                <span>{n.message}</span>
  84                <small>{timeAgo(n.createdAt)}</small>
  85              </button>
  86            ))}
  87          </div>
  88        )}
  89      </div>
  90    );
  91  }
  92
  93  export default function Layout() {
  94    const { user, logout } = useAuth();
  95    const navigate = useNavigate();
  96    const [menuOpen, setMenuOpen] = useState(false);
  97
  98    const doLogout = async () => {
  99      await logout();
 100      navigate('/login');
 101    };
 102
 103    return (
 104      <div className="shell">
 105        <aside className={`sidebar ${menuOpen ? 'open' : ''}`}>
 106          <Link to="/dashboard" className="brand"><Logo /><span>CampusFix</span></Link>
 107          <nav onClick={() => setMenuOpen(false)}>
 108            {NAV[user.role.name].map(([to, label]) => (
 109              <NavLink key={to} to={to} end className={({ isActive }) => (isActive ? 'active' : '')}>{label}</Na
   ↪ vLink>
 110            ))}
 111          </nav>
 112          <div className="sidebar-foot">
 113            <span className="role-pill">{user.role.label}</span>
 114          </div>
 115        </aside>
 116
 117        <div className="main">
 118          <header className="topbar">
 119            <button type="button" className="icon-btn menu-toggle" aria-label="Menu" onClick={() => setMenuOpen(
   ↪ !menuOpen)}>≡</button>
 120            <div className="topbar-right">
 121              <NotificationBell />
 122              <div className="who">
 123                <span className="avatar">{user.name.split(' ').map((p) => p[0]).slice(0, 2).join('')}</span>
 124                <div>
 125                  <strong>{user.name}</strong>
 126                  <small>{user.email}</small>
 127                </div>
 128              </div>
 129              <button type="button" className="btn btn-ghost btn-sm" onClick={doLogout}>Log out</button>
 130            </div>
 131          </header>
 132          <main className="content">
 133            <Outlet />
 134          </main>
 135        </div>
 136      </div>
 137    );
 138  }
```

client/src/components/Logo.jsx

```
export default function Logo({ size = 32 }) {
  return (
    <svg width={size} height={size} viewBox="0 0 32 32" aria-hidden="true">
      <rect width="32" height="32" rx="8" fill="#14365c" />
      <path d="M20.5 8.5a5 5 0 0 0-6.3 6.3l-6 6a1.6 1.6 0 0 0 2.3 2.3l6-6a5 5 0 0 0 6.3-6.3l-2.9 2.9-2.3-.6-.6-2.3z" fill="#f2b84b" />
    </svg>
  );
}
```

client/src/components/Pagination.jsx

```
export default function Pagination({ pagination, onChange }) {
  if (!pagination || pagination.total === 0) return null;
  const { page, pages, total, limit } = pagination;
  const from = (page - 1) * limit + 1;
  const to = Math.min(page * limit, total);
  return (
    <div className="pagination">
      <span className="muted">Showing {from}–{to} of {total}</span>
      <div className="pager">
        <button type="button" className="btn btn-ghost btn-sm" disabled={page <= 1} onClick={() => onChange(page - 1)}>‹ Prev</button>
        <span>Page {page} of {pages}</span>
        <button type="button" className="btn btn-ghost btn-sm" disabled={page >= pages} onClick={() => onChange(page + 1)}>Next ›</button>
      </div>
    </div>
  );
}
```

client/src/components/ProtectedRoute.jsx

```
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

// Blocks pages for visitors who are not logged in, or whose role is not allowed
export default function ProtectedRoute({ roles, children }) {
  const { user, loading } = useAuth();
  const location = useLocation();

  if (loading) return <div className="page-loading">Loading…</div>;
  if (!user) return <Navigate to="/login" replace state={{ from: location }} />;
  if (roles && !roles.includes(user.role.name)) return <Navigate to="/dashboard" replace />;
  return children;
}
```

client/src/components/RequestForm.jsx

```
import { useState } from 'react';

const MAX_FILES = 3;
const MAX_SIZE = 5 * 1024 * 1024;

export function validateRequest(f, files = []) {
  const errors = {};
  if (f.title.trim().length < 5) errors.title = 'Give the issue a short title (at least 5 characters)';
  if (f.title.length > 120) errors.title = 'Title must be 120 characters or fewer';
  if (!f.category) errors.category = 'Choose a category';
  if (!f.location.trim()) errors.location = 'Tell us where the problem is';
  if (f.description.trim().length < 10) errors.description = 'Describe the problem (at least 10 characters)';
  if (files.length > MAX_FILES) errors.files = `Attach at most ${MAX_FILES} files`;
  else if (files.some((x) => x.size > MAX_SIZE)) errors.files = 'Each file must be 5 MB or smaller';
  return errors;
}

// Presentational form — the page decides what to do on submit
export default function RequestForm({ categories, onSubmit, busy }) {
  const [form, setForm] = useState({ title: '', category: '', location: '', priority: 'medium', description: '' });
  const [files, setFiles] = useState([]);
  const [errors, setErrors] = useState({});

  const change = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
    if (errors[e.target.name]) setErrors({ ...errors, [e.target.name]: undefined });
  };

  const submit = (e) => {
    e.preventDefault();
    const v = validateRequest(form, files);
```

```
    setErrors(v);
    if (Object.keys(v).length === 0) onSubmit(form, files);
  };

  return (
    <form onSubmit={submit} noValidate className="form-grid">
      <label className="field span-2">
        <span>Title of the issue *</span>
        <input name="title" value={form.title} onChange={change} placeholder="e.g. Ceiling fan not working" ar
↪ ia-invalid={!!errors.title} />
        {errors.title && <em className="field-error">{errors.title}</em>}
      </label>

      <label className="field">
        <span>Category *</span>
        <select name="category" value={form.category} onChange={change} aria-invalid={!!errors.category}>
          <option value="">Select a category…</option>
          {categories.map((c) => <option key={c._id} value={c._id}>{c.name}</option>)}
        </select>
        {errors.category && <em className="field-error">{errors.category}</em>}
      </label>

      <label className="field">
        <span>Priority</span>
        <select name="priority" value={form.priority} onChange={change}>
          <option value="low">Low — can wait</option>
          <option value="medium">Medium</option>
          <option value="high">High — affecting work/study</option>
          <option value="urgent">Urgent — safety risk</option>
        </select>
      </label>

      <label className="field span-2">
        <span>Location *</span>
        <input name="location" value={form.location} onChange={change} placeholder="Building, floor and room —
↪  e.g. Hostel B, Room 214" aria-invalid={!!errors.location} />
        {errors.location && <em className="field-error">{errors.location}</em>}
      </label>

      <label className="field span-2">
        <span>Description *</span>
        <textarea name="description" rows={5} value={form.description} onChange={change} placeholder="What is
↪ wrong, since when, and anything the officer should know." aria-invalid={!!errors.description} />
        {errors.description && <em className="field-error">{errors.description}</em>}
      </label>

      <label className="field span-2">
        <span>Evidence photos (optional, up to 3 — JPG, PNG or PDF, 5 MB each)</span>
        <input type="file" accept="image/*,application/pdf" multiple onChange={(e) => setFiles(Array.from(e.ta
↪ rget.files))} data-testid="file-input" />
        {errors.files && <em className="field-error">{errors.files}</em>}
        {files.length > 0 && (
          <div className="thumbs">
            {files.map((f) => (
              <div key={f.name} className="thumb">
                {f.type.startsWith('image/') && typeof URL.createObjectURL === 'function'
                  ? <img src={URL.createObjectURL(f)} alt={f.name} />
                  : <span>PDF</span>}
                <small>{f.name}</small>
              </div>
            ))}
          </div>
        )}
      </label>

      <div className="span-2 form-actions">
        <button type="submit" className="btn btn-primary" disabled={busy}>{busy ? 'Submitting…' : 'Submit requ
↪ est'}</button>
      </div>
    </form>
  );
}
```

client/src/components/RequestTable.jsx

```jsx
import { Link } from 'react-router-dom';
import StatusBadge, { PriorityTag } from './StatusBadge';

const fmt = (d) => new Date(d).toLocaleDateString('en-GB', { day: '2-digit', month: 'short', year: 'numeric' });

export default function RequestTable({ items, showRequester, showOfficer, empty = 'No requests found.' }) {
  if (!items.length) return <div className="empty">{empty}</div>;
  return (
    <div className="table-wrap">
      <table className="table">
        <thead>
          <tr>
            <th>Ticket</th>
            <th>Issue</th>
            <th>Category</th>
            {showRequester && <th>Requested by</th>}
            {showOfficer && <th>Officer</th>}
            <th>Priority</th>
            <th>Status</th>
            <th>Date</th>
          </tr>
        </thead>
        <tbody>
          {items.map((r) => (
            <tr key={r._id}>
              <td><Link to={`/requests/${r._id}`} className="ticket">{r.ticketNo}</Link></td>
              <td>
                <Link to={`/requests/${r._id}`} className="cell-title">{r.title}</Link>
                <small className="muted block">{r.location}</small>
              </td>
              <td>{r.category?.name}</td>
              {showRequester && <td>{r.requester?.name}</td>}
              {showOfficer && <td>{r.assignedTo?.name || <span className="muted">Unassigned</span>}</td>}
              <td><PriorityTag priority={r.priority} /></td>
              <td><StatusBadge status={r.status} /></td>
              <td className="nowrap">{fmt(r.createdAt)}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

client/src/components/StatusBadge.jsx

```jsx
export const STATUS_LABELS = {
  pending: 'Pending',
  assigned: 'Assigned',
  in_progress: 'In progress',
  completed: 'Completed',
  cancelled: 'Cancelled',
};

export default function StatusBadge({ status }) {
  return <span className={`badge badge-${status}`}>{STATUS_LABELS[status] || status}</span>;
}

export function PriorityTag({ priority }) {
  return <span className={`priority priority-${priority}`}>{priority}</span>;
}
```

client/src/context/AuthContext.jsx

```jsx
import { createContext, useCallback, useContext, useEffect, useState } from 'react';
import api from '../api';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // On first load, restore the session from the saved token
  useEffect(() => {
    if (!localStorage.getItem('token')) {
      setLoading(false);
      return;
    }
    api.get('/auth/me')
```

```
      .then((res) => setUser(res.data.user))
      .catch(() => localStorage.removeItem('token'))
      .finally(() => setLoading(false));
  }, []);

  const saveSession = (data) => {
    localStorage.setItem('token', data.token);
    setUser(data.user);
    return data.user;
  };

  const login = async (email, password) => saveSession((await api.post('/auth/login', { email, password })).da
↪ ta);
  const register = async (form) => saveSession((await api.post('/auth/register', form)).data);

  const logout = useCallback(async () => {
    try { await api.post('/auth/logout'); } catch { /* token may already be invalid */ }
    localStorage.removeItem('token');
    setUser(null);
  }, []);

  return (
    <AuthContext.Provider value={{ user, loading, login, register, logout, role: user?.role?.name }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

client/src/context/SocketContext.jsx

```
import { createContext, useContext, useEffect, useState } from 'react';
import { io } from 'socket.io-client';
import { useAuth } from './AuthContext';
import { useToast } from './ToastContext';

const SocketContext = createContext(null);

// Opens a real-time connection once the user is logged in.
// Pages listen for "request:updated" to refresh; new notifications pop up as toasts.
export function SocketProvider({ children }) {
  const { user } = useAuth();
  const toast = useToast();
  const [socket, setSocket] = useState(null);

  useEffect(() => {
    if (!user) return undefined;
    const s = io(import.meta.env.VITE_SOCKET_URL || '/', { auth: { token: localStorage.getItem('token') } });
    s.on('notification', (n) => {
      toast(n.message, 'info');
      window.dispatchEvent(new CustomEvent('notifications:changed'));
    });
    setSocket(s);
    return () => s.disconnect();
  }, [user, toast]);

  return <SocketContext.Provider value={socket}>{children}</SocketContext.Provider>;
}

// Re-run `callback` whenever any request changes status (live updates)
export function useRequestUpdates(callback) {
  const socket = useContext(SocketContext);
  useEffect(() => {
    if (!socket) return undefined;
    socket.on('request:updated', callback);
    return () => socket.off('request:updated', callback);
  }, [socket, callback]);
}
```

client/src/context/ToastContext.jsx

```
import { createContext, useCallback, useContext, useState } from 'react';

const ToastContext = createContext(() => {});

// Small success / error messages in the corner of the screen
export function ToastProvider({ children }) {
  const [toasts, setToasts] = useState([]);

  const toast = useCallback((message, type = 'success') => {
    const id = Math.random().toString(36).slice(2);
    setToasts((t) => [...t, { id, message, type }]);
```

```
    setTimeout(() => setToasts((t) => t.filter((x) => x.id !== id)), 4000);
  }, []);

  return (
    <ToastContext.Provider value={toast}>
      {children}
      <div className="toasts" aria-live="polite">
        {toasts.map((t) => (
          <div key={t.id} className={`toast toast-${t.type}`} role="status">{t.message}</div>
        ))}
      </div>
    </ToastContext.Provider>
  );
}

export const useToast = () => useContext(ToastContext);
```

client/src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import '@fontsource/plus-jakarta-sans/400.css';
import '@fontsource/plus-jakarta-sans/600.css';
import '@fontsource/plus-jakarta-sans/700.css';
import './styles.css';
import App from './App';
import { AuthProvider } from './context/AuthContext';
import { ToastProvider } from './context/ToastContext';
import { SocketProvider } from './context/SocketContext';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <ToastProvider>
        <AuthProvider>
          <SocketProvider>
            <App />
          </SocketProvider>
        </AuthProvider>
      </ToastProvider>
    </BrowserRouter>
  </StrictMode>
);
```

client/src/pages/Dashboard.jsx

```
import { useCallback, useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import api from '../api';
import { useAuth } from '../context/AuthContext';
import { useRequestUpdates } from '../context/SocketContext';
import RequestTable from '../components/RequestTable';

const CARDS = {
  user: [['total', 'All my requests'], ['pending', 'Awaiting action'], ['in_progress', 'Being fixed'], ['compl
↪ eted', 'Resolved']],
  officer: [['assigned', 'New jobs'], ['in_progress', 'In progress'], ['completed', 'Completed'], ['total', 'T
↪ otal assigned']],
  admin: [['total', 'Total requests'], ['pending', 'Unassigned'], ['in_progress', 'In progress'], ['completed'
↪ , 'Completed']],
};

const COPY = {
  user: ['Report a problem and follow it until it is fixed.', 'Recent requests'],
  officer: ['Jobs assigned to you by the Works department.', 'Jobs that need your attention'],
  admin: ['Overview of maintenance activity across campus.', 'New requests waiting to be assigned'],
};

export default function Dashboard() {
  const { user, role } = useAuth();
  const [stats, setStats] = useState(null);
  const [recent, setRecent] = useState([]);

  const load = useCallback(() => {
    api.get('/requests/stats').then((r) => setStats(r.data.data));
    const params = { limit: 5 };
    if (role === 'admin') params.status = 'pending';
    if (role === 'officer') params.status = 'assigned';
    api.get('/requests', { params }).then((r) => setRecent(r.data.data));
  }, [role]);

  useEffect(load, [load]);
```

```
  useRequestUpdates(load); // live refresh when any request changes

  const value = (key) => (key === 'total' ? stats?.total : stats?.byStatus?.[key]) ?? '—';

  return (
    <>
      <div className="page-head">
        <div>
          <h1>Good day, {user.name.split(' ')[0]}</h1>
          <p className="muted">{COPY[role][0]}</p>
        </div>
        {role !== 'officer' && <Link to="/requests/new" className="btn btn-primary">+ New request</Link>}
      </div>

      <div className="stats">
        {CARDS[role].map(([key, label]) => (
          <div key={key} className={`stat stat-${key}`}>
            <span className="stat-value">{value(key)}</span>
            <span className="stat-label">{label}</span>
          </div>
        ))}
      </div>

      <section className="card">
        <div className="card-head">
          <h2>{COPY[role][1]}</h2>
          <Link to="/requests" className="link">View all →</Link>
        </div>
        <RequestTable
          items={recent}
          showRequester={role !== 'user'}
          showOfficer={role === 'admin'}
          empty={role === 'user' ? 'You have not submitted any requests yet.' : 'Nothing waiting here — nice w
↳ ork.'}
        />
      </section>
    </>
  );
}
```

client/src/pages/Login.jsx

```
import { useState } from 'react';
import { Link, useLocation, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { useToast } from '../context/ToastContext';
import { errorMessage } from '../api';
import AuthShell from '../components/AuthShell';

export function validateLogin({ email, password }) {
  const errors = {};
  if (!email.trim()) errors.email = 'Email is required';
  else if (!/^\S+@\S+\.\S+$/.test(email)) errors.email = 'Enter a valid email address';
  if (!password) errors.password = 'Password is required';
  return errors;
}

export default function Login() {
  const { login } = useAuth();
  const toast = useToast();
  const navigate = useNavigate();
  const location = useLocation();
  const [form, setForm] = useState({ email: '', password: '' });
  const [errors, setErrors] = useState({});
  const [serverError, setServerError] = useState('');
  const [busy, setBusy] = useState(false);
  const expired = new URLSearchParams(location.search).get('expired');

  const change = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
    if (errors[e.target.name]) setErrors({ ...errors, [e.target.name]: undefined });
  };

  const submit = async (e) => {
    e.preventDefault();
    const v = validateLogin(form);
    setErrors(v);
    setServerError('');
    if (Object.keys(v).length) return;
    setBusy(true);
    try {
      const user = await login(form.email, form.password);
      toast(`Welcome back, ${user.name.split(' ')[0]}`);
```

```
      navigate(location.state?.from?.pathname || '/dashboard', { replace: true });
    } catch (err) {
      setServerError(errorMessage(err));
    } finally {
      setBusy(false);
    }
  };

  return (
    <AuthShell title="Sign in" subtitle="Report and track maintenance issues across campus.">
      {expired && <div className="alert alert-info">Your session expired. Please sign in again.</div>}
      {serverError && <div className="alert alert-error" role="alert">{serverError}</div>}
      <form onSubmit={submit} noValidate>
        <label className="field">
          <span>Email address</span>
          <input name="email" type="email" value={form.email} onChange={change} autoComplete="email" placehold
↪ er="you@campusfix.edu" aria-invalid={!!errors.email} />
          {errors.email && <em className="field-error">{errors.email}</em>}
        </label>
        <label className="field">
          <span>Password</span>
          <input name="password" type="password" value={form.password} onChange={change} autoComplete="current
↪ -password" aria-invalid={!!errors.password} />
          {errors.password && <em className="field-error">{errors.password}</em>}
        </label>
        <button type="submit" className="btn btn-primary btn-block" disabled={busy}>{busy ? 'Signing in…' : 'S
↪ ign in'}</button>
      </form>
      <p className="auth-switch">New student or staff member? <Link to="/register">Create an account</Link></p
↪ >
    </AuthShell>
  );
}
```

client/src/pages/NewRequest.jsx

```
import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import api, { errorMessage } from '../api';
import { useToast } from '../context/ToastContext';
import RequestForm from '../components/RequestForm';

export default function NewRequest() {
  const [categories, setCategories] = useState([]);
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState('');
  const toast = useToast();
  const navigate = useNavigate();

  useEffect(() => {
    api.get('/categories').then((r) => setCategories(r.data.data));
  }, []);

  // multipart/form-data so the photos travel with the form fields
  const submit = async (form, files) => {
    setBusy(true);
    setError('');
    try {
      const data = new FormData();
      Object.entries(form).forEach(([k, v]) => data.append(k, v));
      files.forEach((f) => data.append('attachments', f));
      const res = await api.post('/requests', data);
      toast(`Request ${res.data.data.ticketNo} submitted`);
      navigate(`/requests/${res.data.data._id}`);
    } catch (err) {
      setError(errorMessage(err));
    } finally {
      setBusy(false);
    }
  };

  return (
    <>
      <div className="page-head">
        <div>
          <h1>New service request</h1>
          <p className="muted">Tell us what needs fixing. You'll be notified as it progresses.</p>
        </div>
      </div>
      <section className="card card-pad narrow">
        {error && <div className="alert alert-error" role="alert">{error}</div>}
        <RequestForm categories={categories} onSubmit={submit} busy={busy} />
      </section>
```

```
    </>
  );
}
```

client/src/pages/Register.jsx

```
import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { useToast } from '../context/ToastContext';
import { errorMessage } from '../api';
import AuthShell from '../components/AuthShell';

export function validateRegister(f) {
  const errors = {};
  if (!f.name.trim()) errors.name = 'Full name is required';
  if (!/^\S+@\S+\.\S+$/.test(f.email)) errors.email = 'Enter a valid email address';
  if (f.password.length < 8) errors.password = 'Use at least 8 characters';
  else if (!/\d/.test(f.password)) errors.password = 'Include at least one number';
  if (f.confirm !== f.password) errors.confirm = 'Passwords do not match';
  return errors;
}

export default function Register() {
  const { register } = useAuth();
  const toast = useToast();
  const navigate = useNavigate();
  const [form, setForm] = useState({ name: '', email: '', department: '', phone: '', password: '', confirm: '' });
  const [errors, setErrors] = useState({});
  const [serverError, setServerError] = useState('');
  const [busy, setBusy] = useState(false);

  const change = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
    if (errors[e.target.name]) setErrors({ ...errors, [e.target.name]: undefined });
  };

  const submit = async (e) => {
    e.preventDefault();
    const v = validateRegister(form);
    setErrors(v);
    setServerError('');
    if (Object.keys(v).length) return;
    setBusy(true);
    try {
      const { confirm, ...payload } = form;
      await register(payload);
      toast('Account created — welcome to CampusFix!');
      navigate('/dashboard');
    } catch (err) {
      setServerError(errorMessage(err));
    } finally {
      setBusy(false);
    }
  };

  const field = (name, label, type = 'text', extra = {}) => (
    <label className="field">
      <span>{label}</span>
      <input name={name} type={type} value={form[name]} onChange={change} aria-invalid={!!errors[name]} {...extra} />
      {errors[name] && <em className="field-error">{errors[name]}</em>}
    </label>
  );

  return (
    <AuthShell title="Create your account" subtitle="For students and staff. Officers are added by the administrator.">
      {serverError && <div className="alert alert-error" role="alert">{serverError}</div>}
      <form onSubmit={submit} noValidate>
        {field('name', 'Full name', 'text', { autoComplete: 'name' })}
        {field('email', 'Email address', 'email', { autoComplete: 'email' })}
        <div className="row-2">
          {field('department', 'Department / Faculty')}
          {field('phone', 'Phone number', 'tel')}
        </div>
        <div className="row-2">
          {field('password', 'Password', 'password', { autoComplete: 'new-password' })}
          {field('confirm', 'Confirm password', 'password', { autoComplete: 'new-password' })}
        </div>
        <button type="submit" className="btn btn-primary btn-block" disabled={busy}>{busy ? 'Creating account…' : 'Create account'}</button>
```

```
      </form>
      <p className="auth-switch">Already registered? <Link to="/login">Sign in</Link></p>
    </AuthShell>
  );
}
```

client/src/pages/RequestDetail.jsx

```
import { useCallback, useEffect, useState } from 'react';
import { Link, useNavigate, useParams } from 'react-router-dom';
import api, { errorMessage } from '../api';
import { useAuth } from '../context/AuthContext';
import { useToast } from '../context/ToastContext';
import { useRequestUpdates } from '../context/SocketContext';
import StatusBadge, { PriorityTag, STATUS_LABELS } from '../components/StatusBadge';

const STEPS = ['pending', 'assigned', 'in_progress', 'completed'];
const when = (d) => new Date(d).toLocaleString('en-GB', { day: '2-digit', month: 'short', year: 'numeric', hour: '2-digit', minute: '2-digit' });

// Tracking page: progress bar, details, evidence, timeline and role-specific actions
export default function RequestDetail() {
  const { id } = useParams();
  const { user, role } = useAuth();
  const toast = useToast();
  const navigate = useNavigate();
  const [req, setReq] = useState(null);
  const [error, setError] = useState('');
  const [officers, setOfficers] = useState([]);
  const [officerId, setOfficerId] = useState('');
  const [comment, setComment] = useState('');
  const [busy, setBusy] = useState(false);

  const load = useCallback(() => {
    api.get(`/requests/${id}`).then((r) => setReq(r.data.data)).catch((e) => setError(errorMessage(e)));
  }, [id]);

  useEffect(load, [load]);
  useRequestUpdates(useCallback((evt) => evt.id === id && load(), [id, load]));

  useEffect(() => {
    if (role === 'admin') api.get('/users/officers').then((r) => setOfficers(r.data.data));
  }, [role]);

  const act = async (fn, success) => {
    setBusy(true);
    try {
      await fn();
      toast(success);
      setComment('');
      load();
    } catch (e) {
      toast(errorMessage(e), 'error');
    } finally {
      setBusy(false);
    }
  };

  const setStatus = (status) =>
    act(() => api.patch(`/requests/${id}/status`, { status, comment }), `Marked as ${STATUS_LABELS[status].toLowerCase()}`);

  const assign = () => {
    if (!officerId) return toast('Choose an officer first', 'error');
    return act(() => api.patch(`/requests/${id}/assign`, { officerId, note: comment }), 'Officer assigned');
  };

  const remove = async () => {
    if (!window.confirm('Delete this request permanently?')) return;
    await act(() => api.delete(`/requests/${id}`), 'Request deleted');
    navigate('/requests');
  };

  if (error) return <div className="card card-pad"><div className="alert alert-error">{error}</div><Link to="/requests">← Back to requests</Link></div>;
  if (!req) return <div className="page-loading">Loading request…</div>;

  const isOwner = req.requester._id === user._id;
  const isAssignee = req.assignedTo?._id === user._id;
  const closed = ['completed', 'cancelled'].includes(req.status);
  const stepIndex = STEPS.indexOf(req.status);

  return (
```

```
 73      <>
 74        <Link to="/requests" className="back">← Back</Link>
 75        <div className="page-head">
 76          <div>
 77            <span className="ticket big">{req.ticketNo}</span>
 78            <h1>{req.title}</h1>
 79            <p className="muted">{req.category?.name} · {req.location}</p>
 80          </div>
 81          <div className="head-badges"><PriorityTag priority={req.priority} /><StatusBadge status={req.status} /
    ↪ ></div>
 82        </div>
 83
 84        {req.status !== 'cancelled' && (
 85          <ol className="progress" aria-label="Request progress">
 86            {STEPS.map((s, i) => (
 87              <li key={s} className={i < stepIndex || req.status === 'completed' ? 'done' : i === stepIndex ? 'c
    ↪ urrent' : ''}>
 88                <span className="dot">{i < stepIndex || req.status === 'completed' ? '✓' : i + 1}</span>
 89                {STATUS_LABELS[s]}
 90              </li>
 91            ))}
 92          </ol>
 93        )}
 94
 95        <div className="detail-grid">
 96          <div className="stack">
 97            <section className="card card-pad">
 98              <h2>Description</h2>
 99              <p className="prewrap">{req.description}</p>
100              {req.attachments?.length > 0 && (
101                <>
102                  <h3>Evidence</h3>
103                  <div className="thumbs">
104                    {req.attachments.map((a) => (
105                      <a key={a.url} href={a.url} target="_blank" rel="noreferrer" className="thumb">
106                        {a.mimetype?.startsWith('image/') ? <img src={a.url} alt={a.filename} /> : <span>PDF</sp
    ↪ an>}
107                        <small>{a.filename}</small>
108                      </a>
109                    ))}
110                  </div>
111                </>
112              )}
113            </section>
114
115            <section className="card card-pad">
116              <h2>Activity timeline</h2>
117              <ul className="timeline">
118                {req.timeline.map((t) => (
119                  <li key={t._id}>
120                    <div className="tl-dot" />
121                    <div>
122                      <strong>{t.fromStatus ? `${STATUS_LABELS[t.fromStatus]} → ` : ''}{STATUS_LABELS[t.toStatus
    ↪ ]}</strong>
123                      {t.comment && <p>{t.comment}</p>}
124                      <small className="muted">{t.changedBy?.name} · {when(t.createdAt)}</small>
125                    </div>
126                  </li>
127                ))}
128              </ul>
129            </section>
130          </div>
131
132          <div className="stack">
133            <section className="card card-pad">
134              <h2>Details</h2>
135              <dl className="meta">
136                <dt>Requested by</dt><dd>{req.requester.name}<small>{req.requester.department}</small></dd>
137                <dt>Submitted</dt><dd>{when(req.createdAt)}</dd>
138                <dt>Assigned officer</dt><dd>{req.assignedTo ? <>{req.assignedTo.name}<small>{req.assignedTo.pho
    ↪ ne || req.assignedTo.email}</small></> : <span className="muted">Not yet assigned</span>}</dd>
139                {req.completedAt && (<><dt>Completed</dt><dd>{when(req.completedAt)}</dd></>)}
140              </dl>
141            </section>
142
143            {/* ----- Actions depend on role ----- */}
144            {role === 'officer' && isAssignee && !closed && (
145              <section className="card card-pad action-card">
146                <h2>Update progress</h2>
147                <textarea rows={3} placeholder="Add a note for the requester (optional)" value={comment} onChang
    ↪ e={(e) => setComment(e.target.value)} />
148                <div className="btn-row">
149                  {req.status === 'assigned' && <button type="button" className="btn btn-primary" disabled={busy
```

```
    ↪ } onClick={() => setStatus('in_progress')}>Start job</button>}
  150                 <button type="button" className="btn btn-success" disabled={busy} onClick={() => setStatus('co
    ↪ mpleted')}>Mark completed</button>
  151               </div>
  152             </section>
  153           )}
  154
  155           {role === 'admin' && (
  156             <section className="card card-pad action-card">
  157               <h2>Manage request</h2>
  158               {!closed && (
  159                 <>
  160                   <label className="field">
  161                     <span>{req.assignedTo ? 're-assign to' : 'Assign to officer'}</span>
  162                     <select value={officerId} onChange={(e) => setOfficerId(e.target.value)}>
  163                       <option value="">Select maintenance officer…</option>
  164                       {officers.map((o) => <option key={o._id} value={o._id}>{o.name} — {o.department} ({o.ope
    ↪ nJobs} open)</option>)}
  165                     </select>
  166                   </label>
  167                   <textarea rows={2} placeholder="Instruction / note (optional)" value={comment} onChange={(e)
    ↪  => setComment(e.target.value)} />
  168                   <button type="button" className="btn btn-primary btn-block" disabled={busy} onClick={assign}
    ↪ >Assign</button>
  169                 </>
  170               )}
  171               <div className="field">
  172                 <span>Override status</span>
  173                 <select value="" onChange={(e) => e.target.value && setStatus(e.target.value)}>
  174                   <option value="">Change status…</option>
  175                   {Object.entries(STATUS_LABELS).filter(([k]) => k !== req.status).map(([k, v]) => <option key
    ↪ ={k} value={k}>{v}</option>)}
  176                 </select>
  177               </div>
  178               <button type="button" className="btn btn-danger-ghost btn-block" disabled={busy} onClick={remove
    ↪ }>Delete request</button>
  179             </section>
  180           )}
  181
  182           {role === 'user' && isOwner && req.status === 'pending' && (
  183             <section className="card card-pad action-card">
  184               <h2>Changed your mind?</h2>
  185               <p className="muted">You can cancel while the request is still pending.</p>
  186               <button type="button" className="btn btn-danger-ghost btn-block" disabled={busy} onClick={() =>
    ↪ setStatus('cancelled')}>Cancel request</button>
  187             </section>
  188           )}
  189         </div>
  190       </div>
  191     </>
  192   );
  193 }
```

client/src/pages/Requests.jsx

```
   1 import { useCallback, useEffect, useState } from 'react';
   2 import api, { errorMessage } from '../api';
   3 import { useAuth } from '../context/AuthContext';
   4 import { useToast } from '../context/ToastContext';
   5 import { useRequestUpdates } from '../context/SocketContext';
   6 import RequestTable from '../components/RequestTable';
   7 import Pagination from '../components/Pagination';
   8 import { STATUS_LABELS } from '../components/StatusBadge';
   9
  10 const TITLES = { user: 'My requests', officer: 'Assigned jobs', admin: 'All requests' };
  11
  12 // Search + filter + pagination over the requests the current role can see
  13 export default function Requests() {
  14   const { role } = useAuth();
  15   const toast = useToast();
  16   const [filters, setFilters] = useState({ q: '', status: '', category: '', priority: '' });
  17   const [search, setSearch] = useState('');
  18   const [page, setPage] = useState(1);
  19   const [result, setResult] = useState({ data: [], pagination: null });
  20   const [categories, setCategories] = useState([]);
  21   const [loading, setLoading] = useState(true);
  22
  23   useEffect(() => {
  24     api.get('/categories').then((r) => setCategories(r.data.data));
  25   }, []);
  26
  27   const load = useCallback(() => {
  28     setLoading(true);
```

```
    const params = { page, limit: 8, ...Object.fromEntries(Object.entries(filters).filter(([, v]) => v)) };
    api.get('/requests', { params })
      .then((r) => setResult(r.data))
      .catch((e) => toast(errorMessage(e), 'error'))
      .finally(() => setLoading(false));
  }, [page, filters, toast]);

  useEffect(load, [load]);
  useRequestUpdates(load);

  // Debounce the search box so we don't call the API on every keystroke
  useEffect(() => {
    const t = setTimeout(() => {
      setPage(1);
      setFilters((f) => ({ ...f, q: search }));
    }, 350);
    return () => clearTimeout(t);
  }, [search]);

  const setFilter = (e) => {
    setPage(1);
    setFilters({ ...filters, [e.target.name]: e.target.value });
  };

  const exportCsv = async () => {
    try {
      const params = Object.fromEntries(Object.entries(filters).filter(([, v]) => v));
      const res = await api.get('/reports/export', { params, responseType: 'blob' });
      const url = URL.createObjectURL(res.data);
      const a = document.createElement('a');
      a.href = url;
      a.download = `service-requests-${new Date().toISOString().slice(0, 10)}.csv`;
      a.click();
      URL.revokeObjectURL(url);
      toast('CSV exported');
    } catch (e) {
      toast(errorMessage(e), 'error');
    }
  };

  return (
    <>
      <div className="page-head">
        <div>
          <h1>{TITLES[role]}</h1>
          <p className="muted">{result.pagination ? `${result.pagination.total} request(s)` : ' '}</p>
        </div>
        {role === 'admin' && <button type="button" className="btn btn-ghost" onClick={exportCsv}>↓ Export CSV</button>}
      </div>

      <section className="card">
        <div className="filters">
          <input className="search" type="search" placeholder="Search ticket, title or location…" value={search} onChange={(e) => setSearch(e.target.value)} aria-label="Search" />
          <select name="status" value={filters.status} onChange={setFilter} aria-label="Status">
            <option value="">All statuses</option>
            {Object.entries(STATUS_LABELS).map(([k, v]) => <option key={k} value={k}>{v}</option>)}
          </select>
          <select name="category" value={filters.category} onChange={setFilter} aria-label="Category">
            <option value="">All categories</option>
            {categories.map((c) => <option key={c._id} value={c._id}>{c.name}</option>)}
          </select>
          <select name="priority" value={filters.priority} onChange={setFilter} aria-label="Priority">
            <option value="">Any priority</option>
            {['low', 'medium', 'high', 'urgent'].map((p) => <option key={p} value={p}>{p}</option>)}
          </select>
        </div>
        <div className={loading ? 'loading-fade' : ''}>
          <RequestTable items={result.data} showRequester={role !== 'user'} showOfficer={role === 'admin'} empty="No requests match your search." />
        </div>
        <Pagination pagination={result.pagination} onChange={setPage} />
      </section>
    </>
  );
}
```

client/src/pages/admin/Categories.jsx

```jsx
import { useCallback, useEffect, useState } from 'react';
import api, { errorMessage } from '../../api';
import { useToast } from '../../context/ToastContext';

export default function Categories() {
  const toast = useToast();
  const [items, setItems] = useState([]);
  const [form, setForm] = useState({ name: '', description: '' });

  const load = useCallback(() => {
    api.get('/categories').then((r) => setItems(r.data.data));
  }, []);
  useEffect(load, [load]);

  const run = async (fn, msg) => {
    try {
      await fn();
      toast(msg);
      load();
    } catch (e) {
      toast(errorMessage(e), 'error');
    }
  };

  const add = (e) => {
    e.preventDefault();
    if (!form.name.trim()) return toast('Category name is required', 'error');
    return run(async () => {
      await api.post('/categories', form);
      setForm({ name: '', description: '' });
    }, 'Category added');
  };

  return (
    <>
      <div className="page-head"><div><h1>Request categories</h1><p className="muted">The types of problem users can report.</p></div></div>
      <section className="card card-pad">
        <form className="inline-form" onSubmit={add} noValidate>
          <input placeholder="Category name" value={form.name} onChange={(e) => setForm({ ...form, name: e.target.value })} />
          <input placeholder="Short description" value={form.description} onChange={(e) => setForm({ ...form, description: e.target.value })} />
          <button type="submit" className="btn btn-primary">Add category</button>
        </form>
      </section>
      <section className="card">
        <div className="table-wrap">
          <table className="table">
            <thead><tr><th>Name</th><th>Description</th><th>Status</th><th /></tr></thead>
            <tbody>
              {items.map((c) => (
                <tr key={c._id}>
                  <td><strong>{c.name}</strong></td>
                  <td>{c.description}</td>
                  <td><span className={`badge ${c.isActive ? 'badge-completed' : 'badge-cancelled'}`}>{c.isActive ? 'Active' : 'Hidden'}</span></td>
                  <td className="right nowrap">
                    <button type="button" className="btn btn-ghost btn-sm" onClick={() => run(() => api.put(`/categories/${c._id}`, { isActive: !c.isActive }), 'Category updated')}>
                      {c.isActive ? 'Hide' : 'Show'}
                    </button>
                    <button type="button" className="btn btn-danger-ghost btn-sm" onClick={() => run(() => api.delete(`/categories/${c._id}`), 'Category deleted')}>Delete</button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </section>
    </>
  );
}
```

client/src/pages/admin/Reports.jsx

```
import { useEffect, useState } from 'react';
import api, { errorMessage } from '../../api';
import { useToast } from '../../context/ToastContext';
import Pagination from '../../components/Pagination';
import { STATUS_LABELS } from '../../components/StatusBadge';

const when = (d) => new Date(d).toLocaleString('en-GB', { day: '2-digit', month: 'short', hour: '2-digit', minute: '2-digit' });

export default function Reports() {
  const toast = useToast();
  const [summary, setSummary] = useState(null);
  const [audit, setAudit] = useState({ data: [], pagination: null });
  const [page, setPage] = useState(1);

  useEffect(() => {
    api.get('/reports/summary').then((r) => setSummary(r.data.data));
  }, []);
  useEffect(() => {
    api.get('/reports/audit', { params: { page } }).then((r) => setAudit(r.data));
  }, [page]);

  const exportCsv = async () => {
    try {
      const res = await api.get('/reports/export', { responseType: 'blob' });
      const a = document.createElement('a');
      a.href = URL.createObjectURL(res.data);
      a.download = `campusfix-report-${new Date().toISOString().slice(0, 10)}.csv`;
      a.click();
      toast('Report exported');
    } catch (e) {
      toast(errorMessage(e), 'error');
    }
  };

  if (!summary) return <div className="page-loading">Loading reports…</div>;
  const maxCat = Math.max(1, ...summary.byCategory.map((c) => c.count));
  const resolved = summary.total ? Math.round((summary.byStatus.completed / summary.total) * 100) : 0;

  return (
    <>
      <div className="page-head">
        <div><h1>Reports &amp;amp; audit trail</h1><p className="muted">Performance of the maintenance service at a glance.</p></div>
        <button type="button" className="btn btn-primary" onClick={exportCsv}>↓ Export all (CSV)</button>
      </div>

      <div className="stats">
        <div className="stat"><span className="stat-value">{summary.total}</span><span className="stat-label">Total requests</span></div>
        <div className="stat stat-completed"><span className="stat-value">{resolved}%</span><span className="stat-label">Resolution rate</span></div>
        <div className="stat stat-in_progress"><span className="stat-value">{summary.avgResolutionHours ?? '—'}h</span><span className="stat-label">Avg. time to fix</span></div>
        <div className="stat stat-pending"><span className="stat-value">{summary.byStatus.pending}</span><span className="stat-label">Awaiting assignment</span></div>
      </div>

      <div className="detail-grid even">
        <section className="card card-pad">
          <h2>Requests by category</h2>
          <div className="bars">
            {summary.byCategory.map((c) => (
              <div key={c.name} className="bar-row">
                <span>{c.name}</span>
                <div className="bar"><div style={{ width: `${(c.count / maxCat) * 100}%` }} /></div>
                <strong>{c.count}</strong>
              </div>
            ))}
          </div>
          <h3>By status</h3>
          <div className="status-strip">
            {Object.entries(summary.byStatus).map(([k, v]) => (
              <div key={k} className={`badge badge-${k}`}>{STATUS_LABELS[k]}: {v}</div>
            ))}
          </div>
        </section>
        <section className="card card-pad">
          <h2>Officer workload</h2>
          <table className="table compact">
            <thead><tr><th>Officer</th><th>Open jobs</th><th>Completed</th></tr></thead>
```

```
            <tbody>
              {summary.byOfficer.map((o) => (
                <tr key={o.name}><td>{o.name}</td><td>{o.open}</td><td>{o.completed}</td></tr>
              ))}
            </tbody>
          </table>
        </section>
      </div>

      <section className="card">
        <div className="card-head"><h2>Activity log</h2></div>
        <div className="table-wrap">
          <table className="table">
            <thead><tr><th>When</th><th>User</th><th>Action</th><th>Details</th><th>IP</th></tr></thead>
            <tbody>
              {audit.data.map((a) => (
                <tr key={a._id}>
                  <td className="nowrap">{when(a.createdAt)}</td>
                  <td>{a.actor?.name || '—'}</td>
                  <td><code className="action">{a.action}</code></td>
                  <td>{a.details}</td>
                  <td className="muted">{a.ip || '—'}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
        <Pagination pagination={audit.pagination} onChange={setPage} />
      </section>
    </>
  );
}
```

client/src/pages/admin/Users.jsx

```
import { useCallback, useEffect, useState } from 'react';
import api, { errorMessage } from '../../api';
import { useAuth } from '../../context/AuthContext';
import { useToast } from '../../context/ToastContext';
import Pagination from '../../components/Pagination';

const ROLE_LABEL = { user: 'Student/Staff', officer: 'Maintenance Officer', admin: 'Administrator' };
const EMPTY = { name: '', email: '', password: '', role: 'officer', department: '', phone: '' };

export default function Users() {
  const { user: me } = useAuth();
  const toast = useToast();
  const [q, setQ] = useState('');
  const [roleFilter, setRoleFilter] = useState('');
  const [page, setPage] = useState(1);
  const [result, setResult] = useState({ data: [], pagination: null });
  const [showForm, setShowForm] = useState(false);
  const [form, setForm] = useState(EMPTY);
  const [formError, setFormError] = useState('');

  const load = useCallback(() => {
    api.get('/users', { params: { page, limit: 8, q: q || undefined, role: roleFilter || undefined } })
      .then((r) => setResult(r.data));
  }, [page, q, roleFilter]);

  useEffect(load, [load]);

  const update = async (u, changes, msg) => {
    try {
      await api.put(`/users/${u._id}`, changes);
      toast(msg);
      load();
    } catch (e) {
      toast(errorMessage(e), 'error');
    }
  };

  const remove = async (u) => {
    if (!window.confirm(`Delete ${u.name}?`)) return;
    try {
      await api.delete(`/users/${u._id}`);
      toast('User deleted');
      load();
    } catch (e) {
      toast(errorMessage(e), 'error');
    }
  };

```

```
  const create = async (e) => {
    e.preventDefault();
    setFormError('');
    if (!form.name || !form.email || form.password.length < 8) {
      setFormError('Name, email and a password of at least 8 characters are required');
      return;
    }
    try {
      await api.post('/users', form);
      toast(`${ROLE_LABEL[form.role]} account created`);
      setForm(EMPTY);
      setShowForm(false);
      load();
    } catch (err) {
      setFormError(errorMessage(err));
    }
  };

  const ch = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  return (
    <>
      <div className="page-head">
        <div>
          <h1>User management</h1>
          <p className="muted">Create officer accounts, change roles and deactivate users.</p>
        </div>
        <button type="button" className="btn btn-primary" onClick={() => setShowForm(!showForm)}>{showForm ? '
↪ Close' : '+ Add user'}</button>
      </div>

      {showForm && (
        <section className="card card-pad">
          <h2>New user</h2>
          {formError && <div className="alert alert-error">{formError}</div>}
          <form className="form-grid cols-3" onSubmit={create} noValidate>
            <label className="field"><span>Full name</span><input name="name" value={form.name} onChange={ch}
↪ /></label>
            <label className="field"><span>Email</span><input name="email" type="email" value={form.email} onC
↪ hange={ch} /></label>
            <label className="field"><span>Temporary password</span><input name="password" type="password" val
↪ ue={form.password} onChange={ch} /></label>
            <label className="field"><span>Role</span>
              <select name="role" value={form.role} onChange={ch}>
                {Object.entries(ROLE_LABEL).map(([k, v]) => <option key={k} value={k}>{v}</option>)}
              </select>
            </label>
            <label className="field"><span>Department / Unit</span><input name="department" value={form.depart
↪ ment} onChange={ch} /></label>
            <label className="field"><span>Phone</span><input name="phone" value={form.phone} onChange={ch} />
↪ </label>
            <div className="span-3 form-actions"><button type="submit" className="btn btn-primary">Create acco
↪ unt</button></div>
          </form>
        </section>
      )}

      <section className="card">
        <div className="filters">
          <input name="search" type="search" placeholder="Search name, email or department…" value={q} on
↪ Change={(e) => { setQ(e.target.value); setPage(1); }} />
          <select value={roleFilter} onChange={(e) => { setRoleFilter(e.target.value); setPage(1); }}>
            <option value="">All roles</option>
            {Object.entries(ROLE_LABEL).map(([k, v]) => <option key={k} value={k}>{v}</option>)}
          </select>
        </div>
        <div className="table-wrap">
          <table className="table">
            <thead><tr><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th /></tr></
↪ thead>
            <tbody>
              {result.data.map((u) => (
                <tr key={u._id}>
                  <td><strong>{u.name}</strong></td>
                  <td>{u.email}</td>
                  <td>{u.department || '—'}</td>
                  <td>
                    <select className="inline-select" value={u.role.name} disabled={u._id === me._id}
                      onChange={(e) => update(u, { role: e.target.value }, 'Role updated')}>
                      {Object.entries(ROLE_LABEL).map(([k, v]) => <option key={k} value={k}>{v}</option>)}
                    </select>
                  </td>
                  <td><span className={`badge ${u.isActive ? 'badge-completed' : 'badge-cancelled'}`}>{u.isAct
```

```
    ↪ ive ? 'Active' : 'Inactive'}</span></td>
  123                   <td className="nowrap right">
  124                     {u._id !== me._id && (
  125                       <>
  126                         <button type="button" className="btn btn-ghost btn-sm" onClick={() => update(u, { isAc
    ↪ tive: !u.isActive }, u.isActive ? 'User deactivated' : 'User activated')}>
  127                           {u.isActive ? 'Deactivate' : 'Activate'}
  128                         </button>
  129                         <button type="button" className="btn btn-danger-ghost btn-sm" onClick={() => remove(u)
    ↪ }>Delete</button>
  130                       </>
  131                     )}
  132                   </td>
  133                 </tr>
  134               ))}
  135             </tbody>
  136           </table>
  137         </div>
  138         <Pagination pagination={result.pagination} onChange={setPage} />
  139       </section>
  140     </>
  141   );
  142 }
```

client/src/styles.css

```
    1 /* ---------- Design tokens ---------- */
    2 :root {
    3   --navy: #14365c;
    4   --navy-2: #0e2744;
    5   --amber: #f2b84b;
    6   --bg: #f4f5f2;
    7   --surface: #ffffff;
    8   --ink: #18212b;
    9   --ink-2: #4a5663;
   10   --muted: #7a8591;
   11   --line: #e3e6e1;
   12   --primary: #1d5fa6;
   13   --primary-dark: #154a83;
   14   --success: #1f8a5b;
   15   --danger: #c2412d;
   16   --radius: 10px;
   17   --shadow: 0 1px 2px rgba(20, 30, 40, 0.05), 0 2px 8px rgba(20, 30, 40, 0.04);
   18   font-family: 'Plus Jakarta Sans', system-ui, -apple-system, 'Segoe UI', sans-serif;
   19   color: var(--ink);
   20   background: var(--bg);
   21   font-size: 15px;
   22   line-height: 1.5;
   23 }
   24 * { box-sizing: border-box; }
   25 body { margin: 0; }
   26 h1, h2, h3 { margin: 0; line-height: 1.25; }
   27 h1 { font-size: 1.6rem; font-weight: 700; letter-spacing: -0.01em; }
   28 h2 { font-size: 1.05rem; font-weight: 700; margin-bottom: 0.8rem; }
   29 h3 { font-size: 0.9rem; font-weight: 700; margin: 1.2rem 0 0.6rem; color: var(--ink-2); }
   30 a { color: var(--primary); text-decoration: none; }
   31 a:hover { text-decoration: underline; }
   32 .muted { color: var(--muted); }
   33 .block { display: block; }
   34 .nowrap { white-space: nowrap; }
   35 .right { text-align: right; }
   36 .prewrap { white-space: pre-wrap; margin: 0; color: var(--ink-2); }
   37
   38 /* ---------- Buttons & inputs ---------- */
   39 .btn {
   40   display: inline-flex; align-items: center; justify-content: center; gap: 0.4rem;
   41   font: inherit; font-weight: 600; font-size: 0.9rem;
   42   padding: 0.6rem 1.1rem; border-radius: 8px; border: 1px solid transparent;
   43   cursor: pointer; text-decoration: none !important; transition: background 0.15s, border-color 0.15s;
   44 }
   45 .btn:disabled { opacity: 0.6; cursor: not-allowed; }
   46 .btn-primary { background: var(--primary); color: #fff; }
   47 .btn-primary:hover:not(:disabled) { background: var(--primary-dark); }
   48 .btn-success { background: var(--success); color: #fff; }
   49 .btn-ghost { background: var(--surface); border-color: var(--line); color: var(--ink); }
   50 .btn-ghost:hover:not(:disabled) { border-color: #c8cdc6; background: #fafbf9; }
   51 .btn-danger-ghost { background: transparent; border-color: #efcbc4; color: var(--danger); }
   52 .btn-danger-ghost:hover:not(:disabled) { background: #fdf3f1; }
   53 .btn-sm { padding: 0.35rem 0.7rem; font-size: 0.82rem; }
   54 .btn-block { width: 100%; }
   55 .btn-row { display: flex; gap: 0.6rem; flex-wrap: wrap; }
   56 .link-btn { background: none; border: 0; color: var(--primary); font: inherit; font-size: 0.82rem; cursor: poi
    ↪ nter; }
```

```
.icon-btn { position: relative; background: none; border: 0; cursor: pointer; color: var(--ink-2); padding: 0.4rem; border-radius: 8px; font-size: 1.1rem; }
.icon-btn:hover { background: #eef0ec; }

input, select, textarea {
  font: inherit; font-size: 0.92rem; color: var(--ink);
  width: 100%; padding: 0.6rem 0.75rem;
  border: 1px solid #d5d9d3; border-radius: 8px; background: #fff;
}
input:focus, select:focus, textarea:focus { outline: none; border-color: var(--primary); box-shadow: 0 0 0 3px rgba(29, 95, 166, 0.15); }
[aria-invalid='true'] { border-color: var(--danger); }
textarea { resize: vertical; }
.field { display: flex; flex-direction: column; gap: 0.35rem; margin-bottom: 1rem; }
.field > span { font-size: 0.82rem; font-weight: 600; color: var(--ink-2); }
.field-error { color: var(--danger); font-size: 0.8rem; font-style: normal; }
.row-2 { display: grid; grid-template-columns: 1fr 1fr; gap: 0 0.9rem; }
.form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 0 1rem; }
.form-grid.cols-3 { grid-template-columns: repeat(3, 1fr); }
.span-2 { grid-column: span 2; }
.span-3 { grid-column: span 3; }
.form-actions { display: flex; justify-content: flex-end; }

.alert { padding: 0.7rem 0.9rem; border-radius: 8px; font-size: 0.88rem; margin-bottom: 1rem; }
.alert-error { background: #fdf0ed; color: #962f1f; border: 1px solid #f3cfc7; }
.alert-info { background: #eef4fb; color: #1c4a7a; border: 1px solid #cfe0f2; }

/* ---------- Auth pages ---------- */
.auth { min-height: 100vh; display: grid; grid-template-columns: 1.05fr 1fr; }
.auth-panel {
  background: var(--navy); color: #dbe5f0; padding: 2.5rem 3rem;
  display: flex; flex-direction: column; justify-content: space-between;
  background-image: radial-gradient(circle at 85% 15%, rgba(242, 184, 75, 0.14), transparent 45%),
    linear-gradient(160deg, var(--navy) 0%, var(--navy-2) 100%);
}
.auth-pitch h1 { color: #fff; font-size: 2.1rem; max-width: 15ch; margin-bottom: 1rem; }
.auth-pitch p { max-width: 46ch; color: #b9c8d8; }
.auth-pitch ul { list-style: none; padding: 0; margin: 1.5rem 0 0; }
.auth-pitch li { padding: 0.35rem 0 0.35rem 1.6rem; position: relative; }
.auth-pitch li::before { content: ''; position: absolute; left: 0; top: 0.75rem; width: 0.7rem; height: 0.7rem; border-radius: 3px; background: var(--amber); }
.auth-panel small { color: #8aa0b8; }
.auth-form { display: flex; align-items: center; justify-content: center; padding: 2rem; }
.auth-card { width: 100%; max-width: 440px; }
.auth-card h2 { font-size: 1.5rem; margin-bottom: 0.25rem; }
.auth-card > .muted { margin: 0 0 1.5rem; }
.auth-switch { text-align: center; font-size: 0.9rem; color: var(--ink-2); margin-top: 1.25rem; }

.brand { display: flex; align-items: center; gap: 0.6rem; font-weight: 700; font-size: 1.15rem; color: var(--ink); text-decoration: none !important; }
.brand-light { color: #fff; }

/* ---------- App shell ---------- */
.shell { display: grid; grid-template-columns: 240px 1fr; min-height: 100vh; background: linear-gradient(to right, var(--navy) 240px, var(--bg) 240px); }
.sidebar { background: var(--navy); color: #c9d6e4; padding: 1.25rem 0.9rem; display: flex; flex-direction: column; position: sticky; top: 0; height: 100vh; }
.sidebar .brand { color: #fff; padding: 0 0.5rem 1.5rem; }
.sidebar nav { display: flex; flex-direction: column; gap: 0.2rem; }
.sidebar nav a { color: #c9d6e4; padding: 0.6rem 0.8rem; border-radius: 8px; font-weight: 600; font-size: 0.9rem; text-decoration: none; }
.sidebar nav a:hover { background: rgba(255, 255, 255, 0.07); color: #fff; }
.sidebar nav a.active { background: rgba(242, 184, 75, 0.16); color: #fff; box-shadow: inset 3px 0 0 var(--amber); }
.sidebar-foot { margin-top: auto; padding: 0.5rem; }
.role-pill { display: inline-block; font-size: 0.75rem; font-weight: 600; padding: 0.25rem 0.6rem; border-radius: 99px; background: rgba(255, 255, 255, 0.1); color: #e4ecf4; }

.main { min-width: 0; display: flex; flex-direction: column; }
.topbar { height: 64px; display: flex; align-items: center; justify-content: space-between; padding: 0 2rem; background: var(--surface); border-bottom: 1px solid var(--line); position: sticky; top: 0; z-index: 5; }
.topbar-right { display: flex; align-items: center; gap: 1rem; margin-left: auto; }
.menu-toggle { display: none; }
.who { display: flex; align-items: center; gap: 0.6rem; }
.who strong { display: block; font-size: 0.88rem; }
.who small { display: block; color: var(--muted); font-size: 0.76rem; }
.avatar { width: 34px; height: 34px; border-radius: 50%; background: #e3ecf6; color: var(--navy); font-weight: 700; font-size: 0.8rem; display: grid; place-items: center; }
.content { padding: 1.75rem 2rem 3rem; max-width: 1240px; width: 100%; }
.page-loading { padding: 3rem; text-align: center; color: var(--muted); }

/* Notifications */
.bell { position: relative; }
```

```
.bell-count { position: absolute; top: 0; right: 0; background: var(--danger); color: #fff; font-size: 0.65rem; font-weight: 700; min-width: 17px; height: 17px; border-radius: 9px; display: grid; place-items: center; padding: 0 4px; }
.bell-menu { position: absolute; right: 0; top: 44px; width: 340px; background: #fff; border: 1px solid var(--line); border-radius: var(--radius); box-shadow: 0 10px 30px rgba(20, 30, 40, 0.12); overflow: hidden; z-index: 20; }
.bell-head { display: flex; justify-content: space-between; align-items: center; padding: 0.75rem 1rem; border-bottom: 1px solid var(--line); }
.bell-item { display: block; width: 100%; text-align: left; background: #fff; border: 0; border-bottom: 1px solid #f0f1ee; padding: 0.7rem 1rem; font: inherit; font-size: 0.86rem; cursor: pointer; }
.bell-item:hover { background: #f8f9f7; }
.bell-item.unread { background: #f3f8fd; box-shadow: inset 3px 0 0 var(--primary); }
.bell-item small { display: block; color: var(--muted); font-size: 0.75rem; margin-top: 2px; }
.pad { padding: 1rem; margin: 0; }

/* ---------- Page pieces ---------- */
.page-head { display: flex; justify-content: space-between; align-items: flex-end; gap: 1rem; margin-bottom: 1.5rem; flex-wrap: wrap; }
.page-head p { margin: 0.3rem 0 0; }
.head-badges { display: flex; gap: 0.5rem; align-items: center; }
.back { display: inline-block; margin-bottom: 0.75rem; font-size: 0.88rem; font-weight: 600; }
.card { background: var(--surface); border: 1px solid var(--line); border-radius: var(--radius); box-shadow: var(--shadow); margin-bottom: 1.25rem; }
.card-pad { padding: 1.4rem; }
.card-head { display: flex; justify-content: space-between; align-items: center; padding: 1.1rem 1.4rem 0.3rem; }
.card-head h2 { margin: 0; }
.narrow { max-width: 820px; }
.stack { display: flex; flex-direction: column; }
.empty { padding: 2.5rem 1rem; text-align: center; color: var(--muted); }

.stats { display: grid; grid-template-columns: repeat(4, 1fr); gap: 1rem; margin-bottom: 1.25rem; }
.stat { background: var(--surface); border: 1px solid var(--line); border-radius: var(--radius); padding: 1.1rem 1.25rem; display: flex; flex-direction: column; border-top: 3px solid var(--navy); box-shadow: var(--shadow); }
.stat-value { font-size: 1.9rem; font-weight: 700; letter-spacing: -0.02em; }
.stat-label { color: var(--muted); font-size: 0.85rem; font-weight: 600; }
.stat-pending { border-top-color: #d99a1e; }
.stat-assigned { border-top-color: #6b5bd2; }
.stat-in_progress { border-top-color: var(--primary); }
.stat-completed { border-top-color: var(--success); }

/* Tables */
.table-wrap { overflow-x: auto; }
.table { width: 100%; border-collapse: collapse; font-size: 0.88rem; }
.table th { text-align: left; font-size: 0.74rem; text-transform: uppercase; letter-spacing: 0.04em; color: var(--muted); font-weight: 700; padding: 0.75rem 1.4rem; border-bottom: 1px solid var(--line); background: #fafbf9; }
.table td { padding: 0.8rem 1.4rem; border-bottom: 1px solid #eef0ec; vertical-align: middle; }
.table tr:last-child td { border-bottom: 0; }
.table tbody tr:hover { background: #fbfcfa; }
.table.compact th, .table.compact td { padding: 0.55rem 0.5rem; }
.ticket { white-space: nowrap; font-family: ui-monospace, 'SF Mono', Menlo, monospace; font-size: 0.8rem; font-weight: 600; color: var(--navy); }
.ticket.big { font-size: 0.85rem; color: var(--muted); }
.cell-title { color: var(--ink); font-weight: 600; }
.inline-select { width: auto; padding: 0.3rem 0.5rem; font-size: 0.84rem; }
code.action { font-size: 0.75rem; background: #f0f2ee; padding: 2px 6px; border-radius: 4px; color: var(--ink-2); }

.filters { display: flex; gap: 0.6rem; padding: 1rem 1.4rem; border-bottom: 1px solid var(--line); flex-wrap: wrap; }
.filters .search { flex: 1 1 260px; }
.filters select { width: auto; flex: 0 1 180px; }
.pagination { display: flex; justify-content: space-between; align-items: center; padding: 0.8rem 1.4rem; border-top: 1px solid var(--line); font-size: 0.85rem; }
.pager { display: flex; align-items: center; gap: 0.75rem; }
.loading-fade { opacity: 0.55; transition: opacity 0.2s; }
.inline-form { display: flex; gap: 0.6rem; flex-wrap: wrap; }
.inline-form input { flex: 1 1 200px; }

/* Badges */
.badge { display: inline-block; font-size: 0.75rem; font-weight: 700; padding: 0.22rem 0.6rem; border-radius: 99px; white-space: nowrap; }
.badge-pending { background: #fdf3df; color: #8a5a00; }
.badge-assigned { background: #efecfc; color: #4a3bb0; }
.badge-in_progress { background: #e6f0fa; color: #17508c; }
.badge-completed { background: #e3f4ea; color: #17673f; }
.badge-cancelled { background: #eeefed; color: #6b716a; }
.priority { font-size: 0.75rem; font-weight: 700; text-transform: capitalize; padding-left: 0.85rem; position: relative; color: var(--ink-2); }
.priority::before { content: ''; position: absolute; left: 0; top: 50%; width: 7px; height: 7px; border-radius: 50%; transform: translateY(-50%); background: #9aa39a; }
```

```
.priority-medium::before { background: #3c86c9; }
.priority-high::before { background: #e08a12; }
.priority-urgent { color: var(--danger); }
.priority-urgent::before { background: var(--danger); }
.status-strip { display: flex; gap: 0.4rem; flex-wrap: wrap; }

/* Request detail */
.progress { display: grid; grid-template-columns: repeat(4, 1fr); list-style: none; padding: 1.1rem 1.4rem; margin: 0 0 1.25rem; background: var(--surface); border: 1px solid var(--line); border-radius: var(--radius); counter-reset: s; }
.progress li { display: flex; align-items: center; gap: 0.55rem; font-weight: 600; font-size: 0.88rem; color: var(--muted); position: relative; }
.progress li:not(:last-child)::after { content: ''; flex: 1; height: 2px; background: var(--line); margin: 0 0 0 .75rem; }
.progress li.done::after { background: var(--success); }
.progress .dot { width: 28px; height: 28px; border-radius: 50%; display: grid; place-items: center; background: #eef0ec; font-size: 0.8rem; flex: none; }
.progress li.done { color: var(--ink); }
.progress li.done .dot { background: var(--success); color: #fff; }
.progress li.current { color: var(--primary); }
.progress li.current .dot { background: var(--primary); color: #fff; box-shadow: 0 0 0 4px rgba(29, 95, 166, 0.15); }
.detail-grid { display: grid; grid-template-columns: 1.6fr 1fr; gap: 1.25rem; align-items: start; }
.detail-grid.even { grid-template-columns: 1fr 1fr; }
.meta { display: grid; grid-template-columns: auto 1fr; gap: 0.6rem 1rem; margin: 0; font-size: 0.9rem; }
.meta dt { color: var(--muted); font-weight: 600; }
.meta dd { margin: 0; }
.meta small { display: block; color: var(--muted); font-size: 0.8rem; }
.action-card { border-top: 3px solid var(--amber); }
.action-card textarea { margin-bottom: 0.8rem; }
.action-card .field { margin: 0.4rem 0 0.8rem; }
.timeline { list-style: none; margin: 0; padding: 0; }
.timeline li { display: flex; gap: 0.9rem; padding-bottom: 1.1rem; position: relative; }
.timeline li:not(:last-child)::before { content: ''; position: absolute; left: 5px; top: 14px; bottom: 0; width: 2px; background: var(--line); }
.tl-dot { width: 12px; height: 12px; border-radius: 50%; background: var(--primary); margin-top: 5px; flex: none; box-shadow: 0 0 0 3px #e6f0fa; }
.timeline p { margin: 0.15rem 0; color: var(--ink-2); font-size: 0.9rem; }
.timeline small { font-size: 0.78rem; }
.thumbs { display: flex; gap: 0.6rem; flex-wrap: wrap; margin-top: 0.5rem; }
.thumb { width: 110px; border: 1px solid var(--line); border-radius: 8px; overflow: hidden; background: #fafbf9; text-decoration: none !important; }
.thumb img { width: 100%; height: 80px; object-fit: cover; display: block; }
.thumb > span { height: 80px; display: grid; place-items: center; font-weight: 700; color: var(--danger); }
.thumb small { display: block; font-size: 0.7rem; padding: 0.3rem 0.4rem; color: var(--ink-2); white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }

/* Reports */
.bars { display: flex; flex-direction: column; gap: 0.55rem; }
.bar-row { display: grid; grid-template-columns: 150px 1fr 30px; align-items: center; gap: 0.75rem; font-size: 0.86rem; }
.bar { background: #eef0ec; border-radius: 4px; height: 12px; overflow: hidden; }
.bar > div { height: 100%; background: var(--navy); border-radius: 4px; }
.bar-row strong { text-align: right; }

/* Toasts */
.toasts { position: fixed; bottom: 1.25rem; right: 1.25rem; display: flex; flex-direction: column; gap: 0.5rem; z-index: 50; }
.toast { background: var(--ink); color: #fff; padding: 0.75rem 1rem; border-radius: 8px; font-size: 0.88rem; box-shadow: 0 8px 24px rgba(0, 0, 0, 0.18); max-width: 360px; border-left: 4px solid var(--success); animation: slide 0.2s ease-out; }
.toast-error { border-left-color: #ff7a62; }
.toast-info { border-left-color: var(--amber); }
@keyframes slide { from { transform: translateY(8px); opacity: 0; } }

/* ---------- Responsive ---------- */
@media (max-width: 900px) {
  .auth { grid-template-columns: 1fr; }
  .auth-panel { display: none; }
  .shell { grid-template-columns: 1fr; background: var(--bg); }
  .sidebar { position: fixed; left: -260px; width: 240px; z-index: 30; transition: left 0.2s; }
  .sidebar.open { left: 0; }
  .menu-toggle { display: block; }
  .topbar { padding: 0 1rem; }
  .who div { display: none; }
  .content { padding: 1.25rem 1rem 2rem; }
  .stats { grid-template-columns: 1fr 1fr; }
  .detail-grid, .detail-grid.even, .form-grid, .form-grid.cols-3, .row-2 { grid-template-columns: 1fr; }
  .span-2, .span-3 { grid-column: auto; }
  .progress { grid-template-columns: 1fr 1fr; gap: 0.6rem; }
  .progress li::after { display: none; }
}
```